Chapter 1. Job control statements

This chapter lists, in Table 1 on page 1, all but one of the statements in the MVS™ Job Control Language (JCL), and in Table 2 on page 2, all of the Job Entry Control Language (JECL) statements for the JES2 and JES3 subsystems, together with the purpose of each statement. Later chapters describe each statement in detail. (The PRINTDEV JCL statement, for use by the person starting the Print Services Facility, is documented in *PSF for z/OS: Customization*.)

JCL statements

Table 1. MVS Job Control Language (JCL) Statements

Statement	Name	Purpose
// command	JCL command	Enters an MVS system operator command through the input stream. The command statement is used primarily by the operator. Use the COMMAND statement instead of the JCL command statement.
// COMMAND	command	Specifies an MVS or JES command that the system issues when the JCL is converted. Use the COMMAND statement instead of the JCL command statement.
//* comment	comment	Contains comments. The comment statement is used primarily to document a program and its resource requirements.
// CNTL	control	Marks the beginning of one or more program control statements.
// DD	data definition	Identifies and describes a data set.
/*	delimiter	Indicates the end of data placed in the input stream. **Note:** A user can designate any two characters to be the delimiter.
// ENDCNTL	end control	Marks the end of one or more program control statements.
// EXEC	execute	Marks the beginning of a job step; assigns a name to the step; identifies the program or the cataloged or in-stream procedure to be executed in this step.
// EXPORT	export	Makes specific JCL symbols available to the job step program.
// IF/THEN/ELSE/ENDIF	IF/THEN/ELSE/ ENDIF statement construct	Specifies conditional execution of job steps within a job.

© Copyright IBM Corp. 1988, 2017

Table 1. MVS Job Control Language (JCL) Statements (continued)		
Statement	**Name**	**Purpose**
// INCLUDE	include	Identifies a member of a partitioned data set (PDS) or partitioned data set extended (PDSE) that contains JCL statements to include in the job stream.
// JCLLIB	JCL library	Identifies the libraries that the system searches for: • INCLUDE groups • Procedures named in EXEC statements.
// JOB	job	Marks the beginning of a job; assigns a name to the job.
//	null	Marks the end of a job.
// OUTPUT	output JCL	Specifies the processing options that the job entry subsystem is to use for printing a sysout data set.
// PEND	procedure end	Marks the end of an in-stream or cataloged procedure.
// PROC	procedure	Marks the beginning of an in-stream procedure and may mark the beginning of a cataloged procedure; assigns default values to parameters defined in the procedure.
// SCHEDULE	schedule	Specifies scheduling attributes for a job such as the job group it is associated with and whether the job should be held for a time before execution.
// SET	set	Defines and assigns initial values to symbolic parameters used when processing JCL statements. Changes or nullifies the values assigned to symbolic parameters.
// XMIT	transmit	Transmits input stream records from one node to another.

JECL statements

Table 2. JES2 Job Entry Control Language (JECL) Statements	
JES2 JECL Control Statement	**Purpose**
/*$command	Enters JES2 operator commands through the input stream.
/*JOBPARM	Specifies certain job-related parameters at input time.
/*MESSAGE	Sends messages to the operator via the operator console.
/*NETACCT	Specifies an account number for a network job.

Table 2. JES2 Job Entry Control Language (JECL) Statements (continued)	
JES2 JECL Control Statement	**Purpose**
/*NOTIFY	Specifies the destination of notification messages.
/*OUTPUT	Specifies processing options for sysout data set(s).
/*PRIORITY	Assigns a job queue selection priority.
/*ROUTE	Specifies the output destination or the execution node for the job.
/*SETUP	Requests mounting of volumes needed for the job.
/*SIGNOFF	Ends a remote job stream processing session.
/*SIGNON	Begins a remote job stream processing session.
/*XEQ	Specifies the execution node for a job.
/*XMIT	Indicates a job or data stream to be transmitted to another JES2 node or eligible non-JES2 node.

Table 3. JES3 Job Entry Control Language (JECL) Statements	
JES3 JECL Control Statement	**Purpose**
//**command	Enters JES3 operator commands, except *DUMP and *RETURN, through the input stream.
//*DATASET	Begins an input data set in the input stream.
//*ENDDATASET	Ends the input data set that began with a //*dataset statement.
//*ENDPROCESS	Ends a series of //*PROCESS statements.
//*FORMAT	Specifies the processing options for a sysout or JES3-managed print or punch data set.
//*MAIN	Defines selected processing parameters for a job.
//*NET	Identifies relationships between predecessor and successor jobs in a dependent job control net.
//*NETACCT	Specifies an account number for a network job.
//*OPERATOR	Sends messages to the operator.
//*PAUSE	Halts the input reader.
//*PROCESS	Identifies a nonstandard job.
//*ROUTE	Specifies the execution node for the job.
/*SIGNOFF	Ends a remote job stream processing session.
/*SIGNON	Begins a remote job stream processing session.

Note: JES2 JECL statements (for example /*JOBPARM, /*ROUTE) are now treated as JCL statements. They are given statement numbers visible in the JCL data set and error messages or warning messages associated with them are in the system messages data set. The converter treats these statements as JCL statements starting at z/OS 2.1.

Chapter 3. Format of statements

This topic describes the fields in JCL, JES2, and JES3 statements. It ends with the conventions for continuing statements.

JCL statement fields

A JCL statement consists of one or more 80-byte records. Each record is comparable to an 80-column punched-card image. Each JCL statement is logically divided into the following five fields. All five fields do not appear on every statement; see Table 8 on page 14 for the fields that can appear on each statement.

Identifier field
The identifier field indicates to the system that a statement is a JCL statement rather than data. The identifier field consists of the following:

- Columns 1 and 2 of all JCL statements, except the delimiter statement, contain //
- Columns 1 and 2 of the delimiter statement contain either /* or two other characters that are designated in a DLM parameter to be the delimiter
- Columns 1, 2, and 3 of a JCL comment statement contains //*

Name field
The name field identifies a particular statement so that other statements and the system can refer to it. For JCL statements, code the name as follows:

- The name must begin in column 3.
- The name is 1 through 8 alphanumeric or national ($, #, @) characters. See Table 10 on page 21 for the character sets.
- The first character must be an alphabetic or national ($, #, @).
- The name must be followed by at least one blank.

Operation field
The operation field specifies the type of statement, or, for the command statement, the command. Code the operation field as follows:

- The operation field consists of the characters in the syntax box for the statement.
- The operation follows the name field.
- The operation must be preceded and followed by at least one blank.

Parameter, or operand field
The parameter field, also sometimes referred to as the operand field, contains parameters that are separated by commas. Code the parameter field as follows:

- The parameter field follows the operation field.
- The parameter field must be preceded by at least one blank.

See “Parameter field” on page 15 for details on coding the parameter field.

Comments field
The comments field contains any information that you deem helpful when you code the control statement. Code the comments field as follows:

- The comments field follows the parameter field.
- The comments field must be preceded by at least one blank.

You can code comments after the parameter field even though you continue the parameter field on a subsequent statement; see “Continuing JCL statements” on page 16.

© Copyright IBM Corp. 1988, 2017

For most statements, if you do not code any parameters, do not code any comments.

Table 8. JCL Statement Fields

Statement	**Fields**
JCL Command	// command [parameter] [comments]
COMMAND	//[name] COMMAND 'command command-operand' [comments]
Comment	//* comments
CNTL	//label CNTL [* comments]
DD	//[ddname] DD [parameter [comments]] //[ddname] DD
Delimiter	/* [comments] xx [comments]
ENDCNTL	//[label] ENDCNTL [comments]
EXEC	//[stepname] EXEC parameter [comments]
EXPORT	//[label] EXPORT [comments]

IF/THEN/ELSE/ENDIF	//name IF [relational expression] THEN [comments]
	//name ELSE [comments]
	//name ENDIF [comments]

INCLUDE	//[name] INCLUDE parameter [comments]
JCLLIB	//[name] JCLLIB parameter [comments]
JOB	//jobname JOB [parameter [comments]] //jobname JOB
Null	//
OUTPUT JCL	//name OUTPUT parameter [comments]
PEND	//[name] PEND [comments]
PROC (cataloged)	//[name] PROC [parameter [comments]] //[name] PROC
PROC (in-stream)	//name PROC [parameter [comments]] //name PROC
SCHEDULE	//[name] SCHEDULE parameter [comments]
SET	//[name] SET parameter [comments]
XMIT	//[name] XMIT parameter[,parameter] [comments]

Location of fields on statements: Code the identifier field beginning in column 1 and the name field immediately after the identifier, with no intervening blanks. Code the operation, parameter, and

comments fields in free form. Free form means that the fields do not begin in a particular column. Between fields, leave at least one blank; the blank serves as the delimiter between fields.

Do not code fields, except on the comment statement, past column 71. Columns 73-80 are ignored by z/OS and are typically used for sequence numbers. If the total length of the fields would exceed 71 columns, continue the fields onto one or more following statements. Continuing fields is described under "Continuing JCL statements" on page 16. The comment statement can be coded through column 80.

The maximum length of any JCL statement is limited to 8194 characters.

Use keywords only for parameters or subparameters: Do not use parameter or subparameter keywords from any JCL, JES2, or JES3 statements as symbolic parameters, names, or labels.

Parameter field

The parameter field consists of two types of parameters: **positional parameters** and **keyword parameters**. All positional parameters must precede all keyword parameters. Keyword parameters follow the positional parameters.

Commas: Use commas to separate positional parameters, keyword parameters, and subparameters in the parameter field.

Positional Parameters: A positional parameter consists of:

- Characters that appear in uppercase in the syntax and must be coded as shown
- Variable information, or
- A combination.

For example, DATA on a DD statement, programmer's-name on a JOB statement, and PGM=program-name on an EXEC statement.

Code positional parameters first in the parameter field in the order shown in the syntax. If you omit a positional parameter and code a following positional parameter, code a comma to indicate the omitted parameter. Do not code the replacing comma if:

- The omitted positional parameter is the last positional parameter.
- All following positional parameters are also omitted.
- Only keyword parameters follow.
- All positional parameters are omitted.

Keyword parameters: A keyword consists of characters that appear in uppercase in the syntax and must be coded as shown followed by an equals sign followed by either characters that must be coded as shown or variable information. For example, RD=R and MSGCLASS=class-name on the JOB statement.

Code any of the keyword parameters for a statement in any order in the parameter field after the positional parameters. Because of this positional independence, never code a comma to indicate the absence of a keyword parameter.

Multiple subparameters: A positional parameter or the variable information in a keyword parameter sometimes consists of more than one item, called a subparameter list. A subparameter list can consist of both positional and keyword subparameters. These subparameters follow the same rules as positional and keyword parameters.

When a parameter contains more than one subparameter, separate the subparameters by commas and enclose the subparameter list in parentheses or, if indicated in the syntax, by apostrophes. If the list is a single keyword subparameter or a single positional subparameter with no omitted preceding subparameters, omit the parentheses or apostrophes.

Null positional subparameters: You are allowed to specify null (that is, omitted) positional subparameters except where the Syntax section of a particular parameter states otherwise. (For example, null positional subparameters are **not** allowed on a COND parameter of an EXEC statement or on an AMP parameter of a DD statement.) You specify a null positional subparameter by following the coding rules listed previously for an omitted positional parameter.

JES2 control statement fields

The rules for coding JES2 control statements are the same as the rules for JCL statements, with the following additions:

- Columns 1 and 2 always contain the characters /*. Columns 73-80 are ignored by z/OS and are typically used for sequence numbers.
- Do not code comments on any JES2 statements. Where comments are needed, code a JCL comment statement.
- If you code the same parameter on the same statement more than once, JES2 uses the value in the last parameter.

When coding a JES2 control statement more than once, be aware of the following JES2 actions:

- If the same parameter appears on more than one statement, JES2 uses the value coded on the last statement.
- If the statements contain different parameters, JES2 uses all parameters combined.

JES3 control statement fields

The rules for coding JES3 control statements are the same as the rules for JCL statements, with the following additions:

- Columns 1, 2, and 3 generally contain the characters //* (slash-slash-asterisk). Some JES3 control statements may contain, and certain other JES3 control statements *must* contain only a single slash-asterisk (/*) in columns 1 and 2.
- Columns 3 and 4 must not be blank.
- To code a comment on a JES3 control statement, code a blank after the control statement, and end the comment before column 72. Columns 73-80 are ignored by z/OS and are typically used for sequence numbers.

Continuing statements

You can continue some JCL statements, as described in the following sections.

Continuing JCL statements

When the total length of the fields on a control statement exceeds 71 columns, continue the fields onto one or more records. Each line is a record.

The following are JCL statements that you **cannot** continue. While you cannot continue these statements, you can code as many separate statements as you need.

- JCL Command statement
- Comment statement
- Delimiter statement
- Null statement

For all other JCL statements, you can continue the parameter field or the comments field on the JCL statement. If you continue both the parameter field and the comments field on the same record, the system ignores the indication to continue the comment. How you continue a parameter field depends on whether the parameter is enclosed in apostrophes.

Continuing the parameter field

1. Interrupt the field after a complete parameter or subparameter, including the comma that follows it, at or before column 71.
2. Code // in columns 1 and 2 of the following statement.
3. Code a blank character in column 3 of the following statement. If column 3 contains anything but a blank or an asterisk, the system assumes the following statement is a new statement. The system issues an error message indicating that no continuation is found and fails the job.
4. Continue the interrupted parameter or field beginning in any column from 4 through 16.

Continuing parameter fields enclosed in apostrophes

To continue a parameter that is enclosed in apostrophes:

1. Extend the parameter to column 71. Do not code an apostrophe in column 71 of a JCL statement that is continued. The system interprets the apostrophe in column 71 as the final character in the statement and ignores the continuation.
2. Code // in columns 1 and 2 of the following statement.
3. Continue the parameter in column 16 of the following statement even if this splits the parameter. Trailing blanks or commas within the apostrophes do not indicate a continued statement; the system treats them as part of the parameter.

The following example shows the specification of a long file name in the PARM field:

```
//STEP1 EXEC                          PGM=IEFBR14,PARM=(PARM1,'/DIR1/DIR2
//              /DIR3/DIR4/DIR5/DIR6/DIR7/DIR8/DIR9/DIR10/DIR11/DIR12/DI
//              R13/FILENM')
```

Continuing the comments field

Include comments by following an interrupted parameter field with at least one blank. To continue a comment:

1. Interrupt the comment at a convenient place before column 72, up to and including column 71.
2. Code a nonblank character in column 72.
3. Code // in columns 1 and 2 of the following statement.
4. Code a blank character in column 3 of the following statement.
5. Continue the comments field beginning in any column after column 3.

You can use JCL comment statements as an alternative way to imbed comments in the JCL stream.

Examples of continued statements

Example 1

```
//DD1  DD   DSNAME=SWITCH.LEVEL18.GROUP12,UNIT=3390,
//     VOLUME=339023,SPACE=(TRK,(80,15)),DISP=(,PASS)
```

Example 1 shows continuation of a DD statement. The DD statement is continued from the first card image to the second card image. The comma on the first line indicates that continuation is expected.

Example 2

```
//DS1 DD  DSNAME=INDS,DISP=OLD,CHKPT=EOV,   MY INPUT DATA SET
//     UNIT=SYSSQ,VOLUME=SER=(TAPE01,TAPE02,TAPE03)
```

Example 2 shows continuation of a DD statement with a comment on the first statement. The DD statement is continued from the first card image to the second card image. The comma followed by a blank on the first line indicates that continuation is expected. The comment on the first card image is not continued to the next card image.

Chapter 12. DD statement

Purpose: Use the DD (data definition) statement to describe a data set and to specify the input and output resources needed for the data set.

The parameters you can specify for data set definition are arranged alphabetically in the following pages.

References: For information about the JES initialization parameters that provide installation defaults, see *z/OS JES2 Initialization and Tuning Reference* and *z/OS JES3 Initialization and Tuning Reference*.

Description

Syntax

```
// [ddname             ] DD [positional-parameter][,keyword-parameter]...[comments]
   [procstepname.ddname]

// [ddname             ] DD
   [procstepname.ddname]
```

- The DD statement consists of the characters // in columns 1 and 2 and four fields: name, operation (DD), parameter, and comments. Do not code comments if the parameter field is blank.
- A DD statement is required for each data set.
- The maximum number of DD statements per job step is 3273, based on the number of single DD statements allowed for a TIOT (task input output table) control block size of 64K. This limit can be different depending on the installation-defined TIOT size. The IBM-supplied default TIOT size is 32K. For information about changing the size of the TIOT, see the ALLOCxx parmlib member information in *z/OS MVS Initialization and Tuning Reference*. For information about how dynamic allocation might cause changes to the task input/output table (TIOT), see *z/OS MVS Programming: Authorized Assembler Services Guide*.

  In a JES3 system, the installation might further reduce the maximum number of DD statements per job.

Name field

When specified, code a ddname as follows:

- Each ddname should be unique within the job step. If duplicate ddnames appear in a job step, processing is as follows:
  - **In a JES2 system**: The system performs device and space allocation and disposition processing for both DD statements; however, it directs all references to the first DD statement in the step.
  - **In a JES3 system**: If both DD statements request JES3 or jointly-managed devices, the system cancels the job during JES3 interpretation. If only one or neither DD statement requests a JES3 or jointly-managed device, the system performs device and space allocation processing for both DD statements; however, it directs all references to the first DD statement in the step.
- The ddname must begin in column 3.
- The ddname is 1 through 8 alphanumeric or national ($, #, @) characters.
- The first character must be alphabetic or national ($, #, @).
- The ddname must be followed by at least one blank.

Omitting the ddname: Do not code a ddname when the DD statement defines a data set that is concatenated to the data set of the preceding DD statement. You can code PATH or DSNAME in either or both of the DD statements if your program uses BSAM, QSAM or BPAM.

Note: Prior to z/OS V1R7 you could define indexed sequential data sets using DSORG=IS and a DD statement without a name.

© Copyright IBM Corp. 1988, 2017

Name field when overriding a procedure DD statement: Code the following in the name field of a DD statement that is to override a procedure DD statement:

1. The name of the procedure step that contains the DD statement to be overridden
2. Followed by a period
3. Followed by the ddname of the procedure DD statement that is to be overridden.

Name field when adding a DD statement to a procedure: Code the following in the name field of a DD statement that is to be added to a procedure:

1. The name of the procedure step to which the DD statement is to be added
2. Followed by a period
3. Followed by a ddname of your choosing.

For example:

```
   //PROCSTP1.DDA DD parameters
```

Name field when adding a DD statement to a program: When you code a DD statement with a ddname of procstepname.ddname within a program step, the system:

1. Checks the syntax of both the procstepname qualifier and the ddname qualifier
2. Uses only the ddname qualifier as the statement ddname
3. Adds the DD statement to the program step that contains the statement
4. Issues an informational message because procstepname is coded outside of a procedure.

Special ddnames: Use the following special ddnames only when you want to use the facilities these names represent to the system. These facilities are explained in Chapter 13, “Special DD statements,” on page 287.

```
JOBLIB
STEPLIB
SYSABEND
SYSCKEOV
SYSMDUMP
SYSUDUMP
```

Do not use the following ddnames. They are reserved for compatibility with the prior releases of MVS.

```
JOBAT
STEPCAT
```

Do not use the following ddnames on a DD statement in a JES2 system. They have special meaning to JES2.

```
JESJCLIN
JESJCL
JESMSGLG
JESYSMSG
```

The following ddnames have special meaning to JES3; do not use them on a DD statement in a JES3 system.

```
JCBIN
JCBLOCK
JCBTAB
JESJCLIN
JESInnnn
JESJCL
JESMSGLG
JOURNAL
JST
JESYSMSG
JES3CATLG
J3JBINFO
J3SCINFO
```

```
J3STINFO
STCINRDR
TSOINRDR
```

Operation field

The operation field consists of the characters DD and must be preceded and followed by at least one blank. It can begin in any column.

Parameter field

A DD statement has two kinds of parameters: positional and keyword. All parameters are optional.

Leave the parameter field blank only in the following cases:

- When SMS will provide the necessary DD description.
- When leaving a DD statement within a concatenation unchanged and overriding parameters on subsequent DD statements within that concatenation.

Positional parameters: A DD statement can contain **one** positional parameter. If coded, this positional parameter must precede all keyword parameters.

Table 15. Positional parameters

POSITIONAL PARAMETERS	VALUES	PURPOSE

`[ * ]`
`[DATA]`
See section "* Parameter" on page 93 or "DATA parameter" on page 119
	*: for data sets containing no JCL
	DATA: for data sets containing JCL
		In a non-APPC scheduling environment, begins an in-stream data set.

`DUMMY` See section "DUMMY parameter" on page 174		Specifies no space allocation, no disposition processing, and, for BSAM and QSAM, no I/O.
`DYNAM` See section "DYNAM parameter" on page 176		(Parameter is supported to provide compatibility with previous systems.)

Keyword parameters: A DD statement can contain the following keyword parameters. You can code any of the keyword parameters in any order in the parameter field after a positional parameter, if coded.

Do not use DD statement keywords as symbolic parameters in procedures to be started by a START command from the operator console.

Table 16. Keyword parameters

KEYWORD PARAMETERS	VALUES	PURPOSE
`ACCODE=access-code` See section "ACCODE parameter" on page 96	For ISO/ANSI/FIPS Version 3 tapes, access-code: 1 - 8 characters, first must be upper case A - Z. For ISO/ANSI Version 4 tapes, access-code: 1 - 8 characters, first must be upper case A - Z, number 0 - 9, or one of these special characters: ! * " % & ' ( ) + , - . / : ; < = > ? _	Specifies or changes an accessibility code for an ISO/ANSI/FIPS Version 3 or ISO/ANSI Version 4 tape output data set.

Table 16. Keyword parameters (continued)

KEYWORD PARAMETERS	VALUES	PURPOSE
``` AMP=(subparameter) AMP=('subparameter[,subparameter]...') subparameters: ACCBIAS= [USER] [SYSTEM] [DO] [DW] [SO] [SW] AMORG BUFND=number BUFNI=number BUFSP=bytes CROPS= {RCK} {NCK} {NRE} {NRC} FRLOG= {NONE} {REDO} MSG=SMBBIAS OPTCD= {I } {L } {IL} RECFM= {F } {FB} {V } {VB} RMODE31=[ALL] [BUFF] [CB] [None] SMBDFR= {Y | N} SMBHWT= nn SMBVSP= {nnK | nnM} SMBVSPI= {nnK | nnM} STRNO=number SYNAD=modulename TRACE ``` See section “AMP parameter” on page 97	see *z/OS DFSMS Using Data Sets*	Completes information in an access method control block (ACB) for a VSAM data set.
With SMS only: ``` AVGREC= {U} {K} {M} ``` See section “AVGREC parameter” on page 104	U: space specified in records K: space specified in thousands of records M: space specified in millions of records	Specifies a record request and the quantity of primary and secondary space specified on the SPACE parameter.
``` BLKSIZE= {value} {valueK} {valueM} {valueG} ``` See section “BLKSIZE parameter” on page 106	value: the specified maximum length, in bytes, of a block (The maximum is depending on the device type.) valueK: the specified maximum length, in kilobytes, of a block (The maximum is 2097152K.) valueM: the specified maximum length, in megabytes, of a block (The maximum is 2048M.) valueG: the specified maximum length, in gigabytes, of a block (The maximum is 2G.)	Specifies the maximum length of a block.

Table 16. Keyword parameters (continued)

KEYWORD PARAMETERS	VALUES	PURPOSE

`BLKSZLIM= {value}`
`{valueK}`
`{valueM}`
`{valueG}`
See section "BLKSZLIM parameter" on page 107
	value: 32,760 bytes - 2,147,483,648 bytes (two gigabytes)
	valueK: 32K - 2,097,152K (two gigabytes)
	valueM: 1M - 2048M (two gigabytes)
	valueG: 1G - 2G (two gigabytes)
		Specifies an upper limit on a data set's block size if BLKSIZE is omitted from all sources and the system determines the block size for the data set.

`BURST= {YES}`
`{Y  }`
`{NO }`
`{N  }`
See section "BURST parameter" on page 108
	YES or Y: burster-trimmer-stacker
	NO or N: continuous forms stacker
		Directs output to a stacker on a continuous-forms AFP printer.

CCSID=nnnnn See section "CCSID parameter" on page 110	nnnnn: 1 - 65535	Specifies the coded character set identifier indicating the character code conversion performed on reads from and writes to tapes accessed in ISO/ANSI Version 4 format.

`CHARS= {table-name }`
`{(table-name[,table-name]...)}`
`{DUMP }`
`{(DUMP[,table-name]...) }`
See section "CHARS parameter" on page 112
	1 - 4 table-name subparameters:
	1 - 4 alphanumeric or $, #, @ characters
	DUMP: 204-character print lines on 3800 model 1
		Names coded fonts for printing on an AFP printer. Requests a high-density dump on a SYSABEND or SYSUDUMP DD statement.

`CHKPT=EOV` See section "CHKPT parameter" on page 114		Requests a checkpoint at each end-of-volume except the last.

`CNTL= {*.label }`
`{*.stepname.label }`
`{*.stepname.procstepname.label}`
See section "CNTL parameter" on page 115
	label: names CNTL statement
	stepname: CNTL in named step
	procstepname: step in named procedure
		Causes the system to execute statements following an earlier CNTL statement.

`COPIES= {nnn }`
`{(nnn,(group-value[,group-value]...))}`
`{(,(group-value[,group-value]...)) }`
See section "COPIES parameter" on page 116
	nnn (JES2): 1 - 255
	nnn (JES3): 0 - 255
	1 - 8 group-values (JES2): 1 - 255
	1 - 8 group values (JES3): 1 - 254
		Specifies number of copies printed. For an AFP printer, can instead specify number of copies of each page printed before the next page is printed.

With SMS only: `DATACLAS=data-class-name` See section "DATACLAS parameter" on page 122	data-class-name: installation-defined name of a data class	Specifies the data class for a new data set.

`DCB=(subparameter[,subparameter]...)`
`DCB= ( {dsname } )`
`( {*.ddname } )`
`( {*.stepname.ddname } )`
`( {*.stepname.procstepname.ddname} )`
`( [,subparameter]... )`
See section "DCB parameter" on page 125
	subparameter: see tables in DCB parameter description
	*.ddname: copy DCB parameter from named cataloged data set
	dsname: copy DCB information from named earlier DD statement
	stepname: DD in named step
	procstepname: step in named procedure
		Completes information in data control block (DCB).

Table 16. Keyword parameters (continued)

KEYWORD PARAMETERS	VALUES	PURPOSE
`DDNAME=ddname` See section "DDNAME parameter" on page 138	ddname: names later DD statement	Postpones defining the data set until later in same step: on a DD statement in the calling step or in a procedure called by the step.

`DEST=destination`
 	
`destination (JES2):`
`LOCAL`
`name`
`Nnnnnn`
`NnRmmmmm to NnnnnnRm`
`(node,remote)`
`Rnnnnn or RMnnnnn or RMTnnnnn`
`Unnnnn`
`(node,userid)`
 	
`destination (JES3):`
`ANYLOCAL`
`device-name`
`device-number`
`group-name`
`nodename`
`(node,userid)`
 	
See section "DEST parameter" on page 142
	LOCAL or ANYLOCAL: local device
	
	name: named local or remote device
	
	Nnnnnn: node (1 - 32,767)
	
	NnRmm: node (1 - 32,767) and remote work station (1 - 32,767); 6 digits maximum for n and m combined
	
	Rnnnnn or RMnnnnn or RMTnnnnn: remote terminal (1 - 32,767)
	
	Unnnnn: local terminal (1 - 32,767)
	
	(node,userid): node (1 - 8 alphanumeric or $, #, @ characters) and TSO/E userid (1 - 7 alphanumeric or $, #, @ characters) or VM userid (1 - 8 alphanumeric or $, #, @ characters)
	
	device-number: 3-digit or 4-digit hexadecimal number (/ required before 4-digit number)
	
	device-name: local device (1 - 8 alphanumeric or $, #, @ characters)
	
	group-name: 1 or more local devices or remote stations (1 - 8 alphanumeric or $, #, @ characters)
	
	nodename: node (1 - 8 alpha- numeric or $, #, @ characters)
		Sends a sysout data set to the specified destination.

`DISP=[status]`
`DISP=([status][,normal-termination-disp]`
`[,abnormal-termination-disp])`
 	
See section "DISP parameter" on page 146
	status: NEW, OLD, SHR (for shared), MOD (for data set to be modified)
	
	normal-termination-disp: DELETE, KEEP, PASS, CATLG, or UNCATLG
	
	abnormal-termination-disp: DELETE, KEEP, CATLG, or UNCATLG
		Describes the status of the data set and tells the system to do the following with the data set after normal or abnormal termination of the step or job: delete or keep it on its volume(s), pass it to a later step, or add it to or remove it from the catalog.

`DLM=delimiter` See section "DLM parameter" on page 158	delimiter: 8 characters delimiter: 2 characters (for JES3 only)	In a non-APPC scheduling environment, terminates an in-stream data set.

`DSID= {id }`
`{(id,[V])}`
 	
See section "DSID Parameter" on page 160
	id: 1 - 8 characters
	V: label was verified (only on a SYSIN DD statement)
		Identifies a data set on a diskette of a 3540 Diskette Input/Output Unit.

Table 16. Keyword parameters (continued)

KEYWORD PARAMETERS	VALUES	PURPOSE
{DSNAME} = {dsname } {DSN } {dsname(member-name) } {dsname(generation-number) } {&&dsname(member-name) } {*.ddname } {*.stepname.ddname } {*.stepname.procstepname.ddname} {NULLFILE } See section "DSNAME parameter" on page 162	unqualified dsname: 1 - 8 alphanumeric or $, #, @ characters, -, +0 qualified dsname: multiple names joined by periods member-name: member in PDS or PDSE generation-number: 0 or signed integer stepname: DD in named step procstepname: step in named procedure NULLFILE: dummy data set	Names the data set.
DSNTYPE= {BASIC} {LARGE} {EXTREQ} {EXTPREF} {LIBRARY} {(LIBRARY,1)} {(LIBRARY,2)} {HFS } {PDS } {PIPE } See section "DSNTYPE parameter" on page 170	BASIC: basic format data set LARGE: large format data set EXTREQ: extended format data set EXTPREF: extended format data set (preferred) LIBRARY: partitioned data set extended (PDSE), with optional version number HFS: hierarchical file system (HFS) data set PDS: partitioned data set PIPE: FIFO special file	Specifies the type of data set.
EATTR=[OPT|NO] See section "EATTR parameter" on page 177	OPT: Extended attributes are optional NO: No extended attributes	Indicate whether the data set can support extended attributes (format 8 and 9 DSCBs) or not.
EXPDT= {yyddd } {yyyy/ddd} See section "EXPDT parameter" on page 178	yyddd: expiration date (yy: 2-digit year, ddd: day 001-366) yyyy/ddd: expiration date (yyyy: 4-digit year, ddd: day 001-366)	Specifies an expiration date for the data set.
FCB= {fcb-name } {(fcb-name [,ALIGN]) } [,VERIFY] See section "FCB parameter" on page 180	fcb-name: 1 - 4 alphanumeric or $, #, @ characters ALIGN: operator check forms alignment VERIFY: operator verify FCB image	Specifies FCB image, carriage control tape for 1403 Printer, or data-protection image for 3525 Card Punch.
FILEDATA= {BINARY} {TEXT } {RECORD} See section "FILEDATA parameter" on page 182	BINARY: byte-stream file TEXT: delimited by the EBCDIC newline character RECORD: the data consist of records with prefixes.	Specifies the content type of a z/OS UNIX file.
FLASH= {overlay-name } {(overlay-name[,count])} {NONE } See section "FLASH parameter" on page 184	overlay-name: forms overlay frame (1 - 4 alphanumeric or $, #, @ characters) count: copies with overlay (0 - 255) NONE: suppresses flashing	For printing on a 3800 Printing Subsystem, indicates that the data set is to be printed with the named forms overlay and can specify how many copies are to be flashed.
FREE= {END } {CLOSE} See section "FREE parameter" on page 185	END: unallocate at end of last step CLOSE: unallocate when data set is closed	Specifies when to unallocate the resources for this data set.

Table 16. Keyword parameters (continued)

KEYWORD PARAMETERS	VALUES	PURPOSE
`HOLD= {YES} {Y } {NO } {N }` See section "HOLD parameter" on page 190	YES or Y: holds this sysout data set NO or N: allows normal processing for this sysout data set's output class	Tells the system to hold this sysout data set until released by the operator.
`KEYLEN=bytes` See section "KEYLEN parameter" on page 197	bytes: number of bytes (1-255 for key-sequenced (KS), 0-255 for sequential (PS) or partitioned (PO))	Specifies the length of the keys in the data set.
With SMS only: `KEYOFF=offset-to-key` See section "KEYOFF parameter" on page 198	offset-to-key: position of key (0 to difference of LRECL and KEYLEN minus 1)	Specifies the offset of the first byte of the record key.
`LABEL= ([data-set-seq-no][,label-type] [,PASSWORD]) [,IN ] [,RETPD=nnnn ]) [,NOPWREAD]) [,OUT] [,EXPDT= {yyddd }] [, ]) [, ] [ {yyyy/ddd}]` See section "LABEL parameter" on page 199		
	data-set-seq-no: data set position on tape volume (1 - 4 decimal digits) label-type: SL: IBM standard labels SUL: IBM standard and user labels AL: ISO/ANSI Version 1 and ISO/ANSI/FIPS Version 3 labels AUL: user labels and ISO/ANSI Version 1 and ISO/ANSI/FIPS Version 3 labels NSL: nonstandard labels NL: no labels BLP: bypass label processing LTM: leading tapemark PASSWORD: password required to access data set NOPWREAD: password required to change or delete data set IN: only read BSAM data set opened for INOUT or BDAM data set opened for UPDAT OUT: only write to BSAM data set opened for OUTIN or OUTINX RETPD=nnnn: retention period (nnnn: 1 - 4 decimal digits) EXPDT=yyddd: expiration date (yy: 2-digit year, ddd: day 001 - 366) EXPDT=yyyy/ddd: expiration date (yyyy: 4-digit year, ddd: day 001 - 366)	Specifies information about a data set's label, password, opening, expiration date, and, for a tape data set, relative position on the volume.
`LGSTREAM=name` See section "LGSTREAM parameter" on page 205	name: the name of the prefix that the system logger uses for the forward recovery log stream	Specifies the prefix of the name of the log stream for an SMS-managed VSAM data set.
With SMS only: `LIKE=data-set-name` See section "LIKE parameter" on page 206	data-set-name: dsname of model data set	Specifies the attributes of a new data set.

Table 16. Keyword parameters (continued)

KEYWORD PARAMETERS	VALUES	PURPOSE
`LRECL=(bytes)` See section "LRECL parameter" on page 208	bytes: length in bytes (1-32,760 for PS or PO, 1-32,761 for KS, ES, or RR)	Specifies the length of the records in the data set.
`MAXGENS=maximum-generations` See section "MAXGENS parameter" on page 210	maximum-generations: 0 - 2000000000	Specifies the maximum number of generations for members in a Version 2 PDSE.
With SMS only: `MGMTCLAS=data-class-name` See section "MGMTCLAS parameter" on page 211	data-class-name: installation- defined name of a data class	Specifies the management class for a new data set.

`MODIFY= {module-name }`
`{(module-name[,trc])}`
See section "MODIFY parameter" on page 212
	module-name: 1 - 4 alphanumeric or $, #, @ characters
	trc: table-name in CHARS parameter (0 for first, 1 for second, 2 for third, and 3 for fourth table-name)
		Specifies a copy-modification module in SYS1.IMAGELIB to be used by JES to print the data set on a 3800 Printing Subsystem.

`OUTLIM=number` See section "OUTLIM parameter" on page 214	number: 1 - 16777215 logical records maximum	Limits the logical records in this sysout data set.

`OUTPUT= {reference }`
`{(reference[,reference]...)}`
reference:
• *.name
• *.stepname.name
• *.stepname.procstepname.name
See section "OUTPUT parameter" on page 215
	name: names earlier OUTPUT JCL statement
	stepname: OUTPUT JCL in named step
	procstepname: step in named procedure
		Associates this sysout data set with one or more OUTPUT JCL statements.

`PATH=pathname` See section "PATH parameter" on page 218	pathname: pathname for a file	Specifies the name of a UNIX File.

`PATHDISP=(normal-termination-disposition,`
`abnormal-termination-disposition)`
See section "PATHDISP parameter" on page 221
	normal-termination-disposition: KEEP, DELETE
	abnormal-termination-disposition: KEEP, DELETE
		Tells the system to keep or delete the file after the job step ends.

`PATHMODE=file-access-attribute`
`PATHMODE=(file-access-attribute`
`[,file-access-attribute]...)`
See section "PATHMODE parameter" on page 223
	file-access-attribute for file owner class: SIRUSR, SIWUSR, SIXUSR, SIRWXU
	file-access-attribute for file group class: SIRGRP, SIWGRP, SIXGRP, SIRWXG
	file-access-attribute for file other class: SIROTH, SIWOTH, SIXOTH, SIRWXO
	file-access-attribute to set process IDs: SISUID, SISGID
		Specifies file access attributes when creating a UNIX File.

`PATHOPTS=file-option`
`PATHOPTS=(file-option[,file-option]...)`
See section "PATHOPTS parameter" on page 226
	file-option for access group: ORDONLY, OWRONLY, ORDWR
	file-option for status group: OAPPEND, OCREAT, OEXCL, ONOCTTY, ONONBLOCK, OSYNC, OTRUNC
		Specifies access and status for a file.

Table 16. Keyword parameters (continued)

KEYWORD PARAMETERS	VALUES	PURPOSE
`PROTECT=YES` See section "PROTECT parameter" on page 229		Requests that RACF create a discrete profile to protect a data set on direct access or a tape volume.

Coding RECFM for BDAM Access Method:
`RECFM= {U }`
`{V }`
`{VS }`
`{VBS}`
`{F }`
`{FT }`
 	
Coding RECFM for BPAM Access Method:
`RECFM= {U } [A]`
`{UT } [M]`
`{V }`
`{VB }`
`{VS }`
`{VT }`
`{VBS }`
`{VBT }`
`{VBST}`
`{F }`
`{FB }`
`{FT }`
`{FBT }`
 	
Coding RECFM for BSAM, EXCP, and QSAM Access Methods:
`RECFM= {U } [A]`
`{UT } [M]`
`{F }`
`{FB }`
`{FS }`
`{FT }`
`{FBS }`
`{FBT }`
`{V }`
`{VB }`
`{VS }`
`{VT }`
`{VBS }`
`{VBT }`
`{VBST}`
 	
For BSAM, EXCP, and QSAM using ISO/ANSI/FIPS data sets on tape:
`RECFM= {D } [A]`
`{DB }`
`{DS }`
`{DBS}`
`{U }`
`{F }`
`{FB }`
 	
See section "RECFM parameter" on page 231
	Record format is:
	F: fixed length
	B: blocked
	S: spanned
	V: variable length
	U: undefined length
	T: track-overflow feature
	D: variable-length ISO/ANSI tape records
	Control characters are:
	A: ISO/ANSI code
	M: machine code
		Specifies the format and characteristics of the records in a data set.

Table 16. Keyword parameters (continued)

KEYWORD PARAMETERS	VALUES	PURPOSE

With SMS only:
`RECORG= {KS}`
`{ES}`
`{RR}`
`{LS}`
See section "RECORG parameter" on page 235
	Organization of records:
	KS: key-sequenced
	ES: entry-sequenced
	RR: relative record
	LS: linear space
		Specifies the organization of the records in a VSAM data set.

With SMS only:
`REFDD= {*.ddname }`
`{*.stepname.ddname }`
`{*.stepname.procstepname.ddname}`
See section "REFDD parameter" on page 236
	Referenced DD statement:
	ddname: unqualified name
	stepname: qualified by step name
	procstepname: step in procedure
		Specifies the attributes of a new data set by referring to a previous DD statement.

`RETPD=nnnn` See section "RETPD parameter" on page 238	nnnn: number of days (0-9999)	Specifies the retention period for a new data set.

`RLS= {NRI}`
`{CR }`
`{CRE }`
See section "RLS parameter" on page 239
	NRI: can read uncommitted changes CR: can read only committed changes CRE: ensures that records read by a unit of recovery are not changed by other units of recovery until the reading unit of recovery issues a syncpoint.	Specifies the record-level sharing protocol to be used with a VSAM data set.

With SMS only: `SECMODEL=(profile-name[,GENERIC])` See section "SECMODEL parameter" on page 242	profile-name: name of model profile GENERIC: model is generic profile	Specifies a RACF profile to be used for a new data set.
`SEGMENT=page-count` See section "SEGMENT parameter" on page 244	page-count: number of pages of a sysout data set	Specifies the number of pages produced for the current segment of the sysout data set before the data set is spun-off for output processing. (JES2 only)

For system assignment of space:
`SPACE=({TRK, } (primary-qty [,second-qty] [,directory]) [,RLSE] [,CONTIG] [,ROUND] )`
`{CYL, } [, ] [, ] [,MXIG ]`
`{blklgth,} [,ALX ]`
`{reclgth,} [, ]`
To request specific tracks:
`SPACE=`
`(ABSTR, (primary-qty,address [,directory]))`
To request directory blocks (with SMS only):
`SPACE=(,(,,directory))`
See section "SPACE parameter" on page 245		

Table 16. Keyword parameters (continued)

KEYWORD PARAMETERS	VALUES	PURPOSE
	TRK: allocation in tracks CYL: allocation in cylinders blklgth: allocation in average blocks, 1 - 65535 reclgth: allocation in average records (SMS) primary-qty: number of tracks, cylinders or blocks to be allocated second-qty: additional tracks or cylinders to be allocated, if more are needed directory: number of 256-byte records for PDS directory RLSE: release unused space when data set is closed CONTIG: contiguous primary allocation MXIG: allocation in largest available space ALX: allocation of up to 5 separate contiguous primary quantities ROUND: allocation by block length rounded to integral cylinders ABSTR: allocation at the specified address address: track number of first track to be allocated	Requests space for a new data set on direct access storage.

`SPIN= {UNALLOC} (JES2 only)`
`{NO }`
See section "SPIN parameter" on page 252
	UNALLOC (JES2 only): the data set is available for printing immediately upon unallocation. UNALLOC is supported on JES2 only. NO: the data set is available for printing at the end of the job	Specifies that the output for a sysout data set is available for printing immediately upon unallocation or at the end of the job.

With SMS only: `STORCLAS=storage-class-name` See section "STORCLAS parameter" on page 255	storage-class-name: installation- defined name of a storage class	Specifies the storage class for a new data set.

`SUBSYS= (subsystem-name )`
`([,subsystem-parameter]...)`
See section "SUBSYS parameter" on page 256
	subsystem-name: identifies the subsystem
	subsystem-parameter: specifies information for the subsystem
		Requests a subsystem to process this data set.

`SYSOUT=class`
`SYSOUT=([class] [,writer-name] [,form-name])`
`[,INTRDR ] [,code-name]`
`[, ]`
`SYSOUT=*`
`SYSOUT=(,)`
See section "SYSOUT parameter" on page 261
	class: A - Z, 0 - 9
	writer-name: 1 - 8 alphanumeric or $, #, @ characters
	form-name: 1 - 4 alphanumeric or $, #, @ characters
	code-name: 1 - 4 alphanumeric or $, #, @ characters (JES2 only)
	*: same output class as MSGCLASS parameter on JOB statement
		Defines this data set as a sysout data set and (1) assigns it to an output class, (2) requests external writer to process it, (3) identifies print or punch forms, and (4) refers to the code-name of a JES2 /*OUTPUT statement.

Table 16. Keyword parameters (continued)

KEYWORD PARAMETERS	VALUES	PURPOSE
`TERM=TS` See section "TERM parameter" on page 266		The TERM parameter has no function in an APPC scheduling environment. In a foreground job,indicates that this data set is coming from or going to a TSO/E userid. In a batch job, indicates that this DD statement begins an in-stream data set.

`UCS= {character-set-code }`
`{(character-set-code [,FOLD] [,VERIFY])}`
 	
`{ [, ] }`
 	
See section "UCS parameter" on page 268
	character-set-code: 1 - 4 alpha- numeric or $, #, @ characters
	
	FOLD: operator load chain or train in fold mode
	
	VERIFY: operator verify UCS image
		Specifies universal character set, print train, or font for an AFP printer.

`UNIT= ([ddd ] [,unit-count] [,DEFER])`
`[/ddd ] [,P ]`
`[/dddd ] [, ]`
`[device-type ]`
`[group-name ]`
 	
`UNIT=AFF=ddname`
 	
See section "UNIT parameter" on page 270
	device-number: 3-digit or 4-digit hexadecimal number (/ required before 4-digit number and optional before 3-digit number)
	
	device-type: machine type and model
	
	group-name: 1 - 8 alphanumeric or $, #, @ characters
	
	unit-count: 1 - 59
	
	P: allocate same number of devices as volumes for parallel mount
	
	DEFER: defers mounting until open
	
	SMSHONOR: indicates that the system should honor the device number or group-name for a SMS-managed tape library request
	
	AFF=ddname: requests allocation of same devices as for DD statement ddname
		Requests allocation to a specific device, a type or group of devices, or the same device(s) as another data set. Also can specify how many devices and deferred mounting.

`{VOLUME}=([PRIVATE] [,RETAIN] [,volume-seq-no][,volume-count][,][SER=(serial-number[,serial-number]...)])`
`{VOL } [, ] [, ]`
`[REF=dsname ]`
 	
`[REF=*.ddname ]`
 	
`[REF=*.stepname.ddname ]`
 	
`[REF=*.stepname.procstepname.ddname ]`
 	
`[REF=*.procstepname.ddname ]`
 	
See section "VOLUME parameter" on page 277		

Table 16. Keyword parameters (continued)		
KEYWORD PARAMETERS	**VALUES**	**PURPOSE**
	PRIVATE: requests a private volume RETAIN: requests private tape volume remain mounted and unwound or requests public tape volume be retained at device volume-seq-no: begins processing with volume 1 - 255 of existing multivolume data set volume-count: maximum volumes for output data set (1 - 255) serial-number subparameters (1 - 255): volume serial numbers (1 - 6 alphanumeric, $, #, @, or special characters) REF: copy volume serial numbers from another data set or earlier DD statement, or copy storage class for SMS-managed data sets dsname: from cataloged or passed data set ddname: from named earlier DD statement stepname: DD in named step procstepname: step in named procedure	Identifies the volume(s) on which a data set resides or will reside.

Comments field

The comments field follows the parameter field after at least one intervening blank. If you do not code any parameters on a DD statement, do not code any comments.

Location in the JCL

Most DD statements define data sets to be used in a job step, in a cataloged procedure step, or in an in-stream procedure step; these appear after the EXEC statement for the step. Some DD statements define data sets for the job, for example, the JOBLIB DD statement; these appear after the JOB statement and before the first EXEC statement.

When overriding or adding to procedures: Place DD statements that override, nullify, or add parameters immediately following the EXEC statement that calls the procedure. Place overriding and nullifying DD statements first, followed by all added DD statements. Last in the calling step are any DD * or DD DATA statements with their in-stream data.

To override more than one DD statement in a procedure, place the overriding DD statements in the same order as the overridden DD statements in the procedure.

Concatenating data sets

When data sets are concatenated, the application program can treat them as if they were one logical data set. In general, most of the logical attributes of the first DD statement apply to all of them. You can **concatenate** input data sets for the duration of a job step. Each of the concatenated data sets can reside on a different volume. For details on concatenating data sets, see *z/OS DFSMS Using Data Sets*. Note that you cannot concatenate output data sets.

Two types of concatenation

Partitioned concatenation

This is any combination of the following:

- Partitioned data set (PDS).
- Partitioned data set extended (PDSE).

- z/OS UNIX directory. Code the PATH keyword.

In general, do not code a member name in these cases. If you code a member name on the first DD statement, it causes the system to position to that member first unless the application program overrides it. The application program uses BPAM to read.

Sequential concatenation

There are two types of sequential concatenation:

- Like. The same logical record length (LRECL) value and record format apply to all the data sets. The block size (BLKSIZE) values and device characteristics might differ.
- Unlike. Any of data set characteristics might differ. The application program must have logic to support this capability.

The DD statements are for any combination of sequential disk or tape data sets, members of PDSs and PDSEs, spooled input stream ("sysin") data sets, z/OS UNIX files, a TSO terminal (TERM=TS) and unit record (such as virtual card reader) devices. The application program uses BSAM, QSAM or EXCP to read.

Coding a concatenation

To concatenate data sets, omit the ddnames from all the DD statements except the first in the sequence. The data sets are processed in the same sequence as the DD statements defining them.

Devices for concatenated data sets

Concatenated data sets can reside on different devices and different types of devices. (This may require internal DCB modifications, see *z/OS DFSMS Using Data Sets*.)

Block sizes for concatenated data sets

Concatenated data sets can have different block sizes. In a few cases, the data set with the largest block size must appear first in the concatenation. (Note that you can state a value equal to the largest block size for BLKSIZE on the first DD statement, regardless of what the actual block size of this data set is.) Certain data sets can be concatenated in any order of block size; these are:

- Partitioned data sets (PDSs), and partitioned data sets extended (PDSEs) without member names coded on the DD statements.
- Sequential data sets that are DASD-resident, tape-resident, or in-stream, and are accessed by QSAM and use system-created buffers.
- Sequential data sets that are DASD-resident or in-stream, and are accessed by BSAM.

For these data sets, the BLKSIZE obtained is the largest in the concatenation. Note that this block size can cause invalid attribute combinations when combined with the attributes obtained from the first data set in the concatenation.

If you do not specify a block size, the system can, under certain conditions, determine an optimum block size. For detailed information about system-determined block size, see *z/OS DFSMS Using Data Sets*.

Logical record lengths for concatenated data sets

Concatenated data sets with format-V records can have different logical record lengths as long as the data set with the largest logical record length appears first in the concatenation. (Note that you can state a value equal to the largest logical record length for LRECL on the first DD statement, regardless of what the actual logical record length of this data set is.)

References to concatenated data sets

If you make a **backward reference** to a concatenation (using *.), the system obtains information only from the first data set defined in the sequence of DD statements.

If you make a **forward reference** to a concatenation (using the DDNAME parameter), the forward reference resolves to the first data set in the concatenation. If there are no DD statements between the

forward reference and the concatenation, the rest of the data sets in the concatenation are appended to the first data set in the concatenation. The following example illustrates this.

```
//STEP1      EXEC PGM=IEBGENER
//SYSPRINT   DD   SYSOUT=*
//SYSUT1     DD   DDNAME=INPUT
//INPUT      DD   DSN=TSTDATA1,DISP=SHR
//            DD   DSN=TSTDATA2,DISP=SHR
//SYSUT2     DD   SYSOUT=*
//SYSIN      DD   DUMMY
```

In this example, SYSUT1 resolves to the first data set, TSTDATA1, defined by the DDNAME forward reference INPUT. TSTDATA2, the second data set in the DDNAME forward reference INPUT, is appended to SYSUT1. IEBGENER recognizes TSTDATA1 and TSTDATA2 as input.

If there are any DD statements between the forward reference and the concatenation, the rest of the data sets in the concatenation are appended to the last DD statement preceding the concatenation. For example:

```
//STEP1      EXEC PGM=IEBGENER
//SYSUT1     DD   DDNAME=INPUT
//SYSPRINT   DD   SYSOUT=*
//SYSUT2     DD   SYSOUT=*
//INPUT      DD   DSN=TSTDATA1,DISP=SHR
//            DD   DSN=TSTDATA2,DISP=SHR
//SYSIN      DD   DUMMY
```

In the preceding example, SYSUT1 resolves to the first data set, TSTDATA1, defined in the DDNAME forward reference INPUT. TSTDATA2 is appended to SYSUT2, the last DD statement preceding the concatenation. In this example, IEBGENER recognizes only TSTDATA1 as input.

If a concatenated DD is added to a procedure, the remaining concatenated data sets are concatenated to the last DD in the step named in an override or addition (or to the first step if no step was named in an override or addition). Note that this might result in these concatenated DDs being added to an unexpected DD. The following example illustrates this.

```
//TPROC    PROC
//S1         EXEC PGM=IEFBR14
//DD1        DD   DDNAME=INPUT
//DD2        DD   DSN=MYDSN2,DISP=SHR
//DD3        DD   DSN=MYDSN3,DISP=SHR
//S2         EXEC PGM=IEFBR14
//DDA        DD   DDNAME=INPUT
//DDB        DD   DSN=MINE2,DISP=SHR
//DDC        DD   DSN=MINE3,DISP=SHR
//          PEND
//STEP1      EXEC TPROC
//INPUT      DD   DSN=MYDSN1,DISP=SHR
//            DD   DSN=MYDSN4,DISP=SHR
//S2.INPUT   DD   DSN=MINE1,DISP=SHR
//            DD   DSN=MINE4,DISP=SHR
```

In this example, the result of the DDNAME forward reference INPUT is:

- In step S1, DD1 resolves to data set MYDSN1 and data set MYDSN4 is concatenated to data set MYDSN3.
- In step S2, DDA resolves to data set MINE1 and data set MINE4 is concatenated to data set MINE3.



Attention: The system always issues a warning message, IEF694I, even if all data sets in the concatenation are used.

Do not concatenate data sets after a DUMMY data set

If you define a data set using the **DUMMY parameter**, do not concatenate other data sets after it. When the processing program asks to read a dummy data set, the system takes an end-of-data set exit immediately and ignores any data set that might be concatenated after the dummy.

Do not code other statements between concatenated DD statements

Do not code other types of statements between two or more concatenated data definition (DD) statements. (Comments are the only exception; you can code them between DD statements.) For example, do not code a SET statement as follows:

```
//DD1    DD  DSN=A
//       DD  DSN=B
//       SET  ...
//*      Wrong!!! SET statement not allowed (this comment IS allowed)
//       DD  DSN=C
```

Examples of DD statements and ddnames

Example 1

```
//MYDS   DD  DSNAME=REPORT
//A      DD  DSNAME=FILE
```

Example 2

```
//INPUT  DD  DSNAME=FGLIB,DISP=(OLD,PASS)
//       DD  DSNAME=GROUP2,DISP=SHR
```

In this example, because the ddname is missing from the second DD statement, the system concatenates the data sets defined in these statements.

Example 3

```
//PAYROLL.DAY DD  DSNAME=DESK,DISP=SHR
```

In this example, if procedure step PAYROLL contains a DD statement named DAY, this statement overrides parameters on DD statement DAY. If the step does not contain DD statement DAY, the system adds this statement to procedure step PAYROLL for the duration of the job step.

Example 4

```
//STEPSIX.DD4 DD  DSNAME=TEXT,DISP=(NEW,PASS)
//            DD  DSNAME=ART,DISP=SHR
```

In this example, the second data set is concatenated to the first, and both are added to procedure step STEPSIX. The ddname is omitted from the second DD statement in order to concatenate data set ART to data set TEXT.

Because the system does not allow you to write to a concatenation of data, you need another data set with DISP=OLD in order to read from TEXT. Write to the new DD name before reading from DD4.

* Parameter

Parameter Type

Positional, optional

Purpose

Use the * (asterisk) parameter to begin an in-stream data set. The data records immediately follow the DD * statement; the records might be in any code such as EBCDIC. The data records end when one of the following is found:

- When DLM is not coded on this DD statement, /* in the input stream or // to indicate another JCL statement.
- When DLM is coded on this DD statement, the eight-character delimiter that are specified by the DLM parameter and in a JES2 system only, the // that indicates another JCL statement.
- The input stream runs out of records.

DCB parameter

Parameter Type

Keyword, optional

Purpose

Use the DCB parameter to complete during execution the data set information in the data control block (DCB).

The data control block is constructed by the DCB macro instruction in assembler language programs or by file definition statements or language-defined defaults in programs in other languages.

Note:

1. With SMS, you do not need to use the DCB parameter to specify data set attributes. See the DATACLAS parameter (described on section "DATACLAS parameter" on page 122), the LIKE parameter (described on section "LIKE parameter" on page 206), and the REFDD parameter (described on section "REFDD parameter" on page 236).
2. For JES3 SNA RJP, code DCB=LRECL=nnn; where nnn is 1 to 255 when SYSIN data records are greater than 80 bytes. (The default LRECL is 80 bytes.)
3. Cross checking of DCB subparameters is done when the data set is opened, not when it is allocated. It is therefore possible to create a data set which may not be usable. This may then result in the issuance of a system completion code (ABEND) or unpredictable results when the data set is opened.

References

For more information on constructing the data control block, see *z/OS DFSMS Using Data Sets*.

Syntax

```
[ DCB=(subparameter[,subparameter]...)                          ]

[ DCB= ( {dsname                          }[,subparameter]...) ]
[      ( {*.ddname                        }                  ) ]
[      ( {*.stepname.ddname               }                  ) ]
[      ( {*.stepname.procstepname.ddname  }                  ) ]
```

Parentheses: You can omit the parentheses if you code:

- Only one keyword subparameter.
- Only a data set name, dsname, without any subparameters.
- Only a backward reference without any subparameters. A backward reference is a reference to an earlier DD statement in the job or in a cataloged or in-stream procedure called by a job step. A backward reference is in the form *.ddname or *.stepname.ddname or *.stepname.procstepname.ddname.

For example, DCB=RECFM=FB or DCB=WKDATA or DCB=*.STEP3.DD2

Multiple subparameters: When the parameter contains more than one subparameter, separate the subparameters by commas and enclose the subparameter list in parentheses. For example, DCB=(RECFM=FB,LRECL=133,BLKSIZE=399) or DCB=(*.DD1,BUFNO=4)

Continuation onto another statement: Enclose the subparameter list in only one set of parentheses. End each statement with a comma after a complete subparameter. For example:

```
//INPUT  DD  DSNAME=WKDATA,DCB=(RECFM=FB,LRECL=80,BLKSIZE=800,
//         BUFL=800,BUFNO=4)
```

Alternate syntax for DCB keyword subparameters: All of the DCB keyword subparameters can be specified without the need to code DCB=. For example, the following DD statement:

```
//DDEX  DD  DSNAME=WKDATA,DCB=(RECFM=FB,LRECL=80,BLKSIZE=800),DISP=MOD
```

can also be specified as:

```
//DDEX  DD  DSNAME=WKDATA,RECFM=FB,LRECL=80,BLKSIZE=800,DISP=MOD
```

Note that KEYLEN, LRECL, and RECFM are described as DD parameters.

Note:

- If the BUFMAX subparameter is coded with or without the DCB parameter, it can have a null value only when coded on a DD which either:
  - Overrides a DD in a procedure
  - Is added to a procedure.

Subparameter definition

subparameter
(With SMS, see the DD DATACLAS parameter.)

Specifies a DCB keyword subparameter needed to complete the data control block.

An alphabetic summary of the DCB keyword subparameters follows this parameter description.

You must supply DCB information through the DCB subparameters if your processing program, the data set label, or your language's defined values do not complete the data control block.

dsname
(With SMS, see the DD LIKE parameter.)

Names a cataloged data set. The system is to copy DCB information from the data set's label. The data set must reside on a direct access volume, and the volume must be mounted before the job step is executed.

If dsname represents a VSAM data set, and you are allocating a new data set, you must also supply the RECORG parameter. You can specify RECORG explicitly (through the RECORG parameter), or implicitly, through the DATACLAS or LIKE parameters.

A hyphen is a valid character in a catalogued data set name. A data set name that contains a hyphen must be enclosed in apostrophes if it is used as a DCB subparameter.

The dsname cannot contain special characters, except for periods used in qualifying the name. Do not specify a generation data group (GDG) base name, a GDG relative generation member name, or a member name of a non-GDG data set.

The system copies the following DCB information from the data set label:

DSORG (used in a backward reference)
RECFM
OPTCD
BLKSIZE
LRECL
KEYLEN
RKP

If you do not specify the expiration date of the cataloged data set, the system copies it from the data set label. The system also copies the system code.

If you code any DCB subparameters after the dsname, these subparameters override the corresponding subparameters in the data set label. The system copies from the referenced label only those subparameters not specified on the referencing DD statement.

***.ddname**
***.stepname.ddname**
***.stepname.procstepname.ddname**
(With SMS, see the DD REFDD parameter or the DD LIKE parameter to select a comparable refer back function.)

Specify a backward reference to an earlier DD statement. The system is to copy DCB information from the DCB parameter specified on that DD statement. The DCB parameter of the referenced DD statement must contain subparameters, and it cannot name a cataloged data set or refer to another DD statement.

*.ddname specifies the ddname of an earlier DD statement in the same step. *.stepname.ddname specifies the ddname of a DD statement in an earlier step, stepname, in the same job. *.stepname.procstepname.ddname specifies the ddname of a DD statement in a cataloged or in-stream procedure called by an earlier job step. Stepname is the name of the job step that calls the procedure, and procstepname is the name of the procedure step that contains the DD statement.

If you code any DCB subparameters after the reference, these subparameters override the corresponding subparameters on the referenced DD statement. The system copies from the referenced DD statement only those subparameters not specified on the referencing DD statement.

Do not reference a DD * or a DD DATA statement.

Note: The system also copies the UCS and FCB parameters from the referenced DD statement, unless you override them in the referencing DD statement.

Completing the data control block

The system obtains data control block information from the following sources, with the highest priority first:

1. The data set label.
2. Attributes in the data class if the DISP parameter value is NEW.
3. Parameters that are coded on the DD statement that the REFDD parameter refers to or attributes of the data set that the LIKE parameter refers to.
4. The DCB subparameter of the DD statement or with other keywords.
5. The processing program, that is, the DCB macro instruction in assembler language programs or file definition statements or language-defined defaults in programs in other languages.

Therefore, if you supply information for the same DCB field in your processing program and on a DD statement, the system ignores the DD DCB subparameter. If a DD statement and the data set label supply information for the same DCB field, the system ignores the data set label information.

If you override attributes for a data set that your program is reading, then it does not affect the actual data. The system does not convert data to match the parameters when reading.

Note: When concatenated data sets are involved, the DCB is completed based on the type of data set and how the processing program uses the data set. See *z/OS DFSMS Using Data Sets* for more information.

Relationship to other parameters

See the descriptions of the individual DCB subparameters for the DD parameters and DCB subparameters that should not be coded with a specific DCB subparameter.

Do not code the following parameters with the DCB parameter.

```
AMP
DYNAM
```

With the DDNAME parameter, code **only** the BLKSIZE, BUFNO, and DIAGNS DCB subparameters.

With the QNAME parameter, code **only** the BLKSIZE, LRECL, OPTCD, and RECFM DCB subparameters.

The DD parameter KEYLEN and DCB subparameters KEYLEN, MODE, PRTSP, STACK, and TRTCH apply to specific device types. If you specify one of these subparameters on a DD statement for a device different from the type to which it applies, the system interprets the value incorrectly.

With the SPACE parameter, the value specified for BLKSIZE directly affects the amount of space obtained for data sets allocated in records, and for data sets allocated in blocks where the block length (blklgth) is zero.

For 3540 Diskette Input/Output Units: The VOLUME=SER, DCB=BUFNO, and DSID parameters on a DD * or DD DATA statement are ignored except when they are detected by a diskette reader as a request for an associated data set.

Examples of the DCB parameter

Example 1

```
//DD1   DD    DSNAME=ALP,DISP=(,KEEP),VOLUME=SER=44321,
//            UNIT=3400-6,DCB=(RECFM=FB,LRECL=240,BLKSIZE=960,
//            DEN=1,TRTCH=C)
```

DD statement DD1 defines a new data set named ALP. The DCB parameter contains the information necessary to complete the data control block.

Example 2

```
//DD1A  DD    DSNAME=EVER,DISP=(NEW,KEEP),UNIT=3380,
//            DCB=(RECFM=FB,LRECL=326,BLKSIZE=23472),
//            SPACE=(23472,(200,40))
```

DD statement DD1A defines a new data set named EVER on a 3380. The DCB parameter contains the information necessary to complete the data control block.

```
//DD1B  DD    DSNAME=EVER,DISP=(NEW,KEEP),UNIT=3380,
//            RECFM=FB,LRECL=326,
//            SPACE=(23472,(200,40))
```

DD statement DD1B is the same as the DD1A statement except that it shows the alternate syntax for the DCB keyword subparameters. Also, because BLKSIZE is omitted, the system will select an optimum block size for the data.

Example 3

```
//DD2   DD    DSNAME=BAL,DISP=OLD,DCB=(RECFM=F,LRECL=80,
//            BLKSIZE=80)
//DD3   DD    DSNAME=CNANN,DISP=(,CATLG,DELETE),UNIT=3400-6,
//            LABEL=(,NL),VOLUME=SER=663488,DCB=*.DD2
```

DD statement DD3 defines a new data set named CNANN and requests that the system copy the DCB subparameters from DD statement DD2, which is in the same job step.

Example 4

```
//DD4   DD    DSNAME=JST,DISP=(NEW,KEEP),UNIT=SYSDA,
//            SPACE=(CYL,(12,2)),DCB=(A.B.C,KEYLEN=8)
```

DD statement DD4 defines a new data set named JST and requests that the system copy the DCB information from the data set label of the cataloged data set named A.B.C. If the data set label contains a key length specification, it is overridden by the KEYLEN coded on this DD statement.

Example 5

```
//DD5   DD    DSNAME=SMAE,DISP=OLD,
//            DCB=(*.STEP1.PROCSTP5.DD8,BUFNO=5)
```

DD statement DD5 defines an existing, cataloged data set named SMAE and requests that the system copy DCB subparameters from DD statement DD8, which is contained in the procedure step named

PROCSTP5. The cataloged procedure is called by EXEC statement STEP1. Any of the DCB subparameters coded on DD statement DD8 are ignored if they are specified in the program. If the DCB BUFNO subparameter is not specified in the program, five buffers are assigned.

DCB subparameters

DCB subparameters	Access method								Description of subparameters
	BDAM	BPAM	BSAM	BTAM	EXCP	GAM	QSAM	TCAM	
BFALN	X	X	X		X		X		BFALN={F|D} Specifies that each buffer starts either on a word boundary that is not also a doubleword boundary or on a doubleword boundary. If both BFALN and BFTEK are specified, they must be specified from the same source. Default: D (doubleword) **Note:** Do not code the BFALN subparameter with DCB subparameter GNCP, or with DD parameters DDNAME or QNAME.
BFTEK	X		X	X			X		**BDAM and BSAM**: BFTEK=R **R** Specifies that the data set is being created for or contains variable-length spanned records. Do **not** specify R for a PDSE. **BTAM**: BFTEK=D **D** Specifies that dynamic buffering is to be used in the processing program; if dynamic buffering is specified, a buffer pool also must be defined. **QSAM**: BFTEK={S|A} **S** Specifies simple buffering (default). Simple buffering may be coded at any time for QSAM files. **A** Specifies locate mode logical record interface for spanned records. QSAM obtains a logical record area and assembles the physical record segments of a spanned record into that logical record area. This forms a complete logical record before pointing the user to it. • This parameter value may be specified only for RECFM=VS or RECFM=VBS files; if specified without RECFM=VS|VBS, the specification is ignored. • Locate mode must be used together with this parameter value. **Note:** If you use locate mode on a RECFM=VS|VBS file and BFTEK=A is **not** specified, the results may include processing the segments of a spanned record as separate records, issuance of system completion (abend) code X'002' with reason code X'04', or other unpredictable results. • If you specify BFTEK=A with move mode, a system completion (abend) code X'013' with reason code X'5C' is issued. For information about the locate and move modes in the DCB subparameters BFTEK and VBS, see *z/OS DFSMS Macro Instructions for Data Sets*. If you specify both BFALN and BFTEK, you must specify them from the same source. **Note:** Do not code the BFTEK subparameter with DCB subparameter GNCP, or with DD parameters DDNAME or QNAME. **Note:** For compatibility purposes with previous operating systems, the system accepts BFTEK=E.

DCB subparameters	Access method								Description of subparameters
	BDAM	BPAM	BSAM	BTAM	EXCP	GAM	QSAM	TCAM	
BLKSIZE	X	X	X		X		X	X	BLKSIZE={value|valueK|valueM|valueG} Specifies the maximum length, in bytes, of a block. **value** Specifies the maximum length of a block. The number you specify for BLKSIZE depends on the device type and the record format for the data set. The maximum is 32,760 for DASD data sets and 2,147,483,648 for tape, except for data sets on magnetic tape with ISO/ANSI/FIPS labels, where the minimum value for BLKSIZE is 18 bytes and the maximum is 2048 bytes. (To allow a value greater than 2048, use installation exit routine IFG0193G, described in *z/OS DFSMS Installation Exits*.) **valueK** Specifies the maximum length, in kilobytes, of a block. (1 kilobyte = 1024 bytes.) The maximum is 2097152. If 2097152K is coded, the block size assigned will be the maximum: 2,147,483,648. **valueM** Specifies the maximum length, in megabytes, of a block. (1 megabyte = 1024 kilobytes.) The maximum is 2048. If 2048M is coded, the block size assigned will be the maximum: 2,147,483,648. **valueG** Specifies the maximum length, in gigabytes, of a block. (1 gigabyte = 1024 megabytes.) The maximum is 2G. If 2G is coded, the block size assigned will be the maximum: 2,147,483,648. If you code the BLKSIZE subparameter in the DCB macro instruction or on a DD statement that defines an existing data set with standard labels, the DCB BLKSIZE overrides the block size specified in the label. BLKSIZE can be coded but will have no effect on EXCP processing. The number you specify for BLKSIZE directly affects the amount of space obtained for data sets allocated in records, and for data sets allocated in blocks where the block length (blklgth) is zero. Default: If you do not code BLKSIZE, the system can, under certain conditions, determine an optimum block size for the data. For detailed information about system-determined block size, see *z/OS DFSMS Using Data Sets*. **Note:** Do not code the BLKSIZE subparameter with the BUFSIZE subparameter.
BUFIN								X	BUFIN=buffers Specifies the number of buffers to be assigned initially for receiving operations for each line in the line group. The combined BUFIN and BUFOUT values must not be greater than the number of buffers in the buffer pool for this line group (not including those for disk activity only). Default: 1 **Note:** Do not code the BUFIN subparameter with DCB subparameter BUFNO, or DD parameters DDNAME, QNAME.
BUFL	X	X	X		X		X	X	BUFL=bytes Specifies the length, in bytes, of each buffer in the buffer pool. The maximum is 32,760. **Note:** Do not code the BUFL subparameter with DD parameter DDNAME.
BUFMAX								X	BUFMAX=buffers Specifies the maximum number of buffers to be allocated to a line at one time. Number must be 2 through 15 and must be equal to or greater than the larger of the numbers specified by the BUFIN and BUFOUT subparameters. Default: 2 **Note:** Do not code the BUFMAX subparameter with DCB subparameter NCP, or DD parameters DDNAME, QNAME.
BUFNO	X	X	X	X	X		X		BUFNO=buffers Specifies the number of buffers to be assigned to the DCB. The maximum normally is 255, but can be less because of the size of the region. **Note:** Do not code the BUFNO subparameter with DCB subparameters BUFIN, BUFOUT, or DD parameter QNAME.

DCB subparameters	Access method								Description of subparameters
	BDAM	BPAM	BSAM	BTAM	EXCP	GAM	QSAM	TCAM	
BUFOFF			X				X		BUFOFF={n|L} **n** Specifies the length, in bytes, of the block prefix used with an ASCII tape data set. For input, n can be 0 through 99. For output, n must be 0 for writing an output data set with fixed-length or undefined-length records. **L** Specifies that the block prefix is 4 bytes and contains the block length. BUFOFF=L is valid only with RECFM=D. For output, only BUFOFF=L is valid. **Note:** Do not code the BUFOFF subparameter with DD parameters DDNAME, QNAME.
BUFOUT								X	BUFOUT=buffers Specifies the number of buffers to be assigned initially for sending operations for each line in the line group. The combined number of BUFIN and BUFOUT values must not be greater than the number of buffers in the buffer pool for this line group (not including those for disk activity only) and cannot exceed 15. Default: 2 **Note:** Do not code the BUFOUT subparameter with DCB subparameter BUFNO, or DD parameter DDNAME.
BUFSIZE								X	BUFSIZE=bytes Specifies the length, in bytes, of each of the buffers to be used for all lines in a particular line group. Length must be 31 through 65535 bytes. **Note:** Do not code the BUFSIZE subparameter with DCB subparameter BLKSIZE, or DD parameters DDNAME, QNAME.
CPRI								X	CPRI={R|E|S} Specifies the relative transmission priority assigned to the lines in this line group. **R** Specifies that processor receiving has priority over processor sending. **E** Specifies that receiving and sending have equal priority. **S** Specifies that processor sending has priority over processor receiving. **Note:** Do not code the CPRI subparameter with DCB subparameter THRESH, or DD parameters DDNAME, OUTLIM, QNAME.
CYLOFL									CYLOFL=tracks Specifies the number of tracks on each cylinder to hold the records that overflow from other tracks on that cylinder. The maximum is 99. Specify CYLOFL only when OPTCD=Y. **Note:** Do not code the CYLOFL subparameter with DCB subparameter RESERVE, or DD parameters DDNAME, FCB, QNAME, UCS.
DEN			X		X		X		DEN={1|2|3|4} Specifies the magnetic density, in number of bits-per-inch, used to write a magnetic tape data set. **DEN 7-track tape 9-track tape** 1 556 - 2 800 800 (NRZI) 3 - 1600 (PE) 4 - 6250 (GCR) **NRZI** Non-return-to-zero inverted recording mode. **PE** Phase encoded recording mode. **GCR** Group coded recording mode. **Default:** 800 bpi assumed for 7-track tape and 9-track without dual density. 1600 bpi assumed for 9-track with dual density or phase-encoded drives. 6250 bpi assumed for 9-track with 6250/1600 bpi dual density or group coded recording tape. **Note:** Do not code the DEN subparameter with DD parameters DDNAME, QNAME.

DCB subparameters	Access method								Description of subparameters
	BDAM	BPAM	BSAM	BTAM	EXCP	GAM	QSAM	TCAM	
DIAGNS	X	X	X	X	X	X	X		DIAGNS=TRACE Specifies the OPEN/CLOSE/EOV trace option, which gives a module-by-module trace of OPEN/CLOSE/EOV's work area and the DCB. If the generalized trace facility (GTF) is not running and tracing user events, DIAGNS is ignored. See *z/OS DFSMSdfp Diagnosis* for more information.
DSORG	X	X	X	X	X	X	X	X	DSORG=organization Specifies the organization of the data set and indicates whether the data set contains any location-dependent information that would make the data set unmovable. **Note:** Do not code the DSORG subparameter with DD parameters DDNAME, QNAME, RECORG. **Organization** | **Access Method** PS Physical sequential data set | BSAM,EXCP,QSAM, PSU Physical sequential data set that contains location-dependent information | BSAM,QSAM,EXCP DA Direct access data set | BDAM,EXCP DAU Direct access data set that contains location-dependent information | BDAM,EXCP PO Partitioned data set (PDS or PDSE) | BPAM,EXCP POU Partitioned data set (PDS) that contains location-dependent information | BPAM,EXCP CX Communications line group | BTAM GS Graphic data control block | GAM
EROPT				X			X		EROPT=x **BTAM:** Requests the BTAM on-line terminal test option. x=T **QSAM:** Specifies the option to be executed if an error occurs in reading or writing a record. **x=ACC** System is to accept the block causing the error. **x=SKP** System is to skip the block causing the error. **x=ABE** System is to cause abnormal end of task. **Default** ABE **Note:** Do not code the EROPT subparameter with DD parameters DDNAME, QNAME.

DCB subparameters	Access method								Description of subparameters
	BDAM	BPAM	BSAM	BTAM	EXCP	GAM	QSAM	TCAM	

FUNC			X				X		FUNC={I|R|P|W|D|X|T}
									Specifies the type of data set to be opened for a 3505 Card Reader or 3525 Card Punch. Unpredictable results will occur if coded for other than a 3505 or 3525.
									I
									Data set is for punching and printing cards.
									R
									Data set is for reading cards.
									P
									Data set is for punching cards.
									W
									Data set is for printing.
									D
									Protected data set is for punching.
									X
									Data set is for both punching and printing.
									T
									Two-line print option.
									The only valid combinations of these values are:
									`I WT RWT RPWXT PWX`
									`R RP PW RPWD RPWX`
									`P RPD PWXT RWX RWX`
									`W RW RPW RWXT`
									Default: P, for output data set. R, for input data set.
									Note: Do not code the FUNC subparameter with the data-set-sequence number of the DD LABEL parameter, or DD parameters DDNAME, QNAME.

GNCP						X			GNCP=n Specifies the maximum number of I/O macro instructions that the program will issue before a WAIT macro instruction. **Note:** Do not code the GNCP subparameter with DCB subparameters BFALN, BFTEK, or DD parameters DDNAME, QNAME.
INTVL								X	INTVL={n|0} Specifies the interval, in seconds, between passes through an invitation list. Default: 0 **Note:** Do not code the INTVL subparameter with DD parameters DDNAME, FCB, QNAME, UCS.
IPLTXID								X	IPLTXID=member Specifies the name of the partitioned data set (PDS) member that you want loaded into a 3704/3705 Communications Controller. The DCB IPLTXID subparameter overrides IPLTXID in the TERMINAL macro representing the NCP. **Note:** Do not code the IPLTXID subparameter with DD parameters DDNAME, DSNAME, QNAME.
KEYLEN	X	X	X		X			X	KEYLEN=bytes The KEYLEN keyword subparameter is described on the DD KEYLEN parameter, section "KEYLEN parameter" on page 197.
LIMCT	X								LIMCT={blocks|tracks} Specifies how many blocks (if relative block addressing is used) or how many tracks (if relative track addressing is used) are to be searched for a free block or available space. This kind of search occurs only when DCB OPTCD=E is also specified; otherwise, LIMCT is ignored. If the LIMCT number equals or exceeds the number of blocks or tracks in the data set, the entire data set is searched. **Note:** Do not code the LIMCT subparameter with DD parameters DDNAME, QNAME.
LRECL		X	X		X		X	X	LRECL=bytes The LRECL keyword subparameter is described on the DD LRECL parameter, section "LRECL parameter" on page 208.

DCB subparameters	Access method								Description of subparameters
	BDAM	BPAM	BSAM	BTAM	EXCP	GAM	QSAM	TCAM	

MODE			X		X		X		`MODE= {C [O]}`
									`{E [R]}`
									Specifies the mode of operation to be used with a card reader, a card punch, or a card read-punch.
									C Card image (column binary) mode
									E EBCDIC mode
									O Optional mark read mode
									R Read column eliminate mode
									If you specify R, you must also specify either C or E. Do not code the MODE subparameter for data entered through the input stream except in a JES3 system.
									Do not code MODE=C for JES2 or JES3 output.
									Default: E
									Note: Do not code the MODE subparameter with DCB subparameters KEYLEN, PRTSP, TRTCH, or DD parameters DDNAME, KEYLEN, QNAME.

NCP		X	X						NCP=n Specifies the maximum number of READ or WRITE macro instructions that may be issued before a CHECK macro instruction is issued to test for completion of the I/O operation. The maximum number is 255 for BSAM and BPAM, but may actually be smaller depending on the size of the address space. If chained scheduling is used, the number should be greater than 1. Default: 1 **Note:** Do not code the NCP subparameter with DCB subparameter BUFMAX, or DD parameters DDNAME, QNAME.
NTM									NTM=tracks Specifies the number of tracks to be used for a cylinder index. When the specified number of tracks has been filled, a master index is created. The DCB NTM is needed only when the DCB OPTCB=M. If you specify OPTCD=M but omit NTM, the master index option is ignored. **Note:** Do not code the NTM subparameter with DCB subparameter PCI, or DD parameters DDNAME, QNAME.

OPTCD	X	X	X		X		X	X	Specifies the optional services to be performed by the control program. All optional services must be requested in one source, that is, in the data set label of an existing data set, in the DCB macro, or in the DD DCB parameter. However, the processing program can modify the DCB OPTCD field. Code the characters in any order; when coding more than one, do not code commas between the characters.
									Note: Do not code the OPTCD subparameter with DD parameter DDNAME.
									`BDAM: OPTCD= {A}[E][F][W]`
									`{R}`
									A indicates that the actual device addresses are to be specified in READ and WRITE macro instructions.
									R indicates that relative block addresses are to be specified in READ and WRITE macro instructions.
									E indicates that an extended search (more than one track) is to be performed for a block of available space. LIMCT must also be coded. Do not code LIMCT=0 because it will cause an abnormal termination when a READ or WRITE macro instruction is executed.
									F indicates that feedback can be requested in READ and WRITE macro instructions and the device is to be identified in the same form as it was presented to the control program.
									W requests a validity check for write operations on direct access devices.

DCB subparameters	Access method								Description of subparameters
	BDAM	BPAM	BSAM	BTAM	EXCP	GAM	QSAM	TCAM	
OPTCD (continued)									BPAM: OPTCD= {C|W|CW} **C** has no effect. **W** requests a validity check for write operations. BSAM and QSAM: OPTCD= {B / T / U[C] / C[T][B][U] / H[Z][B] / J[C][U] / W[C][T][B][U] / Z[C][T][B][U] / Q[C][T][B] / Z} **B** requests that the end-of-volume (EOV) routine disregard the end-of-file (EOF) recognition for magnetic tape. For an input data set on a standard-labeled (SL or AL) tape, the EOV routine treats EOF labels as EOV labels until the volume serial list is exhausted. This option allows SL or AL tapes to be read out of volume sequence or to be concatenated to another tape with the same data set name using one DD statement. See "Data Sets that Span Libraries" in *z/OS MVS JCL User's Guide* for a description of allocation processing for multi-volume data sets created in different tape libraries. **C** requests chained scheduling. **H** requests hopper empty exit for optical readers or bypass of DOS checkpoint records. **J** for a data set to be printed on an AFP printer, instructs the system that the logical record for each output data line contains a table reference character (TRC). The TRC identifies which character arrangement table in the CHARS parameter is to be used to print the line. Before specifying OPTCD=J, see *PSF for z/OS: User's Guide*.
OPTCD (continued)									BSAM and QSAM (continued): **Q** indicates that all the user data in the data set is in ASCII. BSAM or QSAM converts the records from ASCII to EBCDIC when reading and converts the records from EBCDIC to ASCII when writing. The data set must reside on magnetic tape and must not contain IBM standard labels. The record format (RECFM) must not be V but can be D. If the label type is ISO/ANSI/FIPS, specified as LABEL=(,AL), the system forces OPTCD=Q. **T** requests user totaling facility. T cannot be specified for a SYSIN or sysout data set. **U** for 1403 or 3211 Printers with the Universal Character Set (UCS) feature and for the 3800, permits data checks and allows analysis by an appropriate error analysis routine. If U is omitted, data checks are not recognized as errors. **W** requests a validity check for write operations on direct access devices. Requests "tape write immediate" mode on a cartridge tape device such as the IBM 3490 Magnetic Tape Subsystem. **Z** for magnetic tape reel input, requests that the control program shorten its normal error recovery procedure. When specified, a data check is considered permanent after five unsuccessful attempts to read a record. OPTCD=Z is ignored if chained scheduling or a tape cartridge is used. For a PDSE, all options except OPTCD=J are ignored. EXCP: OPTCD=Z **Z** for magnetic tape reel input, requests that the control program shorten its normal error recovery procedure. When specified, a data check is considered permanent after five unsuccessful attempts to read a record. OPTCD=Z has no effect on a tape cartridge.

DCB subparameters	Access method								Description of subparameters
	BDAM	BPAM	BSAM	BTAM	EXCP	GAM	QSAM	TCAM	
PCI								X	PCI= {([N][,N])} {([R][,R])} {([A][,A])} {([X][,X])} Specifies (1) whether or not a program-controlled interruption (PCI) is to be used to control the allocation and freeing of buffers and (2) how these operations are to be performed. The first operand applies to receiving operations and the second to sending operations. **N** specifies that no PCIs are taken while filling buffers during receiving operations or emptying buffers during sending operations. **R** specifies that after the first buffer is filled or emptied, a PCI occurs during the filling or emptying of each succeeding buffer. The completed buffer is freed, but no new buffer is allocated to take its place. **A** specifies that after the first buffer is filled or emptied, a PCI occurs during the filling or emptying of the next buffer. The first buffer is freed, and a buffer is allocated to take its place. **X** specifies that after a buffer is filled or emptied, a PCI occurs during the filling or emptying of the next buffer. The first buffer is not freed, but a new buffer is allocated. You can omit the parentheses if you code only the first operand. Default: (A,A) **Note:** Do not code the PCI subparameter with DCB subparameter NTM, or DD parameters DDNAME, QNAME.
PRTSP			X		X		X		PRTSP={0|1|2|3} Specifies the line spacing for an online printer. PRTSP is valid only for an online printer and only if the RECFM is not A or M. PRTSP=2 is ignored if specified with the DD SYSOUT parameter. 0 - spacing is suppressed, 1 - single, 2 - double, 3 - triple spacing JES2 ignores PRTSP for sysout data sets. Default: 1 **Note:** Do not code the PRTSP subparameter with DCB subparameters KEYLEN, MODE, STACK, TRTCH, or DD parameters DDNAME, KEYLEN, QNAME.
RECFM	X	X	X		X		X	X	RECFM=format The RECFM keyword subparameter is described on the DD RECFM parameter, section "RECFM parameter" on page 231.
RESERVE								X	RESERVE=(bytes1,bytes2) Specifies the number of bytes (0 through 255) to be reserved in a buffer for insertion of data by the DATETIME and SEQUENCE macros. **bytes1** indicates the number of bytes to be reserved in the first buffer that receives an incoming message. **bytes2** indicates the number of bytes to be reserved in all the buffers following the first buffer in a multiple-buffer header situation. Default: (0,0) **Note:** Do not code the RESERVE subparameter with DCB subparameters CYLOFL, RKP, or DD parameters DDNAME, KEYOFF, QNAME, UCS.

DCB subparameters	Access method								Description of subparameters
	BDAM	BPAM	BSAM	BTAM	EXCP	GAM	QSAM	TCAM	
RKP					X				RKP=number With SMS, use the DD KEYOFF or DATACLAS parameter. Specifies the position of the first byte of the record key in each logical record. The first byte of a logical record is position 0. If RKP=0 is specified for blocked fixed-length records, the key begins in the first byte of each record. OPTCD=L must not be specified. If RKP=0 is specified for unblocked fixed-length records, the key is not written in the data field. OPTCD=L can be specified. For variable-length records, the relative key position must be 4 or greater, if OPTCD=L is not specified; the relative key position must be 5 or greater, if OPTCD=L is specified. For EXCP processing, RKP can be coded but is ignored. Default: 0 **Note:** Do not code the RKP subparameter with DCB subparameter RESERVE, or DD parameters DDNAME, FCB, KEYOFF, UCS.
STACK			X		X		X		STACK={1|2} Specifies which stacker bin is to receive a card. Default: 1 **Note:** Do not code the STACK subparameter with DCB subparameters KEYLEN, PRTSP, TRTCH, or DD parameters DDNAME, KEYLEN, QNAME.
THRESH								X	THRESH=nn Specifies the percentage of the nonreusable disk message queue records that are to be used before a flush closedown occurs. Default: Closedown occurs when 95 percent of the records have been used. **Note:** Do not code the THRESH subparameter with DCB subparameter CPRI, or DD parameters DDNAME, OUTLIM, QNAME.

In this example, the system sends the sysout data set defined by DD statement DEBIT to the work station that submitted the job, the data set defined by DD statement CALIF to the remote terminal 555, and the data set defined by DD statement FLOR to VM userid 9212U28 at node BOCA.

DISP parameter

Parameter Type

Keyword, optional

Purpose

Use the DISP parameter to describe the status of a data set to the system and tell the system what to do with the data set after termination of the step or job. You can specify one disposition for normal termination and another for abnormal termination. Note that if an abend occurs within a step, the step itself may terminate normally or abnormally, depending on whether an ESTAE (or ESPIE routine) intercepts the abend and requests that processing continue. Normal termination of a step is indicated by the presence of message IEF142I; abnormal termination of a step is indicated by the presence of message IEF472I.

Note: Disposition of the **data set** is controlled solely by the DISP parameter; disposition of the **volume(s)** on which the data set resides is a function of the volume status when the volume is demounted. If the UNIT parameter specifies a device, such as a printer or telecommunications device, that does not involve a data set, do not code the DISP parameter.

If the system obtains unit and volume information for an OLD, MOD, or SHR status, the data set is treated as if it exists, whether or not it is physically on the device.

When any step of a job requests exclusive control of a data set, with an exception of when the job is allowed to downgrade ENQs via the DSENQSHR specification, the system converts all requests for shared control of that data set within that job (DISP=SHR) to requests for exclusive control. One of two methods can be used to request exclusive control:

- DISP=NEW, DISP=MOD, or DISP=OLD on a JCL request.
- DISP=NEW, DISP=MOD, or DISP=OLD on a dynamic allocation request, including dynamic allocation requests that result from the use of certain utility control statements. For example, utility control statements that delete/scratch a data set will result in exclusive use of that data set.

If a dynamic allocation requests exclusive control of a data set then all subsequent DISP=SHR JCL references to that data set within that job will be upgraded to exclusive control. The job will retain exclusive control of that data set until the end of the last step of that job which references that data set in its JCL. For example:

```
//STEP1  EXEC PGM=anypgm1
//DD1    DD  DSN=A.B.C,DISP=SHR
//STEP2  EXEC PGM=IDCAMS
   DELETE  A.B.C
   DEFINE  A.B.C
//STEP3  EXEC PGM=anypgm3
//DD3    DD  DSN=A.B.C,DISP=SHR
//STEP4  EXEC PGM=anypgm4
```

Before the start of STEP1, the job will request shared control of data set A.B.C. In STEP2, the DELETE/DEFINE of data set A.B.C will cause the shared control from STEP1 to be upgraded to exclusive control. In STEP3, control of data set A.B.C will remain exclusive, since it is not possible to downgrade an ENQ from EXCL to SHR in the case that a dynamic allocation upgrades the ENQ. At the end of STEP3, control of data set A.B.C will be released. The ENQ on data set A.B.C will therefore not be held at all during STEP4. If, however, the job also contained a STEP5 which requested use of data set A.B.C, then exclusive control of that data set would continue to be held by the job all the way through both STEP4 and STEP5.

DISP and ENQ: Before starting the first step of a job, the initiator requests control of all of the data sets in that job by issuing an ENQ for each of them, using the value specified for DISP to determine the kind of ENQ issued. The initiator issues the ENQ for each data set at the highest level required for that data set by

any step of the job. For example, if all steps of the job request shared control of a specific data set (DISP=SHR) then the ENQ for that data set is requested as SHR. If, on the other hand, *any* step of the job requests exclusive control of a specific data set (DISP=NEW, DISP=MOD, or DISP=OLD), then the ENQ for that data set is requested EXCL.

If the job is allowed to downgrade ENQs, then ENQs will be downgraded from exclusive control to shared control at the last step for which the data set is referenced EXCL. The downgrade will not occur if the ENQ has been upgraded by a dynamic allocation request in the current or a previous step.

If the job is not allowed to downgrade ENQs, then ENQs cannot be downgraded from EXCL to SHR. If one step needs the ENQ EXCL and a following step only needs it SHR, the ENQ is still held as EXCL.

In both cases, the ENQ is held until the end of the last step which references that data set, at which point the ENQ is released entirely.

DISP and ENQ for generation data sets: The way the initator issues an ENQ to control generation data sets can be different than with other data sets. The initiator only issues the ENQ for the GDG base name for a generation data set that is referenced by either:

- Their relative GDG names (for example, DSN=TEST.GDG.DATASET(0)).
- As GDG ALLs (for example, DSN=TEST.GDG.DATASET)

For example, the initiator issues the ENQ for the GDG base name, TEST.GDG.DATASET for the generation data sets shown in the previous list. Generation data sets referenced by either their relative GDG names or as GDG ALLs are processed this way because the initiator does not know which specific absolute generation names will be required. This is because the conversion from relative generation name to absolute generation name is done during the allocation for the step referencing the data set. The ENQ for the specific, absolute, generation (G0000V00) data set name or names is issued at the start of the step requesting the relative GDG or GDG ALL.

Note that the initiator does not issue an ENQ for the GDG base name for a generation data set that is referenced by its absolute GDG name. Instead it issues an ENQ for that specific G0000V00 data set name.

References

For information about tape data set processing, see *z/OS DFSMS Using Magnetic Tapes*.

Syntax

```
{DISP=[status]                                                    }
{DISP=([status][,normal-termination-disp][,abnormal-termination-disp])}

DISP= ( [NEW] [,DELETE ] [,DELETE ] )
        [OLD] [,KEEP   ] [,KEEP   ]
        [SHR] [,PASS   ] [,CATLG  ]
        [MOD] [,CATLG  ] [,UNCATLG]
        [,  ] [,UNCATLG]
              [,       ]
```

- You can omit the parentheses if you code only the status subparameter.
- If you omit the status subparameter but code subparameters for normal or abnormal termination disposition, you must code a comma to indicate the absence of NEW. For example, DISP=(,KEEP) or DISP=(,CATLG,DELETE).
- If you omit the second subparameter but code the third, you must code a comma to indicate the absence of the second subparameter. For example, DISP=(OLD,,DELETE) or DISP=(,,KEEP).

Subparameter definition

Status subparameter

NEW
Indicates that a new data set is to be created in this step.

OLD
Indicates that the data set exists before this step and that this step requires exclusive (unshared) use of the data set.

If you specify DISP=OLD for an output tape data set and (1) the data set is not protected by RACF or a password or (2) the data set has no expiration date, the system does not verify the data set name in the header label.

SHR
Indicates that the data set exists before this step and that other jobs can share it, that is, use it at the same time. This subparameter can also be coded as SHARE.

If you specify DISP=SHR for an output tape data set and (1) the data set is not protected by RACF or a password or (2) the data set has no expiration date, the system does not verify the data set name in the header label.

MOD
Indicates one of the following:

- The data set exists and records are to be added to the end of it. The data set must be sequential.
- A new data set is to be created.

In either case, MOD specifies exclusive (unshared) use of the data set.

When the data set is opened, the read/write mechanism is positioned after the last sequential record for an existing data set or at the beginning for a new data set. For subsequent OPENs within the same step, the read/write mechanism is positioned after the last sequential record.

Note: You cannot specify DISP=MOD to extend an ISO/ANSI/FIPS Version 3 tape data set unless the ISO/ANSI/FIPS Version 3 label validation installation exit allows the extension. For information on using ISO/ANSI/FIPS Version 3 installation exits, see *z/OS DFSMS Using Magnetic Tapes*.

If the system cannot find volume information for the data set on the DD statement, in the catalog, or passed with the data set from a previous step, the system assumes that the data set is being created in this job step. For a new data set, MOD causes the read/write mechanism to be positioned at the beginning of the data set.

To use DISP=MOD to create a new data set, code one of the following:

- No VOLUME=SER or VOLUME=REF parameter on the DD statement. The data set must not be cataloged or passed from another job step.
- A VOLUME=REF parameter that refers to a DD statement that makes a nonspecific volume request. (A nonspecific volume request is a DD statement for a new data set that can be assigned to any volume or volumes.) If it is tape, the referenced DD must not be opened before the DD with VOL=REF. If it is opened later, it will get a different tape volume. If it is tape and the referenced DD is in a different step it must not be opened before the DD with VOL=REF. If the referenced DD is for a nonspecific volume in the same step then the DD with VOL=REF will always be nonspecific regardless of the order of OPENs in the step. For DASD, one of the following must also be true:
  - The DSNAME parameters in the two DD statements must be different.
  - The two DD statements must request different areas of the same ISAM data set.
- In the case of tape, if you do not specify an explicit volume serial number on the DD statement, then you cannot specify a file sequence number greater than 1 and the system requests the operator to mount a "scratch" tape.

  For a new generation of a generation data group (GDG) data set (where (+n) is greater than 0), you may code VOLUME=REF or VOLUME=SER.

  For an SMS-managed data set the system ignores the volume.

After the system chooses a volume for a new data set, if the system finds another data set with the same name on that volume, the system will try to allocate a different volume. However, SMS-managed data sets require unique data set names. If a new data set is chosen to be SMS-managed and an existing SMS-managed data set has the same name, the request fails.

In a JES3 system, if you code DISP=MOD for a multivolume data set and any of the volumes are JES3-managed, JES3 will not execute the job until all volumes, including scratch volumes being added, are allocated. Such a job will wait on the queue until all volumes are allocated.

To use DISP=MOD to extend an existing data set, code one of the following:

- If the data set is cataloged, do not code a VOLUME=SER or a VOLUME=REF parameter on the DD statement, but code DISP=MOD or DISP=(MOD,KEEP) to make sure that the catalog will be updated with any new volume serial numbers.
- If the data set did not exist at the beginning of the job, but was passed from a prior step and not cataloged, it will be deleted at step termination. However, if you want to keep the data set, you can code DISP=(MOD,KEEP).

Normal termination disposition subparameter

DELETE

Indicates that the data set is no longer needed if this step terminates normally.

For a DASD data set, DELETE means that the space occupied by that data set is available for use by other data sets. The system will physically erase the data set itself *only* if the erase option of a security product, such as RACF, is in effect for this data set. If the erase option is not in effect, the data will remain on the DASD until overwritten by another data set. For information on how to set the erase option, see the documentation for the security product.

For a tape data set, DELETE does not physically erase the data from the tape volume. The data will remain on the tape until overwritten by another data set. If the tape volume is a public volume, specifying DELETE allows the system to reuse the tape volume for other data sets that require a public volume; the system may overwrite the data set.

Note: DELETE requests are always treated as requiring exclusive serialization, preventing other jobs using the data set until it is deleted. This occurs even when DISP=(SHR,DELETE) is coded. As such, data sets with DISP=(SHR,DELETE) will not have its control changed, even when DSENQSHR=ALLOW is specified on the JOB card or in the JOBCLASS.

Existing data sets:

- If you set a retention period on the DD RETPD parameter, an existing data set is deleted only if its retention period is passed; otherwise the data set is kept.
- If you set an expiration date on the DD EXPDT parameter, an existing data set is deleted if the expiration date has passed.

  If the storage administrator specified OVRD_EXPDT(YES) in the IGDSMSxx member of SYS1.PARMLIB, you can override the expiration date or retention period for SMS-managed data sets by specifying DELETE on the DD DISP parameter. In that case, the data set will be deleted whether or not the expiration date or the retention period has passed. See *z/OS MVS Initialization and Tuning Reference* for information on the IGDSMSxx parmlib member.

New data sets:

A new data set is deleted at the end of the step even though a retention period or expiration date is also specified. See the DD EXPDT or RETPD parameters.

If the system retrieves volume information from the catalog because the DD statement does not specify VOLUME=SER or VOLUME=REF, then DELETE implies UNCATLG: the system deletes the data set and removes its catalog entry.

KEEP

Indicates that the data set is to be kept on the volume if this step terminates normally.

Without SMS, only KEEP is valid for VSAM data sets. VSAM data sets should not be passed, cataloged, uncataloged, or deleted.

With SMS, all dispositions are valid for VSAM data sets; however, UNCATLG is ignored.

For new SMS-managed data sets, KEEP implies CATLG.

PASS
Indicates that the data set is to be passed for use by a subsequent step in the same job.

With SMS, the system replaces PASS with KEEP for existing VSAM data sets. When you refer to the data set later in the job, the system obtains data set information from the catalog.

Note:

1. A data set can be passed only within a job.
2. If you specify DISP=(NEW,PASS) but, at the end of the job, one or more data sets were not received by any job step, then the maximum number of DD statements you can specify decreases by one. (The size of the TIOT controls how many DD statements are allowed per job step.) For example, if the current limit is 1635 DD statements, you can specify DISP=(NEW,PASS), and up to 1634 DD statements.
3. Coding PASS does not ensure that the operator will not unload the volume or that the system will not demount it to accommodate another job step allocation. Either can occur when the device on which the volume is mounted is not allocated to the job step that specified PASS or, for unlabeled tapes, when the volume requires verification. If the system does demount a volume for which RETAIN was requested, it will do so by issuing message IEF234E R (retain) for that volume. When the system reaches the next step requiring that volume, it will request the operator to remount the volume on an available device of the appropriate type.

CATLG
Indicates that, if the step terminates normally, the system is to place an entry pointing to the data set in the catalog. Note that the data set is kept.

An unopened tape data set is cataloged, unless the volume request is nonspecific or unless the data set is allocated to a dual-density tape drive but no density is specified. A nonspecific volume request is a DD statement for a new data set that can be assigned to any volume or volumes.

For information about the rules for cataloged data set names, see *z/OS DFSMS Access Method Services Commands*.

UNCATLG
Indicates that, if the step terminates normally, the system is to delete (1) the entry pointing to the data set in the catalog and (2) unneeded indexes, except for the highest level entry. Note that the data set is kept.

With SMS, UNCATLG is ignored for SMS-managed data sets and VSAM data sets (KEEP is implied).

By default, if the system retrieves volume information from the catalog, UNCATLG is processed. If not, the UNCATLG request is rejected and UNCATLG is treated as KEEP. Volume information is not retrieved from the catalog when VOLUME=REF or VOLUME=SER is specified on the DD statement, or for new data sets. However, if the ALLOCxx Parmlib setting for SYSTEM VERIFY_UNCAT is TRACK, MSGTRACK, or LOGTRACK, the UNCATLG request is honored and the data set is uncataloged regardless of whether or not the volume information in the catalog matches the volume information that is used for the dataset.

Abnormal termination (conditional) disposition subparameter

DELETE
Indicates that the data set's space on the volume is to be released if this step terminates abnormally. The space can be used for other data sets; the data set is not erased from the space.

Note: DELETE requests are always treated as requiring exclusive serialization, preventing other jobs using the data set until it is deleted. This occurs even when DISP=(SHR,DELETE) is coded. As such, data sets with DISP=(SHR,DELETE) will not have its control changed, even when DSENQSHR=ALLOW is specified on the JOB card or in the JOBCLASS.

Existing data sets:

- If you set a retention period on the DD RETPD parameter, an existing data set is deleted only if its retention period is passed; otherwise the data set is kept.

- If you set an expiration date on the DD EXPDT parameter, an existing data set is deleted if the expiration date has passed.

  You can override the expiration date or retention period for SMS-managed DASD data sets using the OVRD_EXPDT(YES) parameter in the IGDSMSxx SYS1.PARMLIB member. In that case, the data set will be deleted whether or not the data set has expired or the retention period has passed. See *z/OS MVS Initialization and Tuning Reference* for information on the IGDSMSxx parmlib member.

New data sets:

A new data set is deleted at the end of the step even though a retention period or expiration date is also specified. See the DD EXPDT or RETPD parameters.

If the system retrieves volume information from the catalog because the DD statement does not specify VOLUME=SER or VOLUME=REF, then DELETE implies UNCATLG: the system deletes the data set and removes its catalog entry.

For a cataloged, passed data set, the catalog is not updated.

KEEP
Indicates that the data set is to be kept on the volume if this step terminates abnormally.

Without SMS, only KEEP is valid for VSAM data sets. VSAM data sets should not be passed, cataloged, uncataloged, or deleted.

With SMS, all dispositions are valid for VSAM data sets; however, UNCATLG is ignored.

For new SMS-managed data sets, KEEP implies CATLG.

CATLG
Indicates that, if the step terminates abnormally, the system is to place an entry pointing to the data set in the catalog. Note that the data set is kept.

An unopened tape data set is cataloged, unless the volume request is nonspecific or unless the data set is allocated to a dual-density tape drive but no density is specified.

For a cataloged, passed data set, the catalog is not updated. A passed, not received data set is not cataloged if the data set name has a first-level qualifier of a catalog name or alias.

UNCATLG
Indicates that, if this step terminates abnormally, the system is to delete (1) the entry pointing to the data set in the catalog and (2) unneeded indexes, except for the highest level entry. Note that the data set is kept.

For a cataloged, passed data set, the catalog is not updated.

With SMS, UNCATLG is ignored for SMS-managed data sets and VSAM data sets (KEEP is implied).

By default, if the system retrieves volume information from the catalog, UNCATLG is processed. If not, the UNCATLG request is rejected and UNCATLG is treated as KEEP. Volume information is not retrieved from the catalog when VOLUME=REF or VOLUME=SER is specified on the DD statement, or for new data sets. However, if the ALLOCxx Parmlib setting for SYSTEM VERIFY_UNCAT is TRACK, MSGTRACK, or LOGTRACK, the UNCATLG request is honored and the data set is uncataloged regardless of whether or not the volume information in the catalog matches the volume information that is used for the dataset.

Defaults

- If you omit the status subparameter, the default is NEW.
- If you omit the normal termination disposition subparameter, the default is DELETE for a NEW data set or KEEP for an existing data set.
- If you omit the abnormal termination disposition subparameter, the default is the disposition specified or implied by the second subparameter. However, if the second subparameter specified PASS, the default abnormal termination disposition is DELETE for a NEW data set or KEEP for an existing data set.

- If you omit the DISP parameter, the default is a NEW data set with a disposition of DELETE for both normal and abnormal termination disposition. Thus, you can omit the DISP parameter for a data set that is created and deleted during a step.

Relationship to other parameters

Do not code the following parameters with the DISP parameter.

```
*
BURST
CHARS
COPIES
DATA
DDNAME
DYNAM
FLASH
MODIFY
QNAME
SYSOUT
```

Disposition of QSAM data sets

Do not code DISP=MOD if the data control block (DCB) specifies RECFM=FBS and the data set is processed by QSAM. If you do and a block is shorter than the specified block size, QSAM assumes that the short block is the last block and starts end-of-file processing. By this action, QSAM can embed short blocks in your data set and so affect the number of records per track.

Disposition of generation data sets

See Topic "VSAM Data Sets" in *z/OS MVS JCL User's Guide* for additional information about disposition processing for generation data sets.

Disposition of temporary data sets

Specify a normal termination disposition of PASS or DELETE for a temporary data set or for a data set with a system-generated name, that is, when a DSNAME parameter is omitted from the DD statement.

For a temporary data set name, the system ignores any abnormal termination disposition specified in the third subparameter and always PASSes the data set to subsequent steps.

Disposition of partitioned data sets (PDSs and PDSEs)

When you specify DISP=MOD or DISP=NEW for a partitioned data set (PDS) or partitioned data set extended (PDSE), and you also specify a member name in the DSNAME parameter, the member name must not already exist. If the member name already exists, the system terminates the job.

When you specify DISP=OLD for a PDS or a PDSE, and you also specify a member name in the DSNAME parameter, the data set must already exist. If the member name already exists and the data set is opened for output, the system replaces the existing member with the new member. If the member name does not already exist and the data set is opened for output, the system adds the member to the data set.

When you specify DISP=MOD for a PDS or a PDSE, and you do not specify a member name, the system positions the read/write mechanism at the end of the data set. The system does not make an automatic entry into the directory.

When you specify DISP=MOD for a PDS or a PDSE, and you do specify a member name, the system positions the read/write mechanism at the end of the data set. If the member name already exists, the system terminates the job.

When you specify DISP=SHR for a partitioned data set extended (PDSE) and also specify a member name, then:

- If the member name exists, the member can have one writer or be shared by multiple readers, or

- If the member name does not exist, the member can be added to the data set. Thus, multiple jobs can access different members of the data set and add new members to the data set concurrently — but concurrent update access to a specific member (or update and read by other jobs) is not valid.

Adding a volume to a cataloged data set

If you want to add a volume to a cataloged data set and have it properly cataloged after it is kept or passed, code the volume count subparameter of the VOLUME parameter to make the system use the values in the system catalog to process the data set. The following DD statement shows how to keep and extend a cataloged data set using the system catalog. Assume that this data set was created with a volume count of 2.

```
//DDEX2 DD DSNAME=OPER.DATA,DISP=(MOD,KEEP),
//            VOLUME=(,,,3),UNIT=(,P)
```

The VOLUME parameter references the system catalog for volume information about the data set and increases the maximum number of volumes for OPER.DATA. Because the UNIT parameter requests parallel mounting, the system must allocate the same number of units as the number of volumes in the VOLUME parameter; in this case, 3.

The following is an example of the messages in the job log after the job completes.

```
IEF285I    OPER.DATA                                          KEPT
IEF285I    VOL SER NOS= 333001,333002,333003.
IEF285I    OPER.DATA                                          RECATALOGED
IEF285I    VOL SER NOS= 333001,333002,333003.
```

Non-SMS-managed data sets: If you do not reference the catalog when adding a volume to a cataloged data set, the system does not update the catalog with the newly referenced volumes.

DISP=MOD for a multivolume data set

Minimizing tape mounts

When you code DISP=MOD and the volume information is for a multivolume data set, normally the first volume(s) will be mounted on the devices(s) allocated. Then, if the data set is opened for output, OPEN starts with the last volume. If the number of tape volumes is more than the number of allocated devices, the system asks the operator to demount the first volume(s) and mount the last. To have the last tape volume mounted without first mounting and then demounting the earlier volume(s), code VOLUME=REF or DEFER in the UNIT parameter, or a volume sequence number in the VOLUME parameter.

Determining the last volume

If a data set that is not a striped data set resides on multiple volumes, you can code a volume sequence number to specify the volume on which reading or writing is to begin. If you do not code a volume sequence number and the data set is not striped, the system must identify the volume that contains the logical end of the data. Data might not have been written on all the volumes. After the system identifies the last volume, it positions the read/write mechanism on that volume.

In DASD and tape data set labels there is an indicator on the last volume containing user data. When you do not specify a volume sequence number, the system looks in the data set label for the indicator that identifies the last volume, and then selects the volume on which to begin writing as follows:

In the DD: DISP section “How the System Determines the Last Volume”, SMS-managed DASD and Non-SMS-managed DASD processing are described separately. The Non-SMS-managed DASD description can be deleted. The SMS-managed DASD subtitle can be changed to be merely DASD.

DASD: The system tests the data set label on the first volume in the list. If the label indicates it contains the end of the data set, the system selects that volume. Otherwise, it checks each subsequent volume until it finds one that has a last-volume indicator. (To begin writing, the system will not select later volumes that might also have the last-volume indicator by virtue of having previously contained parts of the data set.)

Tape: See the information on Minimizing Tape Mounts.

Extending on a volume other than the last

When you code DISP=MOD for a multivolume data set, use the volume count and volume sequence number subparameters of the VOLUME parameter if you want to keep the system from positioning the read/write mechanism after the last record on the last volume. For example:

```
//DDEX1  DD  DSNAME=OPER.DATA,DISP=(MOD,KEEP),VOLUME=(,,1,2)
```

The volume sequence number of 1 specifies that you want to use the first volume, and the volume count of 2 specifies that the data set requires two volumes.

Effect of DCB=dsname parameter

If the DCB parameter refers to a cataloged data set, the system obtains the volume sequence number from the label of the data set, unless the volume sequence number is coded on the DD statement.

Thus, for the following DD statement, even though DISP=MOD is specified, the system positions the read/write mechanism after the last record on the volume specified in the volume sequence number in the label; this volume may or may not be the last volume.

```
//DD1   DD   DSNAME=MULTI1,DISP=MOD,DCB=CATDD
```

To control which volume is processed, code a volume sequence number.

```
//DD2   DD   DSNAME=MULTI2,DISP=MOD,DCB=CATDD,VOLUME=(,,2)
```

Summary of disposition processing

Table 17. Summary of Disposition Processing

DISP subparameters:			**Disposition (if data set was allocated):**			
Status	**Normal termination disposition**	**Abnormal termination disposition**	**At normal end of step**	**At abnormal end of step**		**At End of Job**
				Step abnormally terminated	**If later allocation failed in step**	
NEW permanent data set or MOD treated as new	none	none	deleted	deleted	deleted	
	KEEP		kept	kept		
	DELETE		deleted	deleted		
	CATLG		cataloged	cataloged		
	PASS		passed	passed	passed	deleted
	PASS	DELETE KEEP CATLG UNCATLG	passed	passed	passed	If all steps terminated normally: deleted If a step terminated abnormally: third subparameter disposition
	DELETE KEEP CATLG UNCATLG	KEEP	second subparameter disposition	kept	deleted	
		DELETE		deleted		
		CATLG		cataloged		
NEW temporary data set	none	DELETE KEEP CATLG UNCATLG	deleted	deleted	deleted	
	DELETE					
	PASS		passed			deleted
NEW data set in step to be automatically restarted	DELETE KEEP PASS CATLG UNCATLG	DELETE KEEP CATLG UNCATLG		deleted		

Table 17. Summary of Disposition Processing (continued)

DISP subparameters:			Disposition (if data set was allocated):			
Status	**Normal termination disposition**	**Abnormal termination disposition**	**At normal end of step**	**At abnormal end of step**		**At End of Job**
				Step abnormally terminated	**If later allocation failed in step**	
NEW data set in step to be restarted at checkpoint	DELETE KEEP PASS CATLG UNCATLG	DELETE KEEP CATLG UNCATLG		kept, if being used when checkpoint was taken		
OLD or SHR or MOD treated as old	none	none	kept	kept	kept	
	KEEP					
	DELETE		deleted	deleted		
	CATLG		cataloged or, if new volumes were added, recataloged	cataloged or, if new volumes were added, recataloged		
	UNCATLG		uncataloged	uncataloged		
	PASS		passed	passed	passed	kept

Table 17. Summary of Disposition Processing (continued)

DISP subparameters:			Disposition (if data set was allocated):			
Status	**Normal termination disposition**	**Abnormal termination disposition**	**At normal end of step**	**At abnormal end of step**		**At End of Job**
				Step abnormally terminated	**If later allocation failed in step**	
OLD or SHR or MOD treated as old (continued)	PASS	DELETE KEEP CATLG UNCATLG	passed	passed	passed	If all steps terminated normally: kept, if originally old; deleted, if originally new If a step terminated abnormally: third subparameter disposition
	DELETE KEEP CATLG UNCATLG	KEEP	second parameter disposition	kept	kept, if step was receiving originally old data set; deleted, if step was receiving originally new data set	
		DELETE		deleted		
		CATLG		cataloged or, if new volumes were added, recataloged		
		UNCATLG		uncataloged		
OLD permanent data set passed to this job step	none	none	deleted, if data set was originally new; kept, if originally old	deleted, if data set was originally new; kept, if originally old		
OLD data set in step to be automatically restarted	DELETE KEEP PASS CATLG UNCATLG	DELETE KEEP CATLG UNCATLG		kept		
OLD data set in step to be restarted at checkpoint	DELETE KEEP PASS CATLG UNCATLG	DELETE KEEP CATLG UNCATLG		kept, if being used when checkpoint was taken		

Examples of the DISP parameter

Example 1

```
//DD2  DD  DSNAME=FIX,UNIT=3420-1,VOLUME=SER=44889,
//         DISP=(OLD,,DELETE)
```

DD statement DD2 defines an existing data set and implies by the omitted second subparameter that the data set is to be kept if the step terminates normally. The statement requests that the system delete the data set if the step terminates abnormally.

Example 2

```
//STEPA  EXEC PGM=FILL
//DD1    DD   DSNAME=SWITCH.LEVEL18.GROUP12,UNIT=3390,
//         VOLUME=SER=LOCAT3,SPACE=(TRK,(80,15)),DISP=(,PASS)
//STEPB  EXEC PGM=CHAR
//DD2    DD   DSNAME=XTRA,DISP=OLD
//DD3    DD   DSNAME=*.STEPA.DD1,DISP=(OLD,PASS,DELETE)
//STEPC  EXEC PGM=TERM
//DD4    DD   DSNAME=*.STEPB.DD3,DISP=(OLD,CATLG,DELETE)
```

DD statement DD1 defines a new data set and requests that the data set be passed. If STEPA abnormally terminates, the data set is deleted because it is a new data set, the second subparameter is PASS, and an abnormal termination disposition is not coded.

DD statement DD3 in STEPB receives this passed data set and requests that the data set be passed. If STEPB abnormally terminates, the data set is deleted because of the third subparameter of DELETE.

DD statement DD4 in STEPC receives the passed data set and requests that the data set be cataloged at the end of the step. If STEPC abnormally terminates, the data set is deleted because of the abnormal termination disposition of DELETE.

DD statement DD2 defines an old data set named XTRA. When STEPB terminates, normally or abnormally, this data set is kept.

Example 3

```
//SMSDD5 DD  DSNAME=MYDS5.PGM,DATACLAS=DCLAS05,STORCLAS=SCLAS05,
//          DISP=(NEW,KEEP)
```

DD statement SMSDD5 defines a new SMS-managed data set and requests that the data set be kept (which implies that it be cataloged).

Example 4

```
//SMSDD7 DD  DSNAME=MYDS7.PGM,DISP=(OLD,UNCATLG)
```

DD statement SMSDD7 defines an existing SMS-managed data set (the data set had been assigned a storage class when it was created) and requests that the data set be uncataloged. However, the data set is kept because UNCATLG is ignored for SMS-managed data sets.

DLM parameter

Parameter Type

Keyword, optional

Purpose

Use the DLM parameter to specify a delimiter to terminate this in-stream data set. When the DLM parameter assigns a different delimiter, the in-stream data records can include standard delimiters, such as /* and //, in the data.

Subparameter definition

keylabel

Specifies the label DSKEYLBL for the encryption key used by the system to encrypt the data set.

The key label, one to 64 characters, should be defined in the CKDS by the ICSF administrator at your installation.

For a description of key label, see ICSF publication *z/OS Cryptographic Services Integrated Cryptographic Service Facility Application Programmer's Guide.*

Note:

Quotes are required. DSKEYLBL is effective only if the new data set is on DASD. It is ignored for device types other than DASD, including DUMMY.

Syntax

```
DSKEYLBL='keylabel'
```

Note:

Quotes are required.

Example of the DSKEYLBL parameter

In this example, DD statement DD1 defines a new data set named DSN1 and requests that the system creates the data set as an SMS-managed encrypted data set using key label LABELFORDSN1 to encrypt and decrypt the data. Key label LABELFORDSN1 should be defined in the CKDS.

```
//DD1 DD DSN=DSN1,DISP=(NEW,CATLG),DATACLAS=DCENCRYP, STORCLAS=SCDASD,
// DSKEYLBL='LABELFORDSN1'
```

DSNAME parameter

Parameter Type

Keyword, optional

Purpose

Use the DSNAME parameter to specify the name of a data set. For a new data set, the specified name is assigned to the data set; for an existing data set, the system uses the name to locate the data set.

References

Data sets are described in *z/OS DFSMS Using Data Sets*.

The names of all data sets that are to be cataloged or SMS-managed must conform to the rules for cataloged data set names. For information about the rules for cataloged data set names, refer to *z/OS DFSMS Using Data Sets*.

Syntax

```
 {DSNAME}  =name
 {DSN   }

name for permanent data set:

     dsname
     dsname(member)
     dsname(generation)

name for temporary data set:

     &&dsname
     &&dsname(member)

name for in-stream or sysout data set:

     &&dsname

name copied from earlier DD statement:

     *.ddname
     *.stepname.ddname
     *.stepname.procstepname.ddname

name for dummy data set:

     NULLFILE
```

- You can abbreviate DSNAME as DSN.
- Avoid starting a data set name with JES or SYS1. The system uses these characters for system data sets.
- If the data set name begins with a blank character, the system assigns the data set with a unique temporary data set name, and ignores the name specified on the DSNAME parameter
- The system ignores blank characters at the end of a data set name.
- Blanks can be included in a data set name if the name is enclosed in apostrophes, such as DSNAME='AB CD'. However, do not code blanks in the name for an in-stream or sysout data set; for example, SYSOUT=P,DSNAME='&&AB CD' is not valid.
- If the data set is to be managed through SMS, you cannot enclose the data set name in apostrophes. However, the following exception applies: You can enclose the data set name on the DSNAME parameter in apostrophes if the data set is to be assigned to, or already resides on, an SMS-managed mountable tape volume.
- Any data set name enclosed in apostrophes on the DSNAME parameter will be treated as an unqualified name. Data sets with an unqualified name cannot be cataloged.
- The system does not check data set names enclosed in apostrophes for valid characters or valid length. When SMS is not installed or active incorrect characters or length result in data set allocation, but the data set is not cataloged. When SMS is active, it will fail the job for incorrect characters or length.

Non-significant special characters: When a data set name contains special characters that are not significant to the system, other than hyphens, enclose it in apostrophes. For example, DSNAME='DS/29'.

Code each apostrophe that is part of the data set name as two consecutive apostrophes. For example, code DAYS'END as DSNAME='DAYS''END'.

The system ignores blank characters at the end of a data set name, even if the data set name is enclosed in apostrophes.

Significant special characters: The following special characters are significant to the system. Do not enclose them in apostrophes.

- Periods to indicate a qualified data set name. However, you must enclose in apostrophes a period immediately before a right parenthesis, immediately after a left parenthesis, or immediately before a comma; for example, DSNAME='(.ABC)' and DSNAME='(ABC.)' and DSNAME='A.B.C.'.
- Double ampersands to identify a temporary data set name. Note that if you use apostrophes, DSNAME='&&AB' and DSNAME='&AB' refer to the same data set.
- Double ampersands to identify an in-stream or sysout data set name.
- Parentheses to enclose the member name of a partitioned data set (PDS) or partitioned data set extended (PDSE), or the generation number of a generation data set.
- Plus (+) or minus (-) sign to identify a generation of a generation data group.
- The asterisk to indicate a backward reference.

On a DD statement in a cataloged or in-stream procedure, if the data set name is a symbolic parameter, do not enclose it in apostrophes. If it is enclosed in apostrophes, the system performs correct substitution only if the symbolic parameter enclosed in apostrophes is preceded by a symbolic parameter not enclosed in apostrophes.

The data set name should not contain the 44 special characters (X'04') created by the 12-4-9 multiple punch or any operation that converts the value of characters to X'04'.

Subparameter definition

The data set names you specify on DSNAME are described in the following topics:

- Data Set Name for Permanent Data Set
- Data Set Name for Temporary Data Set
- Data Set Name for In-Stream or Sysout Data Set
- Data Set Name Copied from Earlier DD Statement
- Data Set Name for Dummy Data Set

Data set name for permanent data set

Assign a permanent data set either an unqualified or a qualified name:

Unqualified name

1 through 8 alphanumeric or national ($, #, @) characters, a hyphen, or a character X'C0'. The first character must be alphabetic or national ($, #, @). For example, DSNAME=ALPHA is an unqualified data set name.

For the characters allowed in ISO/ANSI/FIPS tape data set names, see information about label definition and organization in *z/OS DFSMS Using Magnetic Tapes*.

Qualified name

Multiple unqualified names joined by periods. Each qualifier is coded like an unqualified name; therefore, the name must contain a period after every 8 characters or fewer. For example, DSNAME=ALPHA.PGM is a qualified data set name. The maximum length of a qualified data set name is:

- 44 characters, including periods.
- For a generation data group, 35 characters, including periods.
- For an output tape data set, 17 characters, including periods. If longer than 17 characters, only the rightmost 17 characters, excluding trailing blanks, are written to the tape header label. For more information, see *z/OS DFSMS Using Magnetic Tapes*.

Name for RACF-protected data set

The z/OS Security Server, which includes RACF, expects the data set name to have a high-level qualifier that is defined to RACF. See the *z/OS Security Server RACF Security Administrator's Guide* for details. RACF uses the entire data set name, from 1 through 44 characters, when protecting a tape data set.

Cataloged data set name

For information about the rules for cataloged data set names, see *z/OS DFSMS Access Method Services Commands*.

dsname
Specifies a data set name.

dsname(member)
Specifies the name of the permanent partitioned data set (PDS) or the partitioned data set extended (PDSE), and the name of a member within that data set. If the member does not exist and DISP=OLD or DISP=SHR is specified, the allocation will succeed, but the job will fail when the data set is opened for input. If the member does not exist and the data set is opened for output, the system will add the member to the data set.

member
1 to 8 alphanumeric or national characters, or a character X'C0'. The first character must be alphabetic, national, +, or -. If the first character is + or -, the member is a part of a generation data group.

dsname(generation)
Specifies the name of a generation data group (GDG) and the generation number (zero or a signed integer) of a generation data set within the GDG.

Note: A VSAM data set cannot be a generation data set.

generation

- The first character of a relative generation number is +, -, or 0.
- All characters of a relative generation number that follow the +, -, or 0 must be numeric (0 through 9).
- The numeric portion (not + or -) of a relative generation number must be expressed in 1 to 3 numeric characters. For example, +100, -002, +4, -09, 000.
- A relative generation number cannot exceed 255.

To retrieve all generations of a generation data group, omit the generation number.

Data set name for temporary data set

A temporary data set is a data set that you create and delete within a job. (For information about coding data set names with the DD *, DATA, and SYSOUT parameters, see "Data Set Name for In-Stream or Sysout Data Set.")

Note: SMS manages a temporary data set if you specify a storage class (with the DD STORCLAS parameter) or if an installation-written automatic class selection (ACS) routine selects a storage class for the temporary data set.

When you define a temporary data set, you can code the DSNAME parameter or omit it; in either case, the system generates a qualified name for the temporary data set.

When you use DSNAME for a temporary data set, code the name as two ampersands (&&) followed by a character string 1 to 8 characters in length:

- The first character following the ampersands must be alphabetic or national.
- The remaining characters must be alphanumeric or national.

The format of the qualified name the system generates depends on whether or not you specified a data set name on the DSNAME parameter:

- All temporary data set names begin as follows:

```
SYSyyddd.Thhmmss.RA000.jjobname
```

  where:

  yy
  indicates the year

  ddd
  indicates the Julian day

  hh
  indicates the hour

  mm
  indicates the minute

  ss
  indicates the second

  jjobname
  indicates the name of the job

  Date fields and time fields in the system-generated name reflect:

  – For JCL allocations, when the job was processed by the Interpreter. (For JES2 this is when an initiator selects the job for execution. For JES3 this is at CI time.)
  – For dynamic allocations, when the dynamic allocation request was issued.
- If you do not specify a data set name, or TEMPDSFORMAT(UNIQUE) is in effect, the full format of the temporary data set name is:

```
SYSyyddd.Thhmmss.RA000.jjobname.Rggnnnnn
```

  where:

  gg
  01, or, in a sysplex:

  – for JCL allocations, the system identifier of the system that interpreted the job.
  – for dynamic allocations, the system identifier of the system on which the job executed.

  nnnnn
  a number that is unique within a system
- If you do specify a data set name and TEMPDSFORMAT(INCLUDELABEL) is in effect, the full format of the temporary data set name is:

```
SYSyyddd.Thhmmss.RA000.jjobname.dsetname.Hgg
```

  where:

  dsetname
  the 1 to 8 character DSNAME coded following the two ampersands (&&)

  gg
  01, or, in a sysplex:

  – for JCL allocations, the system identifier of the system which interpreted the job.
  – for dynamic allocations, the system identifier of the system on which the job executed.

If you use DSNAME, note that the system-generated qualified name for the temporary data set will **not** be unique under the following conditions:

- Multiple tasks or APPC transactions having identical jobnames execute at exactly the same time, and
- The tasks or transactions contain DD statements with identical temporary data set names.

To ensure that a temporary data set name is unique, do not code a temporary data set name. Allow the system to assign one.

Only the job that creates a temporary data set has access to it to read and write data and to scratch the data set.

Note:

1. In general, the system treats a single ampersand (&) followed by a character string of 1 to 8 characters as a symbolic parameter. (See "Using system symbols and JCL symbols" on page 37.) However, if you code a data set name as a symbolic parameter (by coding DSNAME=&xxxxxxxx), and do not assign a value to or nullify the symbolic parameter, the system will process it as a temporary data set name.
2. The SYSTEM TEMPDSFORMAT(UNIQUE|INCLUDELABEL) option in the parmlib member ALLOCxx enables allocation to use a more unique format for the data set name when DSN=&&mydsn is specified. The unique data set name allows jobs with the same data set names specified to run at the same time, without requiring the JCL programmer to remove the DSN=&&mydsn or to add data set name referback syntax. The system setting for this option may affect the data set name generated for a temporary data set.

&&dsname
Specifies the name of a temporary data set.

&&dsname(member)
Specifies the name of a temporary partitioned data set (PDS) or partitioned data set extended (PDSE) and a member within that data set.

member
1 - 8 alphanumeric or national characters, or a character X'C0'. The first character must be alphabetic or national.

Data set name for in-stream or sysout data set

Use the DSNAME parameter to assign a data set name to an in-stream data set (defined with the DD * or DD DATA parameter) and to a sysout data set (defined with the DD SYSOUT parameter). When defining an in-stream or sysout data set, you can code the DSNAME parameter or omit it; if omitted, the system generates a name for the data set.

The data set name for in-stream and sysout data sets consists of two ampersands (&&) followed by one through eight 8 alphanumeric or national ($, #, @,) characters, a hyphen, or a character X'C0'. The first character following the ampersands must be alphabetic or national ($, #, @).

The system generates a qualified name for the in-stream or sysout data set. The qualified name contains:

- The userid of the job
- The jobname
- The jobid
- A system-assigned identifier
- The data set name from the DSNAME parameter (if DSNAME is coded), or a question mark (?) if DSNAME is not coded.

The format of the name is:

```
userid.jobname.jobid.Ddsnumber.name
```

where name is the dsname or a question mark (?).

When the system checks a user's authority to access a SYSOUT data set, the check is made against the JESSPOOL class using the fully qualified name, preceded by the node name and a period:

```
nodename.userid.jobname.jobid.Ddsnumber.name
```

Profiles of this format may be defined in your security system to allow other users access to your SYSOUT data sets. A profile is not necessary for you to access your own data sets.

Note: A single ampersand before a data set name in a cataloged or in-stream procedure signifies a symbolic parameter. However, if no value is assigned to the name on either the EXEC statement that calls the procedure, a PROC statement in the procedure, or a previous SET statement, the system treats the name as the last qualifier of the data set name for an in-stream or sysout data set.

&&dsname
Specifies the last qualifier of the system-generated data set name for an in-stream or sysout data set.

Data set name copied from earlier dd statement

A backward reference is a reference to an earlier statement in the job or in a cataloged or in-stream procedure called by this or an earlier job step. A backward reference can be coded in the DSNAME parameter to copy a data set name from an earlier DD statement.

When copying the data set name, the system also copies the following from the DD statement:

- Whether or not the data set is a PDS or a PDSE.
- Whether or not the data set is a temporary data set.

***.ddname**
Asks the system to copy the data set name from earlier DD statement ddname.

***.stepname.ddname**
Asks the system to copy the data set name from DD statement, ddname, in an earlier step, stepname, in the same job.

***.stepname.procstepname.ddname**
Asks the system to copy the data set name from a DD statement in a cataloged or in-stream procedure. Stepname is the name of this job step or an earlier job step that calls the procedure, procstepname is the name of the procedure step that contains the DD statement, and ddname is the name of the DD statement.

Data set name for dummy data set

NULLFILE
Specifies a dummy data set. NULLFILE has the same effect as coding the DD DUMMY parameter. NULLFILE must be coded as a single-word parameter. For instance, IBM does not support the use of NULLFILE to obtain a dummy data set for these (or other) formats:

- When followed by a member name
- As a qualifier in a qualified data set name
- As a temporary data set name.

Relationship to other parameters

Do not code the following parameters with the DSNAME parameter.

```
DDNAME
DYNAM
QNAME
```

Do not code the DCB IPLTXID subparameter with the DSNAME parameter.

Reserved Data Set Names: Do not code the following data set names on the DSNAME parameter with the *, DATA, or SYSOUT parameter (an in-stream or sysout data set); the names are reserved for system use.

```
JESJCL
JESJCLIN
JESMSGLG
JESYSMSG
```

With DD AMP parameter: When you code an AMP parameter for a VSAM data set, do not code a DSNAME:

- That contains parentheses, a minus (hyphen), or a plus (+) sign.
- That is in the form for ISAM.
- That is in the form for PAM (partitioned access method).
- That names a generation data group.

With DD DISP parameter: You can create a permanent data set by specifying a qualified or unqualified data set name, the disposition must be other than DELETE.

The following example illustrates how to create a permanent data set:

```
//REPORT   DD DSN=DEHART.APAR.REPORT,SPACE=(CYL,(5,5)),
//            DISP=(NEW,CATLG),UNIT=SYSALLDA,
//            DCB=(LRECL=121,RECFM=FBA,BLKSIZE=1210)
```

You can create a temporary data set by specifying a:

- &&dsname or by omitting the DSNAME parameter
- Qualified or unqualified data set name and specifying, either explicitly or implicitly, DISP=(NEW,DELETE).

The following two examples illustrate how to create a temporary data set:

```
//MYDD1    DD DSN=TEMP1,UNIT=3480,DISP=(,DELETE),SPACE=(TRK,(1,1))
```

```
//DD2      DD UNIT=SYSALLDA,SPACE=(TRK,1),DISP=(NEW,PASS)
```

Note: When you code a disposition of CATLG for a data set, do not code a DSNAME name in apostrophes.

Examples of the DSNAME parameter

Example 1

```
//DD1   DD  DSNAME=ALPHA,DISP=(,KEEP),
//        UNIT=3390,VOLUME=SER=389984
```

DD statement DD1 defines a new data set and names it ALPHA. DD statements in later job steps or jobs may retrieve this data set by specifying ALPHA in the DSNAME parameter, unit information in the UNIT parameter, and volume information in the VOLUME parameter.

Example 2

```
//DDSMS1   DD  DSNAME=ALPHA.PGM,DISP=(NEW,KEEP),DATACLAS=DCLAS1,
//          MGMTCLAS=MCLAS1,STORCLAS=SCLAS1
```

DD statement DDSMS1 defines a new SMS-managed data set and names it ALPHA.PGM. DD statements in later job steps or jobs may retrieve this data set by specifying ALPHA.PGM in the DSNAME parameter.

Example 3

```
//DD2   DD  DSNAME=LIB1(PROG12),DISP=(OLD,KEEP),UNIT=3390
//       VOLUME=SER=882234
```

DD statement DD2 retrieves member PROG12 from the partitioned data set named LIB1.

- Specify or imply DISP=NEW.
- Specify in the LABEL parameter a label type of:
  - SL or SUL for IBM standard labels.
  - AL or AUL for ISO/ANSI Version 1 or ISO/ANSI/FIPS Version 3 tape labels.
  - NSL for nonstandard labels. In this case, the NSL installation exit routine must issue a RACDEF or RACROUTE TYPE=DEFINE macro instruction.
  - NL for no labels.
  - BLP for bypass label processing.
  - LTM for leading tapemark.

Note that RACF cannot fully protect unlabeled tapes because RACF cannot verify the volume serial number directly; the operator must verify the volume serial number when mounting the tape volume.

Requirements for protecting a direct access data set

A DD statement that contains a PROTECT parameter to establish RACF protection for a direct access data set must:

- Name a permanent data set in the DSNAME parameter.
- Specify a status of DISP=NEW or MOD treated as NEW. RACF can establish protection only when the data set is being created.

Examples of the PROTECT parameter

Example 1

```
//DASD  DD  DSNAME=USER37.MYDATA,DISP=(,CATLG),
//      VOLUME=SER=333000,UNIT=3390,SPACE=(TRK,2),PROTECT=YES
```

This DD statement requests RACF protection for the new direct access data set USER37.MYDATA.

Example 2

```
//TAPEVOL  DD  DSNAME=MHB1.TAPEDS,DISP=(NEW,KEEP),LABEL=(,NL),
//           VOLUME=SER=T49850,UNIT=3390,PROTECT=YES
```

This DD statement requests RACF protection for tape volume T49850. Because a specific tape volume is requested, it automatically has the PRIVATE attribute. The volume has no labels.

Example 3

```
//TAPEDS  DD  DSNAME=INST7.NEWDS,DISP=(NEW,CATLG),LABEL=(2,SUL),
//          VOLUME=SER=223344,UNIT=3390,PROTECT=YES
```

This DD statement requests RACF protection for INST7.NEWDS, which is the second data set on tape volume 223344. Because a specific tape volume is requested, it automatically has the PRIVATE attribute. The volume has IBM standard and user labels; the RACF TAPEDSN option must be active.

RECFM parameter

Parameter type

Keyword, optional

Purpose

Use the RECFM parameter to specify the format and characteristics of the records in a new data set. RECFM overrides the record format that is specified in the data set label, and with SMS, RECFM overrides the record format defined in the DATACLAS parameter for the data set. See “Overrides” on page 123.

RECFM also overrides any contradictory information from the LIKE data set, the REFDD DD statement, and the data class. Examples include RECORG, reuse, and initial load.

Code the RECFM parameter when you want to (1) specify the record format for the data set or (2) with SMS, override the record format defined in the data class of the data set.

The syntax of the RECFM parameter is described in the following topics:

- Coding RECFM for BDAM Access Method
- Coding RECFM for BPAM Access Method
- Coding RECFM for BSAM, EXCP, and QSAM Access Methods

Coding RECFM for BDAM access method

Syntax: BDAM access method:

```
RECFM=     {U  }
           {V  }
           {VS }
           {VBS}
           {F  }
           {FT }
```

U
indicates that the records are undefined length.

V
indicates that the records are variable length.

VS
indicates that the records are variable length and spanned.

VBS
indicates that the records are variable length, blocked, and spanned, and that the problem program must block and segment the records.

F
indicates that the records are fixed length.

T
indicates that the records may be written using the track-overflow feature.

Default: undefined-length, unblocked records.

Coding RECFM for BPAM access method

Syntax: BPAM Access Method

```
RECFM=     {U   } [A]
           {UT  } [M]
           {V   }
           {VB  }
           {VS  }
           {VT  }
           {VBS }
           {VBT }
           {VBST}
           {F   }
           {FB  }
           {FT  }
           {FBT }
```

A or M can be coded with any record format, such as: RECFM=FBA

A
indicates that the records contain ISO/ANSI control characters.

B
indicates that the records are blocked.

F
indicates that the records are fixed length.

M
indicates that the records contain machine code control characters.

T
indicates that the records may be written using the track-overflow feature.

U
indicates that the records are undefined length.

V
indicates that the records are variable length.

Default: U

Coding RECFM for BSAM, EXCP, and QSAM access methods

Syntax: BSAM, EXCP, and QSAM Access Methods

```
RECFM=      {U   } [A]
            {UT  } [M]
            {F   }
            {FB  }
            {FS  }
            {FT  }
            {FBS }
            {FBT }
            {V   }
            {VB  }
            {VS  }
            {VT  }
            {VBS }
            {VBT }
            {VBST}
```

A or M can be coded with any record format, such as: RECFM=FBA

For BSAM, EXCP, and QSAM using ISO/ANSI/FIPS data sets on tape:

```
RECFM=      {D  } [A]
            {DB }
            {DS }
            {DBS}
            {U  }
            {F  }
            {FB }
```

A can be coded with any record format, such as: RECFM=FBA

A or M cannot be specified if the PRTSP subparameter is specified.

A
indicates that the record contains ISO/ANSI device control characters.

B
indicates that the records are blocked.

D
indicates that the records are variable-length ISO/ANSI tape records.

F
indicates that the records are fixed length.

M
indicates that the records contain machine code control characters.

S
(1) For fixed-length records, indicates that the records are written as standard blocks, that is, no truncated blocks or unfilled tracks within the data set, with the exception of the last block or track. (2) For variable-length records, indicates that a record can span more than one block.

T
indicates that the records can be written using the track-overflow feature, if required.

U
indicates that the records are undefined length. U is invalid for an ISO/ANSI/FIPS Version 3 tape data set.

V
indicates that the records are variable length. V cannot be specified for (1) a variable-length ISO/ANSI tape data set (specify D for this data set), (2) a card reader data set, or (3) a 7-track tape unless the data conversion feature (TRTCH=C) is used.

Default: U

Overrides

RECFM overrides the record format that is specified in the data set label, and with SMS, RECFM overrides the record format defined in the DATACLAS parameter for the data set. See "Overrides" on page 123. RECFM also overrides any contradictory information from the LIKE data set, the REFDD DD statement, and the data class. Examples include RECORG, reuse, and initial load.

Relationship to other parameters

Do not code the following DD parameters with the RECFM parameter.

*	DDNAME
AMP	DYNAM
DATA	RECORG
DCB=DSORG	
DCB=RECFM	

Examples of the RECFM parameter

Example 1

```
//DD1B  DD    DSNAME=EVER,DISP=(NEW,KEEP),UNIT=3380,
//            RECFM=FB,LRECL=326,SPACE=(23472,(200,40))
```

In the example, the record format of fixed block (FB) is used for the new data set EVER.

Example 2

```
//SMSDS6  DD  DSNAME=MYDS6.PGM,DATACLAS=DCLAS06,DISP=(NEW,KEEP),
//          RECFM=FB
```

In the example, the record format of fixed block (FB) overrides the record format defined in the data class for the data set.

PATH	PATHOPTS	PATHMODE
PATHDISP	RLS	FILEDATA
LGSTREAM		

Note: The SYMLIST parameter has an effect only on a DD statement for an internal reader.

Example of the SYMLIST parameter

In the following example, FIRSTJOB submits SECJOB. The symbolic parameters DSNAME and VOLSER are passed by the SYSUT2 SYMLIST parameter in FIRSTJOB. These symbols are used in the SYSUT1 DD statement in SECJOB:

```
//FIRSTJOB JOB    MSGLEVEL=(1,1),MSGCLASS=A,NOTIFY=IBMUSER
//MYEXPRT  EXPORT SYMLIST=(DSNAME,VOLSER)
//MYSET1   SET DSNAME=HASP.TEST.MACLIB
//MYSET2   SET VOLSER=J2COM1
//STEP1    EXEC   PGM=IEBGENER
//SYSPRINT DD     SYSOUT=*
//SYSUT2   DD     SYSOUT=(A,INTRDR),SYMLIST=(DSNAME,VOLSER)
//SYSIN    DD     DUMMY
//SYSUT1   DD     DISP=SHR,DSN=TEST.JCL(SECJOB)
```

The previous job assumes that the following job is in the SECJOB member in the TEST.JCL data set:

```
//SECJOB   JOB    MSGLEVEL=(1,1),MSGCLASS=A,NOTIFY=IBMUSER
//STEP1    EXEC   PGM=IEBGENER
//SYSPRINT DD     SYSOUT=*
//SYSUT2   DD     SYSOUT=*
//SYSIN    DD     DUMMY
//*        &DSNAME and &VOLSER from FIRSTJOB are used
//SYSUT1   DD     DISP=SHR,DSN=&DSNAME,VOL=SER=&VOLSER.,
//                UNIT=3390
```

SYSOUT parameter

Parameter type

Keyword, optional

Purpose

Use the SYSOUT parameter to identify this data set as a system output data set, usually called a sysout data set.

Do not use the SYSOUT parameter for an SMS-managed data set (one with an assigned storage class).

The SYSOUT parameter also:

- Assigns this sysout data set to an output class. The attributes of each output class are defined during JES initialization.
- Optionally requests an external writer to process the sysout data set rather than JES. An external writer is an IBM- or installation-written program.
- Optionally identifies the forms on which the data set is to be printed or punched.
- Optionally refers to a JES2 /*OUTPUT statement for processing parameters.

The sysout data set is processed according to the following processing options, in override order:

1. The options specified on this sysout DD statement.
2. The options specified on a referenced OUTPUT JCL statement.
3. The options specified on a referenced JES2 /*OUTPUT statement or on a JES3 //*FORMAT statement.

4. The installation default options for the requested output class.

Note:

1. If a sysout data set has the same class as the JOB statement MSGCLASS parameter, the job log appears on the same output listing as this sysout data set.
2. An installation should maintain a list of available output classes and their attributes. Some classes should be used for most printing and punching, but others should be reserved for special processing. Each class is processed by an output writer. The system operator starts the output writers for the commonly used output classes. If you plan to specify a special output class, ask the operator to start the output writer for that class. If the writer is not started before the job produces the sysout data set, the data set is retained until the writer is started.
3. If the automatic restart manager (ARM) restarts a job, JES discards all non-spin sysout data sets created during the previous execution. (You can avoid losing that output by adding SPIN=UNALLOC to the DD statement for the SYSOUT data set.)

References

For information on output writers and external writers, see *z/OS MVS Using the Subsystem Interface*.

Syntax

```
SYSOUT= { class                                  }
        { *                                      }
        { ([class] [,writer-name] [,form-name]) }
                   [,INTRDR     ] [,code-name]

SYSOUT=(,)
```

- You can omit the parentheses if you code only a class.
- All of the subparameters are positional. Code a comma to indicate an omitted subparameter as follows:
  - If you omit the class, code a comma to indicate the omission. For example, when other subparameters follow, code SYSOUT=(,INTRDR,FM26). When other subparameters do not follow, code a null class as SYSOUT=(,).
  - If you omit a writer-name but code a form-name or code-name, code a comma to indicate the omission. For example, SYSOUT=(A,,FM26).
  - Omission of the third subparameter does not require a comma. For example, SYSOUT=A or SYSOUT=(A,INTRDR).

Subparameter definition

class

Identifies the output class for the data set. The class is one character: A through Z or 0 through 9, which you may optionally include in quotation marks. The attributes of each output class are defined during JES initialization; specify the class with the desired attributes.

The CLASS value specified on the DD statement when used with the INTRDR option is the output class (MSGCLASS) of the job going through the internal reader, assuming that the job (going through the internal reader) does not specify MSGCLASS= on its job statement. If it does, the MSGCLASS from the JOB statement overrides the class on the SYSOUT=(A,INTRDR).

*****

Requests the output class in the MSGCLASS parameter on the JOB statement.

In a JES2 system you can also use the dollar-sign ($) to request the output class in the MSGCLASS parameter on the JOB statement.

(,)
Specifies a null class. A null class must be coded to use the CLASS parameter on a referenced OUTPUT JCL statement.

Specifying SYSOUT=(,) nullifies the SYSOUT class, but designates the data set as a sysout data set that JES will process.

Specifying SYSOUT=, nullifies the entire SYSOUT parameter, and causes the system to process the data set as a normal non-subsystem data set. Because there is no DSNAME parameter, the system treats the data set as a temporary data set. To avoid allocation failures, you might need to supply UNIT or SPACE information.

writer-name
Identifies the member name (1 to 8 alphanumeric characters) of an installation-written program.

An external writer is a started task used to process output. Because the external writer is a started task, it has a userid associated with it. Process output with an external writer by naming the writer on the DD statement that defines the output:

```
//MYOUTPUT  DD SYSOUT=(A,XTWTR)
```

In order for the writer to process that output, the writer's userid must be in a RACF access list. The access list permits the writer's userid to the SYSOUT data set. The writer's userid is the userid specified in the started procedure table for the writer task. If your installation's policy requires security labels, the security label associated with the external writer must be equal to or greater than the security label associated with the SYSOUT. For more information, see your security administrator.

Do not code STDWTR as a writer-name. STDWTR is reserved for JES and used as a parameter in the MVS operator's MODIFY command.

In a JES3 system, do not code NJERDR as a writer-name. NJERDR is reserved for JES3.

INTRDR
Tells JES that this sysout data set is to be sent to the internal reader as an input job stream.

form-name
Identifies the print or punch forms. form-name is 1 through 4 alphanumeric or national ($, #, @) characters.

code-name
Identifies an earlier JES2 /*OUTPUT statement from which JES2 is to obtain processing characteristics. The code-name must be the same as the **code** parameter on the JES2 /*OUTPUT statement.

Note:

- code-name is supported only on JES2 systems.
- Do not specify the code-name subparameter when the job or job step contains a default OUTPUT JCL statement.

Defaults

In a JES2 system, if you do not specify a class on this DD statement or a referenced OUTPUT JCL statement, JES2 assigns the sysout data set to the output class defined by the MSGCLASS value of the JOB statement. See the override order shown under "Purpose" for how this default is established.

If you do not code a writer-name subparameter on this DD statement or a referenced OUTPUT JCL statement, the installation's job entry subsystem processes the sysout data set.

If you do not code a form-name subparameter on this DD statement or a referenced OUTPUT JCL statement, JES uses an installation default specified at initialization.

Overrides

The class subparameter of the DD statement SYSOUT parameter overrides an OUTPUT JCL CLASS parameter. On the DD statement, you must code a null class in order to use the OUTPUT JCL CLASS parameter; for example:

```
//OUTDS DD SYSOUT=(,),OUTPUT=*.OUT1
```

The writer-name subparameter of the DD statement SYSOUT parameter overrides an OUTPUT JCL WRITER parameter.

The form-name subparameter of the DD statement SYSOUT parameter overrides an OUTPUT JCL FORMS parameter. Note that the SYSOUT form-name subparameter can be only four characters maximum while both the OUTPUT JCL FORMS form-name and the JES initialization default form names can be eight characters maximum.

Relationship to other parameters

Do not code the following DD parameters with the SYSOUT parameter.

*	DDNAME	LIKE	ROACCESS
AMP	DISP	PROTECT	
CHKPT	DYNAM	QNAME	
DATA	EXPDT	RETPD	
DATACLAS	LABEL	SUBSYS	
		VOLUME	

Ignored Parameters: Because JES allocates sysout data sets, the UNIT and SPACE parameters are ignored, if coded on a sysout DD statement.

Parameters on Procedure DD Statements that are Overridden: When an overriding DD statement contains a SYSOUT parameter, the system ignores a UNIT parameter on the overridden DD statement in the cataloged or in-stream procedure.

Naming a Sysout Data Set: Code the DSNAME parameter with the SYSOUT parameter if you wish to assign the last qualifier of the system-generated name to a sysout data set.

SYSOUT and DEST Subparameters: Do not code the SYSOUT writer-name subparameter when coding a DEST userid subparameter. These subparameters are mutually exclusive. You can code:

```
//VALID1  DD  SYSOUT=D,DEST=(node,userid)
//VALID2  DD  SYSOUT=(D,writer-name),DEST=(node)
```

With DCB Subparameters: JES2 ignores DCB=PRTSP=2 on a DD statement that also contains a SYSOUT parameter.

For JES, it is not necessary to select a specific BLKSIZE on the DCB parameter for performance reasons because the subsystem selects its own blocking.

INTRDR with OUTPUT Parameter: Do not code an OUTPUT parameter when the writer-name subparameter is INTRDR.

Relationship to other control statements

A sysout DD statement can directly or indirectly reference an OUTPUT JCL statement. The parameters on the referenced OUTPUT JCL statement combine with the parameters on the sysout DD statement to control the processing of the sysout data set. See "OUTPUT parameter" on page 215 and Chapter 24, "OUTPUT JCL statement," on page 443.

SYSOUT cannot specify a code-name subparameter in a job or job step that contains an OUTPUT JCL statement; in this case, JES2 treats the third subparameter as a form-name, instead of a reference to a JES2 /*OUTPUT statement.

Backward references: Do not refer to an earlier DD statement that contains a SYSOUT parameter.

Starting an external writer when requested

When a statement supplying processing options for a sysout data set specifies an external writer, the writer must be started before it can print or punch the data set. The writer is started by a system command from the operator or in the input stream. If the writer is not started before the job produces the sysout data set, the data set is retained until the writer is started.

Held classes in a JES2 system

A sysout data set is held if the sysout DD statement contains HOLD=YES or the OUTPUT JCL statement specifies OUTDISP=HOLD.

Held classes in a JES3 system

If CLASS specifies a class-name that is defined to JES3 as a held class for the output service hold queue (Q=HOLD), all of the new output characteristics might not be included in the data set on the writer queue when (1) the data set is moved from the hold queue to the output service writer queue (Q=WTR), (2) the data set includes an OUTPUT JCL statement, and (3) the NQ= or NCL= keyword is used.

For more information, see *z/OS JES3 Initialization and Tuning Guide*.

Significance of output classes

To print this sysout data set and the messages from your job on the same output listing, code one of the following:

- The same output class in the DD SYSOUT parameter as in the JOB MSGCLASS parameter.
- DD SYSOUT=* to default to the JOB MSGCLASS output class.
- DD SYSOUT=(,) to default to one of the following:
  1. The CLASS parameter in an explicitly or implicitly referenced OUTPUT JCL statement. In this case, the OUTPUT JCL CLASS parameter should specify the same output class as the JOB MSGCLASS parameter.
  2. The JOB MSGCLASS output class, if no OUTPUT JCL statement is referenced or if the referenced OUTPUT JCL statement contains either CLASS= or CLASS=*.

Examples of the SYSOUT parameter

Example 1:

```
//DD1    DD     SYSOUT=P
```

In this example, the DD statement specifies that JES is to write the sysout data set to the device handling class P output.

Example 2:

```
//DD2    DD     DSNAME=&&PAYOUT1,SYSOUT=P
```

7. Font list on the PAGEDEF parameter in the PSF cataloged procedure.

See "PAGEDEF parameter" on page 507 for more information.

Relationship to other parameters

Do not code the following DD parameters with the UCS parameter.

*	DYNAM
AMP	KEYOFF
DATA	PROTECT
DDNAME	QNAME

Do not code the UCS parameter with the DCB subparameters CYLOFL, INTVL, RESERVE, and RKP.

The FOLD and VERIFY subparameters are meaningful only when you specify a printing device directly on a DD statement, for example, UNIT=00E, thus bypassing JES sysout processing.

Using special character sets

To use a special character set, SYS1.IMAGELIB must contain an image of the character set, and the chain or train for the character set must be available. IBM provides standard special character sets, and the installation may provide user-designed special character sets.

Examples of the UCS parameter

Example 1

```
//DD1 DD UNIT=1403,UCS=(YN,,VERIFY)
```

In this example, the DD statement requests a 1403 Printer. The UCS parameter requests the chain or train for special character set code YN. Because VERIFY is coded, the system will display the character set image on the printer before the data set is printed.

Example 2

```
//DD2  DD  SYSOUT=G,UCS=PN
```

In this example, the DD statement requests the device for output class G. If the device is a printer with the UCS feature, the system loads the UCS image for code PN. If the device is an impact printer, the system asks the operator to mount the chain or train for PN, if it is not already mounted. If the device is a 3800, the system uses the UCS subparameter to select the character-arrangement table. Otherwise, the system ignores the UCS parameter.

UNIT parameter

Parameter Type: Keyword, optional

Note: With SMS, you do not need to use the UNIT parameter to specify a device for an SMS-managed data set. Use the STORCLAS parameter (described in "STORCLAS parameter" on page 255) or let an installation-written automatic class selection (ACS) routine select a storage class for the data set.

Also with SMS, for a non-SMS-managed data set, if your storage administrator has set a system default unit under SMS, you do not need to specify UNIT. Check with your storage administrator.

Purpose: Use the UNIT parameter to ask the system to place the data set on:

- A specific device.
- A certain type or group of devices.
- The same device as another data set.

The UNIT parameter can also tell the system how many devices to assign and request that the system defer mounting the volume until the data set is opened.

Syntax

```
{UNIT=([ddd          ] [,unit-count] [,DEFER][,SMSHONOR]) }
       [/ddd         ] [,P          ] [,     ]
       [/dddd        ] [,           ]
       [device-type
       [group-name

{UNIT=AFF=ddname                                 }
```

- You can omit the parentheses if you code only the first subparameter.
- All of the subparameters are positional. If you omit unit-count or P but code DEFER, code a comma to indicate the omission; one device is assigned to the data set. For example, UNIT=(3490,,DEFER,).
- Device-type is mutually exclusive with SMSHONOR.

Subparameter definition

device-number

Identifies a specific device by a 3-digit or 4-digit hexadecimal number. Precede a 4-digit number with a slash (/). A 3-digit number can be specified with or without a slash.

Attention: Specify a device number only when necessary. When you specify a device number, the system can assign only that specific device. If the device is already being used, the job must be delayed or canceled. If the device number is changed or logically moved, the allocation request will fail.

However, for a permanently mounted direct access device, such as a 3390 Direct Access Storage, specifying a device type (UNIT=3390) and a volume serial number in the VOLUME=SER parameter has the same result as specifying a device number in the UNIT parameter.

In a JES3 system, if any DD UNIT parameter in a job specifies a device-number for a device that is JES3-managed or jointly JES3/MVS managed, either the JOB statement must contain a SYSTEM or SYSAFF parameter, or the JES3 //*MAIN statement must contain a SYSTEM parameter.

For an SMS-managed tape library request where SMSHONOR is not specified, or an SMS-managed DASD request, the device number is ignored. For an SMS-managed tape library request where SMSHONOR is specified, the system attempts to allocate the specified device, provided it is in the storage group(s) assigned to the request by SMS.

device-type

Requests a device by its generic name, which is an IBM-supplied name that identifies a device by its machine type and model. For example, UNIT=3390.

When a device-type name contains a hyphen, do not enclose it in apostrophes, for example, UNIT=3400-5.

Obtain the list of device types you can specify from your installation.

If you specify the device-type subparameter, SMS ignores it.

For a 3480 Magnetic Tape Subsystem in compatibility mode, code UNIT=3400-9 or a group-name.

group-name
Requests a group of devices by a symbolic name. The installation must have assigned the name to the device(s) during system initialization or IBM must have assigned the name. The group-name is 1 through 8 alphanumeric characters.

For an SMS-managed tape library request where SMSHONOR is not specified, or an SMS-managed DASD request, the group-name is ignored. For an SMS-managed tape library request, if SMSHONOR is specified along with the group-name, the system attempts to allocate to the subset of devices in the group-name, if they are also selected by SMS.

Group names: A group-name can identify a single device or a group of devices. A group can consist of devices of the same or different types. For example, a group can contain two or more different types of direct access storage devices (DASD) or two or more different types of tape devices, or even a mixture of both direct access and tape devices. Note, however, that IBM does not recommend that a group contain both direct access storage devices (DASD) and tape devices.

Note: A group name is called an esoteric name in Hardware Configuration Definition (HCD) terminology.

Allocation from groups: The system assigns a device from the group. If a group consists of only one device, the system assigns that device. If the group consists of more than one device type, the units requested are allocated from the same device type. For example, if GPDA contains 3380 Disk Storage and 3390 Direct Access Storage devices, a request for two units would be allocated to two 3380s or to two 3390s.

Extending data set: If a data set that was created using the group-name subparameter is to be extended, the system allocates additional devices of the same type as the original devices. However, the additional devices might not necessarily be from the same group.

SYSALLDA: IBM assigned group-names include SYSALLDA, which contains all direct access devices defined to the system.

SYS3480R and SYS348XR: SYS3480R and SYS348XR are IBM-assigned group names. SYS3480R contains 3480, 3480X, and 3490 Magnetic Tape Subsystems. SYS348XR contains 3480X and 3490 Magnetic Tape Subsystems.

Use these group names to override the device type eligibility retrieved by the system when referencing existing 3480- or 3480 XF-formatted data sets. Specifically, use SYS3480R when you want to read 3480-formatted data sets and use SYS348XR when you want to read 3480 XF-formatted data sets.

Note: LABEL=(n,,,IN) is the system-managed tape library equivalent of either UNIT=SYS3480R or UNIT=SYS348XR.

unit-count
Specifies the number of devices for the data set. "Unit-count" is a decimal number from 1 through 59.

Number of devices allocated: The system uses the unit-count to determine how many devices to allocate. For tapes, the system uses the unit-count subparameter to allocate the specified number of system-managed or non-system-managed units. If you also specify P (for parallel mount) in the UNIT parameter, and for SMS-managed DASD, the system uses the **highest** of the following numbers to determine how many devices and volumes to allocate:

- Unit-count specified in the UNIT parameter
- Volume-count specified in the VOLUME parameter
- Number of volume serial numbers implicitly or explicitly specified

You may receive more devices than the unit-count requests if you specify VOLUME=REF or a permanently resident or reserved volume. And, if two DD statements in a step request the same volume and either DD statement requests any other volume(s), the system assigns an additional device.

Unit count for received or VOLUME=REF data sets: The system assigns one device when the DD statement receives a passed data set or refers in a VOLUME=REF subparameter to a cataloged data

set or earlier DD statement for volume and unit information. Code a unit-count subparameter if the data set needs more than one device.

Unit count when device number specified: When the first subparameter requests a specific device, the unit count must be 1 or omitted. Only when the device is a communication device can the unit count be higher than 1.

Unit count when SMSHONOR specified: When SMSHONOR is specified, only the subset of devices that are selected by SMS and are within the specified group-name are eligible. When a unit count is specified, the requested number of devices are selected from the subset of devices. If the subset does not contain enough devices to satisfy the requested unit count, the request fails.

P

Asks the system to allocate the same number of devices as requested in the VOLUME volume-count or SER subparameter, whichever is higher. Thus, all volumes for the data set are mounted in parallel.

If you specify the P subparameter for system-managed DASD, the system ignores it. If you specify the P subparameter for system-managed tape libraries, the system honors it.

DEFER

Asks the system to assign the data set to device(s) but requests that the volume(s) not be mounted until the data set is opened. To defer mounting, DEFER must be specified or implied for all DD statements that reference the volume.

If you specify the DEFER subparameter for system-managed DASD, the system ignores it. If you specify the DEFER subparameter for system-managed tape libraries, the system honors it.

DEFER when data set is never opened: If you request deferred mounting of a volume and the data set on that volume is never opened by the processing program, the volume is never mounted during the job step.

Restrictions on DEFER: Do not code DEFER:

- For a new data set on direct access. The system ignores DEFER.
- On a SYSCKEOV DD statement.

SMSHONOR

Asks the system to honor the device number or group-name and allocate to the device number or group-name specified in the case of an SMS tape library request.

Use only device number or group-name (user-defined esoteric) when you use SMSHONOR. The following subset of devices must be consistent with libraries assigned to the storage class specified on the request or selected by ACS routines of the installation:

1. Subset of devices that are in the specified group name.
2. Subset of devices that are with the specified device number, which are selected by the storage class assigned to the request.

Note: If the device number or group-name does not intersect with the storage class, the request fails.

If you specify the SMSHONOR subparameter on a non-SMS system, the system ignores the keyword.

If you specify a group name in a JES3 environment, all devices in the group must be of the same device type and from the same tape library. There are no such restrictions in a non-JES3 environment.

AFF=ddname

Requests that the system allocate different data sets residing on different, removable volumes to the same device during execution of the step. This request is called **unit affinity**, where "ddname" is the ddname of an earlier DD statement in the same step. Use unit affinity to reduce the number of devices that are used in a job step; request that an existing data set be assigned to the same device(s) as another existing data set.

If you specify the UNIT=AFF subparameter for system-managed DASD, the system ignores it. If you specify the UNIT=AFF subparameter for system-managed tape libraries, the system attempts to honor it. For system-managed tape libraries with OA44357 installed, SMS construct and job-related

information is associated with the DD rather than a device. This allows for multiple DDs with different SMS construct information to be allocated to the same device using AFF=ddname.

Under certain conditions the system ignores unit affinity. See *z/OS MVS JCL User's Guide* for more information.

In a JES3 environment, UNIT=AFF=ddname may not be honored. See *z/OS MVS JCL User's Guide* and *z/OS HCD Planning* for information about device eligibility and unit affinity.

Restrictions on UNIT=AFF: Do not code UNIT=AFF=ddname:

- With DISP=NEW if the data set referenced in the AFF subparameter resides on a direct access device. This restriction applies only to non-SMS-managed DASD. If coded, the system terminates the job. If the referenced data set can be allocated to either tape or DASD, the system allocates both requests to tape devices.
- On a DD * or DD DATA statement or on a DD statement containing a SUBSYS parameter. The system ignores the UNIT=AFF and defaults the device to SYSALLDA.
- When the DD statement referenced in the AFF subparameter contains FREE=CLOSE.
- With the STORCLAS parameter.
- With an affinity specification to an earlier DD statement that requests SYS3480R or SYS348XR on the group-name subparameter, unless volume affinity also exists. Volume affinity exists when two DD statements both reference a data set on the same volume. Do not also specify DISP=OLD or DISP=MOD; attempting to write 3480 data to a 3490 drive, or 3490 data to a 3480 drive, fails during OPEN processing with ABEND 813-04 accompanied by message IEC149I.

Overrides

If you code SYSOUT and UNIT on the same statement, the SYSOUT parameter overrides the UNIT parameter.

The system also obtains device information when the system obtains volume serial information from:

- A VOLUME=REF=dsname reference to an earlier data set.
- A VOLUME=REF=ddname reference to an earlier DD statement.
- The volume(s) for a passed data set.
- The catalog for a cataloged data set.

However, you can override the retrieved device information if the device you specify is a subset of the retrieved device information; otherwise the system ignores the overriding device information. For example, if the retrieved unit grouping is 3390, and the specified unit subparameter is 3390A (a subset of 3390), then the system allocates from the devices contained in 3390A.

If you have 3490 Magnetic Tape Subsystem models A10 and A20 defined to your system and you use one of the IBM-generated group names SYS3480R or SYS348XR, the system overrides the device type retrieved from the catalog with a device from the esoteric device group.

For more about how the system uses device information it retrieves from the catalog, see the text about the relationship of the UNIT and VOLUME parameters for non-SMS-managed data sets in *z/OS MVS JCL User's Guide*.

Note: LABEL=(n,,,IN) is the system-managed tape library equivalent of either UNIT=SYS3480R or UNIT=SYS348XR.

You can mount 3480-formatted or 3480X-formatted (18-track formatted) tape volumes, that are not extended, on a 3490 tape device (36-track write, 18-track or 36-track read).

Relationship to other parameters

Do not code the following DD parameters with the UNIT parameter.

*	DYNAM
DATA	QNAME
DDNAME	

Do not code the UNIT DEFER subparameter on a SYSCKEOV DD statement.

To allocate a device, such as a printer or telecommunications device, that does not involve a data set, do not code the DISP parameter.

See also "Restrictions on UNIT=AFF."

Relationship to other control statements

When SMSHONOR is specified, a device number or a group name must be specified.

Location in the JCL

When a DD statement contains a UNIT=AFF=ddname parameter, the DD statement referenced in the AFF subparameter must be defined earlier in the job step; otherwise, the system treats the DD statement containing UNIT=AFF as a DD DUMMY statement.

The following example illustrates a case where the system treats the DD statement containing the UNIT=AFF as a DD DUMMY statement:

```
   //STEP  EXEC  PGM=TKM
   //DD1   DD    DDNAME=DD5
   //DD2   DD    DSNAME=A,DISP=OLD
   //DD3   DD    DSNAME=C,DISP=SHR,UNIT=AFF=DD1
   //DD5   DD    DSNAME=B,DISP=SHR
```

DD3 requests unit affinity to DD1. Although DD1 occurs earlier in the job step than DD3, it refers to DD5 that is located after DD3. Because DD1 is not completely defined, the system treats DD3 as a dummy statement.

Examples of the UNIT parameter

Example 1:

```
//STEP2  EXEC PGM=POINT
//DDX  DD  DSNAME=EST,DISP=MOD,VOLUME=SER=(42569,42570),
//     UNIT=(3480,2)
//DDY  DD  DSNAME=ERAS,DISP=OLD,UNIT=3480
//DDZ  DD  DSNAME=RECK,DISP=OLD,
//     VOLUME=SER=(40653,13262),UNIT=AFF=DDX
```

DD statement DDX requests two 3480 tape devices, DD statement DDZ requests the same two devices as DDX. Note that the operator will have to change volumes on the two 3480 devices during execution of the job step.

DD statement DDY requests one 3480 tape device.

Example 2:

```
//DD1  DD  DSNAME=AAG3,DISP=(,KEEP),
//       VOLUME=SER=13230,UNIT=3400-5
```

This DD statement defines a new data set and requests that the system assign any 3420 Magnetic Tape Unit that can operate in 6250 BPI NRZI nine-track format.

Example 3:

```
//DD2  DD  DSNAME=X.Y.Z,DISP=OLD,UNIT=(,2)
```

Chapter 13. Special DD statements

Use special DD statements to specify private catalogs, private libraries, and data sets for storage dumps and checkpoints. This topic provides descriptions of these special statements.

Description

Syntax

```
//ddname  DD keyword-parameter[,keyword-parameter]...  [comments]
```

Special ddnames

The special data sets are identified by the following ddnames:

JOBLIB
STEPLIB
SYSABEND
SYSCHK
SYSCKEOV
SYSIN
SYSMDUMP
SYSUDUMP

Except for SYSIN, code these ddnames only when you want the special data sets.

JOBLIB DD statement

Purpose: Use the JOBLIB DD statement to:

- Create a private library.
- Identify a private library that the system is to search for the program named in each EXEC statement PGM parameter in the job. Only if the system does not find the program in the private library, does it search the system libraries.

A private library is a partitioned data set or partitioned data set extended on a direct access device. Each member is an executable, user-written program.

Syntax

```
//JOBLIB DD parameter[,parameter]...  [comments]
```

Parameters on JOBLIB DD statements

When retrieving a cataloged library:

- Code the DSNAME parameter.
- Code the DISP parameter. The status subparameter must be OLD or SHR. The disposition subparameters should indicate what you want done with the private library after its use in the job.

© Copyright IBM Corp. 1988, 2017

- Do not code VOLUME or UNIT.

When retrieving a library that is not cataloged:

- Code the DSNAME parameter.
- Code the DISP parameter. The DISP parameter must be DISP=(OLD,PASS) or DISP=(SHR,PASS). SHR indicates that the data set is old, but allows other jobs to use the library.
- Code the UNIT parameter.
- Code the VOLUME parameter.

When creating a library:

- Code the DSNAME parameter to assign the library a name.
- Code the UNIT parameter. The library must be allocated to a direct access device.
- Code a VOLUME parameter, unless a nonspecific request is to be made for any volume.
- Code the SPACE parameter, allowing enough space for the entire library on one direct access volume. Specify space for the PDS directory.
- Code a DISP parameter. The status is NEW. Code CATLG as the disposition, if you intend to keep the library you are creating. Code PASS as the disposition, if you wish the library to be available throughout the job, but deleted at job termination. Note that you must code a disposition; otherwise, the system assumes DELETE and deletes the library at the end of the first step.

Note: Do not use VSAM for a JOBLIB library.

When adding members to the library:

- In the DSNAME parameter, follow the library name with the name of the program being added to the library. For example, DSNAME=LIBRARY(PROGRAM).
- Code the status in the DISP parameter as MOD. If you cataloged the library when you created it, do not code a disposition. Otherwise, code PASS or CATLG.
- If the JOBLIB library is being created in the job, the JOBLIB DD DISP specified CATLG, and the first step adds a member to it, supply unit and volume information in the first step by coding: VOLUME=REF=*.JOBLIB. This parameter is needed because the library is not actually cataloged until the first step completes execution. Otherwise, unit and volume information should not be supplied for a cataloged library.
- Do not code a SPACE parameter. The JOBLIB DD statement requests space for the entire library.

Other parameters: Code the DCB parameter if complete data control block information is not contained in the data set label. Do not specify FREE=CLOSE; CLOSE is ignored.

Do not code a UNIT=AFF parameter on a JOBLIB statement where the object of the affinity is the same JOBLIB statement. In other words, a JOBLIB statement should not have an affinity back to itself.

Relationship to other control statements

Concatenating job libraries: To specify more than one private library for a job:

- Code a JOBLIB DD statement.
- Immediately follow this statement with DD statements that define other private libraries. Omit a ddname from these subsequent DD statements.

The system searches the libraries for the program in the same order as the DD statements.

Overriding a JOBLIB: If you want the system to ignore the JOBLIB for a particular job step and the step does not require another private library, define the system library on a STEPLIB DD statement. For example, specify:

```
//STEPLIB DD  DSNAME=SYS1.LINKLIB,DISP=SHR
```

For this particular job step, the system will search SYS1.LINKLIB, as specified on the STEPLIB DD statement, for the program requested in the EXEC statement. The system will not search the JOBLIB.

EXEC statement COND parameter: If COND=ONLY is specified on the EXEC statement of a job step and a JOBLIB DD statement is being used, the system does not pass the unit and volume information to any succeeding steps, and the system must search the catalog for the JOBLIB data set's unit and volume information.

Location in the JCL

- The JOBLIB DD statement must immediately follow the JOB statement and any JES statements. There must be no intervening EXEC or other DD statements between the JOBLIB DD statement and the JOB statement.
- If libraries are concatenated to the JOBLIB library, the concatenated DD statements must immediately follow the JOBLIB DD statement.
- Do not include a JOBLIB DD statement in an in-stream or cataloged procedure.

Relationship of a JOBLIB to a STEPLIB

Use a STEPLIB DD statement to define a private library for one job step in a job. If you include a STEPLIB DD statement for a job step and a JOBLIB DD statement for the entire job, the system first searches the step library and then the system library for the requested program. The system ignores the job library for a step that has a STEPLIB DD statement.

Examples of the JOBLIB DD statement

Example 1:

```
//PAYROLL JOB  JONES,CLASS=C
//JOBLIB  DD   DSNAME=PRIVATE.LIB4,DISP=(OLD,PASS)
//STEP1   EXEC PGM=SCAN
//STEP2   EXEC PGM=UPDATE
//DD1     DD   DSNAME=*.JOBLIB,DISP=(OLD,PASS)
```

The private library requested on the JOBLIB DD statement is cataloged. The system passes catalog information to subsequent job steps. The system searches for the programs SCAN and UPDATE first in PRIVATE.LIB4, then in SYS1.LINKLIB. DD statement DD1 refers to the private library requested in the JOBLIB DD statement.

Example 2:

```
//PAYROLL JOB  FOWLER,CLASS=L
//JOBLIB  DD   DSNAME=PRIV.DEPT58,DISP=(OLD,PASS),
//             UNIT=3390,VOLUME=SER=D58PVL
//STEP    EXEC PGM=DAY
//STEP2   EXEC PGM=BENEFITS
//DD1     DD   DSNAME=*.JOBLIB,VOLUME=REF=*.JOBLIB,DISP=(OLD,PASS)
```

The private library requested on the JOBLIB DD statement is not cataloged; therefore, unit and volume information is specified. The system searches for the programs DAY and BENEFITS first in PRIV.DEPT58, then in SYS1.LINKLIB. DD statement DD1 refers to the private library requested in the JOBLIB DD statement.

Example 3:

```
//TYPE   JOB  MSGLEVEL=(1,1)
//JOBLIB DD   DSNAME=GROUP8.LEVEL5,DISP=(NEW,CATLG),
//            UNIT=3390,VOLUME=SER=148562,SPACE=(CYL,(50,3,4))
//STEP1  EXEC PGM=DISC
//DDA    DD   DSNAME=GROUP8.LEVEL5(RATE),DISP=MOD,
//            VOLUME=REF=*.JOBLIB
//STEP2  EXEC PGM=RATE
```

The private library requested on the JOBLIB DD statement does not exist yet; therefore, the JOBLIB DD statement contains all the parameters required to define the library. The library is created in STEP1, when DD statement DDA defines the new member RATE for the library. Therefore, the system searches

SYS1.LINKLIB for the program named DISC. In STEP2, the system searches for the program RATE first in GROUP8.LEVEL5.

Example 4:

```
//PAYROLL JOB  BIRDSALL,TIME=1440
//JOBLIB  DD   DSNAME=KRG.LIB12,DISP=(OLD,PASS)
//        DD   DSNAME=GROUP31.TEST,DISP=(OLD,PASS)
//        DD   DSNAME=PGMSLIB,UNIT=3390,
//           DISP=(OLD,PASS),VOLUME=SER=34568
```

The three DD statements concatenate the three private libraries. The system searches the libraries for each program in this order:

- KRG.LIB12
- GROUP31.TEST
- PGMSLIB
- SYS1.LINKLIB

STEPLIB DD statement

Purpose: Use the STEPLIB DD statement to:

- Create a private library.
- Identify a private library that the system is to search for the program named in the EXEC statement PGM parameter. If the system does not find the program in the private library, only then does the system search the system libraries.

The private library is a partitioned data set (PDS) or partitioned data set extended (PDSE) on a direct access device. Each member is an executable, user-written program.

Subsequent job steps in the same job may refer to or receive a private library defined on a STEPLIB DD statement. Also, you can place a STEPLIB DD statement in an in-stream or cataloged procedure.

Syntax

```
//STEPLIB DD parameter[,parameter]...  [comments]
```

Parameters on STEPLIB DD statements

When retrieving a cataloged library:

- Code the DSNAME parameter.
- Code the DISP parameter. The status subparameter must be OLD or SHR. The disposition subparameters should indicate what you want done with the private library after its use in the job step.
- Do not code VOLUME or UNIT.

When retrieving a library passed from a previous step: In the passing job step, code a DISP disposition subparameter of PASS when a step library is to be used by subsequent steps in the job.

In a receiving step:

- Code in the DSNAME parameter either the name of the step library or a backward reference of the form *.stepname.STEPLIB. If the step library is defined in a procedure, the backward reference must include the procedure step name: *.stepname.procstepname.STEPLIB.
- Code the DISP parameter. The status subparameter must be OLD. The disposition subparameters should indicate what you want done with the private library after its use in the receiving step.

When retrieving a library that is neither cataloged nor passed:

- Code the DSNAME parameter.
- Code the DISP parameter. The status subparameter must be OLD or SHR. The disposition subparameters should indicate what you want done with the private library after its use in the job step.
- Code the UNIT parameter.
- Code the VOLUME parameter.

When creating a library:

- Code the DSNAME parameter to assign the library a name.
- Code the UNIT parameter. The library must be allocated to a direct access device.
- Code a VOLUME parameter, unless a nonspecific request is to be made for any volume.
- Code the SPACE parameter, allowing enough space for the entire library on one direct access volume. Specify space for the PDS directory.
- Code a DISP parameter. The status is NEW. Code CATLG as the disposition, if you intend to keep the library you are creating. Code PASS as the disposition, if you wish the library to be available to a following step. Note that you must code a disposition; otherwise, the system assumes DELETE and deletes the library at the end of the step.

Note: Do not use VSAM for a STEPLIB library.

When adding members to the library:

- In the DSNAME parameter, follow the library name with the name of the program being added to the library. For example, DSNAME=LIBRARY(PROGRAM).
- Code the status in the DISP parameter as MOD. If the library is cataloged, do not code a disposition. Otherwise, code PASS or CATLG.
- If the library is cataloged, do not code unit and volume information. Otherwise, code UNIT and VOLUME.
- Do not code a SPACE parameter. The STEPLIB DD statement requests space for the entire library.

Other parameters: Code the DCB parameter if complete data control block information is not contained in the data set label. Do not specify FREE=CLOSE; CLOSE is ignored.

Relationship to other control statements

Concatenating step libraries: To specify more than one private library for a step:

- Code a STEPLIB DD statement.
- Immediately follow this statement with DD statements that define other private libraries. Omit a ddname from these subsequent DD statements.

The system searches the libraries for the program in the same order as the DD statements.

Overriding a JOBLIB: If you want the system to ignore the JOBLIB for a particular job step and the step does not require another private library, define the system library on a STEPLIB DD statement. For example, specify:

```
//STEPLIB DD  DSNAME=SYS1.LINKLIB,DISP=SHR
```

For this particular job step, the system will first search SYS1.LINKLIB, as specified on the STEPLIB DD statement, for the program requested in the EXEC statement. The system will not search the JOBLIB.

Location in the JCL

Place a STEPLIB DD statement in any position among the DD statements for a step.

If libraries are concatenated to the STEPLIB library, the concatenated DD statements must immediately follow the STEPLIB DD statement.

Relationship of a STEPLIB to a JOBLIB

Use a JOBLIB DD statement to define a private library that the system is to use for an entire job. If you include a JOBLIB DD statement for the job and a STEPLIB DD statement for an individual job step, the system first searches the step library and then the system library for the program requested in the EXEC statement. The system ignores the JOBLIB library for that step.

Examples of the STEPLIB DD statement

Example 1

```
//PAYROLL JOB  BROWN,MSGLEVEL=1
//STEP1   EXEC PROC=LAB14
//STEP2   EXEC PGM=SPKCH
//STEPLIB DD   DSNAME=PRIV.LIB5,DISP=(OLD,KEEP)
//STEP3   EXEC PGM=TIL80
//STEPLIB DD   DSNAME=PRIV.LIB12,DISP=(OLD,KEEP)
```

The system searches PRIV.LIB5 for the program SPKCH and PRIV.LIB12 for TIL80. The system catalogs both private libraries.

Example 2

```
//PAYROLL JOB  BAKER,MSGLEVEL=1
//JOBLIB  DD   DSNAME=LIB5.GROUP4,DISP=(OLD,PASS)
//STEP1   EXEC PGM=SNZ12
//STEP2   EXEC PGM=SNAP10
//STEPLIB DD   DSNAME=LIBRARYP,DISP=(OLD,PASS),
//           UNIT=3390,VOLUME=SER=55566
//STEP3   EXEC PGM=A1530
//STEP4   EXEC PGM=SNAP11
//STEPLIB DD   DSNAME=*.STEP2.STEPLIB,
//           DISP=(OLD,KEEP)
```

The system searches LIBRARYP for program SNAP10; LIBRARYP is passed to subsequent steps of this job. The STEPLIB DD statement in STEP4 refers to the LIBRARYP library defined in STEP2; the system searches LIBRARYP for SNAP11. Since a JOBLIB DD statement is included, the system searches for programs SNZ12 and A1530 first in LIB5.GROUP4, then in SYS1.LINKLIB.

Example 3

```
//PAYROLL JOB  THORNTON,MSGLEVEL=1
//JOBLIB  DD   DSNAME=LIB5.GROUP4,DISP=(OLD,PASS)
//STEP1   EXEC PGM=SUM
//STEPLIB DD   DSNAME=SYS1.LINKLIB,DISP=OLD
//STEP2   EXEC PGM=VARY
//STEP3   EXEC PGM=CALC
//STEPLIB DD   DSNAME=PRIV.WORK,DISP=(OLD,PASS)
//        DD   DSNAME=LIBRARYA,DISP=(OLD,KEEP),
//           UNIT=3390,VOLUME=SER=44455
//        DD   DSNAME=LIB.DEPT88,DISP=(OLD,KEEP)
//STEP4   EXEC PGM=SHORE
```

For STEP2 and STEP4, the system searches the private library named LIB5.GROUP4 defined in the JOBLIB DD statement first for programs VARY and SHORE. For STEP1, the system searches SYS1.LINKLIB first for program SUM, because the STEPLIB DD statement names the system library.

A concatenation of private libraries is defined in STEP3. The system searches for the program named CALC in this order: PRIV.WORK, LIBRARYA, LIB.DEPT88, SYS1.LINKLIB. If a later job step refers to the STEPLIB DD statement in STEP3, the system will search for the program in the private library named PRIV.WORK and, if it is not found there, in SYS1.LINKLIB; the concatenated libraries are not searched.

SYSABEND, SYSMDUMP, and SYSUDUMP DD statements

Purpose: Use a SYSABEND, SYSMDUMP, or SYSUDUMP DD statement in a job step to direct the system to produce a dump. The system produces the requested dump:

- If the step terminates abnormally.
- If the step starts to terminate abnormally, but system recovery procedures enable the step to terminate normally.

The dump DD statements for requesting dumps are:

SYSABEND DD statement
Produces a dump of user and system areas; this dump contains all the areas dumped in a SYSUDUMP, plus:

- The local system queue area (LSQA), including subpools 229, 230, and 249
- The input/output system (IOS) control blocks for the failing task.

The dump is formatted, so that it can be printed directly.

SYSMDUMP DD statement
Produces a dump of the system areas and the program's address space. The dump is unformatted and machine-readable. It must be processed by the interactive problem control system (IPCS) and therefore should not be directed to SYSOUT. z/OS R6 introduced system-determined BLKSIZE support for SYSMDUMPs. If you wish to control the BLKSIZE for compatibility with tools developed for earlier releases, add the following DCB attributes to your SYSMDUMP DD statement.

```
DCB=RECFM=FB,LRECL=4160,BLKSIZE=4160
```

SYSUDUMP DD statement
Produces a dump of user areas. The dump is formatted, so that it can be printed directly.

The dump contents are as described only when the installation uses the IBM-supplied defaults for the dumps. The contents of these dumps can be set during system initialization and/or can be changed for an individual dump in the ABEND macro instruction, in a CHNGDUMP command, and by a SLIP command. For details, see *z/OS MVS Initialization and Tuning Guide*.

Dumps are optional; use a dump DD statement only when you want to produce a dump.

References: For information on how to interpret dumps, see *z/OS MVS Diagnosis: Tools and Service Aids*.

Syntax

```
//SYSABEND  DD parameter[,parameter]...  [comments]
//SYSMDUMP  DD parameter[,parameter]...  [comments]
//SYSUDUMP  DD parameter[,parameter]...  [comments]
```

Location in the JCL

Do not place in the same job step two DD statements with the same dump ddname.

Storing a dump

If you wish to store a dump instead of having it printed, code the following parameters on the dump DD statement:

- The DSNAME parameter.
- The UNIT parameter.
- The VOLUME parameter. This parameter is optional and not recommended. The system will select a volume.
- The DISP parameter. The data set's status is NEW. Because you want to store the data set, make the data set's abnormal termination disposition KEEP or CATLG.
- The SPACE parameter, if the dump is written on direct access.

Tip: SYSABEND, SYSUDUMP, and SYSMDUMP can use extended format sequential data sets to exploit striping, compression, or both. Striping speeds the writing process and compression reduces the space

consumed and speeds the I/O bound dump process. Extended format sequential data set hold more than 64K tracks per volume making it an attractive destination for dumps. DSNAME=LARGE also allows data sets to use more than 64K tracks per volume.

Note: Do not use VSAM for dump data sets.

SYSMDUMP Requirements: The SYSMDUMP DD statement must specify a magnetic tape unit or a direct access device. Do not direct SYSMDUMP to SYSOUT.

With the exception of the following facility, the system processes dump data sets according to the disposition to which they are allocated. To keep only the first SYSMDUMP dump written to a dump data set, specify the following on the SYSMDUMP DD statement:

- DSNAME=SYS1.SYSMDPxx, where xx is 00 through FF and indicates the specific dump data set to be used. SYSMDPxx is a preallocated data set that must have end-of-file (EOF) mark as its first record.
- DISP=SHR
- FREE=CLOSE for multiple job steps

Note: This restriction is not enforced. If SYSOUT is used, the resulting dump will be unusable for diagnosis.

See *z/OS MVS Diagnosis: Tools and Service Aids* for a description of the SYS1.SYSMDPxx naming convention and an explanation of how the system manages the dump data sets.

Printing a dump

To print a dump for either a SYSABEND or SYSUDUMP DD statement, code one of the following on the DD statement for the output data set:

- A UNIT parameter that specifies a printer.
- The SYSOUT parameter that specifies a print output class.

To print a dump for a SYSMDUMP DD statement, use the following program:

IPCS
This program is described in *z/OS MVS IPCS User's Guide*. When using IPCS, the data set disposition affects the collection of events.

If you print the dump in a JES3 system on a 3800 Printing Subsystem, code CHARS=DUMP for a dump with 204 characters per line and FCB=STD3 for 8 lines per inch.

Overriding dump DD statements

To change the type of dump requested in a dump DD statement in a cataloged or in-stream procedure, the ddname of the overriding DD statement in the calling step must be different from the dump ddname of the procedure DD statement.

Duplicate dump requests

You can code more than one dump request in a job step using DD statements that have **different** ddnames. When you do this, the system uses the last dump DD statement it encounters.

When the system finds dump DD statements with duplicate ddnames, processing is as follows:

- **In a JES2 system**, the job fails with message IEA912I.
- **In a JES3 system**:
  - If both DD statements request JES3- or jointly-managed devices, the job is cancelled during JES3 interpretation.
  - If only one or neither statement requests JES3- or jointly-managed devices, the job fails with message IEA912I.

Examples of the SYSABEND, SYSMDUMP, and SYSUDUMP DD statements

Example 1

```
//STEP2    EXEC PGM=A
//SYSUDUMP DD   SYSOUT=A
```

The SYSUDUMP DD statement specifies that you want the dump routed to system output class A.

Example 2

```
//SYSMDUMP DD  DSNAME=DUMP,DISP=(NEW,KEEP),
//          UNIT=3390,VOLUME=SER=147958
```

The SYSMDUMP DD statement specifies that the dump is to be stored on a tape. Because the LABEL parameter is not coded, the tape must have IBM standard labels.

Example 3

```
//STEP1    EXEC PGM=PROGRAM1
//SYSABEND DD   DSNAME=DUMP,UNIT=3390,DISP=(,PASS,KEEP),
//          VOLUME=SER=1234,SPACE=(TRK,(40,20))
//STEP2    EXEC PGM=PROGRAM2
//SYSABEND DD   DSNAME=*.STEP1.SYSABEND,DISP=(OLD,DELETE,KEEP)
```

Both SYSABEND DD statements specify that the dump is to be stored. The space request in STEP1 is ample and will not inhibit dumping due to insufficient space. If STEP1 does not abnormally terminate but STEP2 does, the system writes the dump for STEP2 in the space allocated in STEP1. In both steps, an abnormal termination disposition of KEEP is specified so that the dump is stored if either of the steps abnormally terminates. If both of the steps successfully execute, the second DISP subparameter, DELETE, in STEP2 instructs the system to delete the data set and free the space acquired for dumping.

Example 4

```
//STEP     EXEC PGM=EXSYSM
//SYSMDUMP DD   UNIT=3390,VOLUME=SER=123456,SPACE=(CYL,(0,1)),
//           DISP=(NEW,DELETE,KEEP),DSNAME=MDUMP
```

The SYSMDUMP DD statement allocates dump data set MDUMP to a direct access device.

Example 5

```
//JOB1     JOB
//STEP     EXEC PGM=EXSYSMDP
//SYSMDUMP DD   DSNAME=SYS1.SYSMDP00,DISP=SHR

//JOB2     JOB
//STEP     EXEC PGM=EXSYSMDP
//SYSMDUMP DD   DSNAME=SYS1.SYSMDP00,DISP=SHR
```

Only the SYSMDUMP dump written by the first job will be in data set SYS1.SYSMDP00. All subsequent jobs receive message IEA849I, indicating that the data set is full.

Note: When you specify a DSNAME of SYS1.SYSMDPxx with DISP=SHR, the system writes the first SYSMDUMP dump on the data set. You must offload this first SYSMDUMP dump and write an EOF mark at the beginning of the SYS1.SYSMDPxx data set before subsequent dumps can be written to that data set.

SYSCHK DD statement

Purpose: Use the SYSCHK DD statement to define a checkpoint data set that the system is to write during execution of a processing program. Use this statement again when the step is restarted from a checkpoint written in the data set.

Note: If restart is to begin at a step, as indicated by the RD parameter on the EXEC statement, do not use a SYSCHK DD statement.

References: For detailed information about the checkpoint/restart facilities, see *z/OS DFSMSdfp Checkpoint/Restart*.

Syntax

```
//SYSCHK DD parameter[,parameter]...  [comments]
```

Parameters on SYSCHK DD statements

When creating a checkpoint data set:

- Code a SPACE parameter, but do not request secondary space.
  - The **primary space** request must be large enough to hold all checkpoints. Although your program or the system can write checkpoints in secondary space, the system **cannot** perform a restart from checkpoints in secondary space.
  - If you do **not** request **secondary space** and the primary space fills up, the job abnormally terminates. You can successfully restart the job at the last checkpoint; however, when the processing program or system writes the next checkpoint the job abnormally terminates again.
  - If you **do** request **secondary space** and the primary space fills up, the processing program or the system writes one invalid checkpoint followed by successful checkpoints. An attempt to restart from one of the checkpoints following the invalid checkpoint results in abnormal termination.
- Code the RLSE subparameter of the SPACE parameter only if the processing program opens the checkpoint data set and the checkpoint data set remains open until the end of the program. If you specify RLSE, the system releases unused space after the first CLOSE macro instruction.

  Do **not** code the RLSE subparameter:
  - If the processing program opens the checkpoint data set before writing each checkpoint and closes the checkpoint data set after writing each checkpoint. The system releases all unused space while closing the data set after the first checkpoint, leaving no space for additional checkpoints.
  - If the system opens the checkpoint data set. The system opens and closes the checkpoint data set before it writes the first checkpoint. With RLSE specified, the system would release all space before the first checkpoint could be written.
- Code the **CONTIG** subparameter of the SPACE parameter to request contiguous space. The system otherwise provides additional primary space using extents. If the extents are **not** contiguous, any checkpoints in these extents cannot be used for a successful restart.

When retrieving a cataloged checkpoint data set:

- Code the DSNAME parameter.
- Code the DISP parameter to specify a status of OLD and a disposition of KEEP.
- Code the VOLUME parameter. If the checkpoint entry is on a tape volume other than the first volume of the checkpoint data set, code the volume serial number or volume sequence number to identify the correct volume. The serial number of the volume on which a checkpoint entry was written appears in the console message issued after the checkpoint entry is written.
- Code the UNIT parameter, if you coded the VOLUME parameter, because the system will not look in the catalog for unit information.

When retrieving a checkpoint data set that is not cataloged:

- Code the DSNAME parameter. If the checkpoint data set is a partitioned data set (PDS), do not code a member-name in the DSNAME parameter.
- Code the DISP parameter to specify a status of OLD and a disposition of KEEP.
- Code the VOLUME parameter. The serial number of the volume on which a checkpoint entry was written appears in the console message issued after the checkpoint entry is written.

- Code the UNIT parameter.

Other parameters:

- Code the LABEL parameter if the checkpoint data set does not have standard labels.
- Code DCB=TRTCH=C if the checkpoint data set is on 7-track magnetic tape with nonstandard labels or no labels.
- If the volume containing the checkpoint data set is to be mounted on a JES3-managed device, do not code the DEFER subparameter of the UNIT parameter on the SYSCHK DD statement.

Note: Do not use VSAM for a checkpoint data set, and do not use a partitioned data set extended (PDSE) for a checkpoint data set.

Relationship to other control statements

Code the RESTART parameter on the JOB statement; without it, the system ignores the SYSCHK DD statement.

Location in the JCL

- When writing checkpoints, place the SYSCHK DD statement after any JOBLIB DD statements, if coded; otherwise, after the JOB statement.
- When restarting a job from a checkpoint, place the SYSCHK DD statement immediately before the first EXEC statement of the resubmitted job.

Examples of the SYSCHK DD statement

Example 1

```
//JOB1   JOB  RESTART=(STEP3,CK3)
//SYSCHK DD   DSNAME=CHLIB,UNIT=3390,
//            DISP=OLD,VOLUME=SER=456789
//STEP1  EXEC PGM=A
```

The checkpoint data set defined on the SYSCHK DD statement is not cataloged.

Example 2

```
//JOB2   JOB  RESTART=(STEP2,NOTE2)
//JOBLIB DD   DSNAME=PRIV.LIB3,DISP=(OLD,PASS)
//SYSCHK DD   DSNAME=CHECKPTS,DISP=(OLD,KEEP),
//            UNIT=3390,VOLUME=SER=438291
//STEP1  EXEC PGM=B
```

The checkpoint data set defined on the SYSCHK DD statement is not cataloged. Note that the SYSCHK DD statement follows the JOBLIB DD statement.

SYSCKEOV DD statement

Purpose: Use the SYSCKEOV DD statement to define a checkpoint data set for checkpoint records from the checkpoint at end-of-volume (EOV) facility. The checkpoint at EOV facility is invoked by a DD CHKPT parameter.

References: For information on the DD CHKPT parameter, see “CHKPT parameter” on page 114. For information on checkpoint/restart facilities, see *z/OS DFSMSdfp Checkpoint/Restart*.

Syntax

```
//SYSCKEOV DD parameter[,parameter]...  [comments]
```

Parameters on SYSCKEOV DD statements

When creating a checkpoint data set:

- Code a SPACE parameter, but do not request secondary space. The **primary space** request must be large enough to hold all checkpoints; if not, the job abnormally terminates.
- Do not code the RLSE subparameter of the SPACE parameter.
- Code the CONTIG subparameter of the SPACE parameter to request contiguous space. The system otherwise provides additional primary space using extents.
- The SYSCKEOV DD statement must define a BSAM data set, but cannot define a partitioned data set extended (PDSE).
- Code DISP=MOD to reduce loss of checkpoint data in case of a system failure during checkpointing.

Other parameters:

- Do not code on the SYSCKEOV DD statement the following:
  - CHKPT=EOV parameter.
  - DCB parameter. All DCB information is provided by the checkpoint at EOV facility.
  - DEFER subparameter of the UNIT parameter.
- If you code the LABEL parameter, you must specify LABEL=(,SL) for IBM standard labels.
- If the SYSCKEOV data set resides on a direct access storage device, that device cannot be shared with another processor.

Location in the JCL

If you code a CHKPT parameter on any DD statements in a job step, place a SYSCKEOV DD statement in the DD statements for the step.

Example of the SYSCKEOV DD statement

```
//SYSCKEOV DD  DSNAME=CKPTDS,UNIT=TAPE,DISP=MOD
```

This statement defines a checkpoint data set for checkpoint at EOV records.

SYSIN DD statement

Purpose: By convention, people often use a SYSIN DD statement to begin an in-stream data set. In-stream data sets begin with a DD * or DD DATA statement; these DD statements can have any valid ddname, including SYSIN. If you omit a DD statement before input data, the system provides a DD * statement with the ddname of SYSIN.

Syntax

```
//SYSIN DD parameter[,parameter]...  [comments]
```

Parameters on SYSIN DD statements

The first parameter is an * or DATA, to signal that an in-stream data set follows immediately.

Do not code a symbolic in place of (one that would resolve to) the * or DATA positional parameter on a SYSIN type DD statement.

Location in the JCL

A SYSIN DD statement appears at the beginning of an in-stream data set.

Examples of SYSIN DD statements

```
//STEP1  EXEC  PGM=READ
//SYSIN  DD    *
         .
         .
         data
         .
//OUT1   DD    SYSOUT=A
//STEP2  EXEC  PGM=WRITE
//SYSIN  DD    DATA,DLM=17
         .
         .
         .
17
```

Chapter 16. EXEC statement

Purpose: Use the EXEC (execute) statement to identify the program or cataloged or in-stream procedure that this job step is to execute and to tell the system how to process the job step. The EXEC statement marks the beginning of each step in a job or a procedure.

A job can have a maximum of 255 job steps. This maximum includes all steps in any procedures the EXEC statements call.

The parameters you can specify for step processing are arranged alphabetically.

References: For information about the JES initialization parameters that provide installation defaults, see *z/OS JES2 Initialization and Tuning Reference* and *z/OS JES3 Initialization and Tuning Reference*.

Description

Syntax

```
//[stepname]  EXEC  positional-parm[,keyword-parm]...[,symbolic-parm=value]...
                                [comments]
```

The EXEC statement consists of the characters // in columns 1 and 2 and four fields: name, operation (EXEC), parameter, and comments.

An EXEC statement is required for each job step.

Name field

A stepname is optional, but is needed for the following. When a stepname is needed, it must be unique within the job, including stepnames in any procedures called by the job. If stepnames are not unique within the job, results might be unpredictable; but in most cases, references to non-unique stepnames will resolve to the first occurrence of that stepname.

- Referring to the step in later job control statements.
- Overriding parameters on an EXEC statement or DD statement in a cataloged or in-stream procedure step.
- Adding DD statements to a cataloged or in-stream procedure step. However, a stepname is not required when adding to the first step in a procedure.
- Performing a step or checkpoint restart at or in the step.
- Identifying a step in a cataloged or in-stream procedure.

Code a stepname as follows:

- The stepname must begin in column 3.
- The stepname is 1 through 8 alphanumeric or national ($, #, @) characters.
- The first character must be alphabetic or national ($, #, @).
- The stepname must be followed by at least one blank.
- The stepname may be preceded by up to 8 alphanumeric or national characters and then separated by a period. If the stepname is coded in this way, the characters up to and including the period are ignored.

Stepnames for started tasks

When JCL runs as a started task, the system assigns a stepname of `ssssssss` (when the START command was S `membername.ssssssss`) or `STARTING` (when the START command was

© Copyright IBM Corp. 1988, 2017

S membername). Embedded procedures that refer back to the invoking procedure, as on a COND parameter, need to specify the stepname the system assigns.

Operation field

The operation field consists of the characters EXEC and must be preceded and followed by at least one blank. It can begin in any column.

Parameter field

An EXEC statement has two kinds of parameters: positional and keyword.

Do not use EXEC statement parameter keywords as symbolic parameters, names, or labels.

Positional Parameters: An EXEC statement must contain **one** of the positional parameters: PGM, PROC, or procedure name. This positional parameter must precede all keyword parameters.

Table 19. Positional parameters

Positional parameters	Values	Purpose

`PGM= {program-name                    }`
`     {*.stepname.ddname             }`
`     {*.stepname.procstepname.ddname}`
`     {JCLTEST                       }`
`     {JSTTEST                       }`
 	
See section "PGM parameter" on page 328
	program-name: 1 - 8 alphanumeric or $, #, @ characters member containing program
	
	stepname: DD in named step
	
	procstepname: step in named procedure
	
	JCLTEST and JSTTEST: scan for syntax without executing the job (JES3 only)
		Names the program the system is to execute or, for JES3 only, requests syntax check without execution

`{PROC=procedure-name}`
`{procedure-name     }`
 	
See section "PROC and procedure name parameters" on page 330
	procedure-name: 1 - 8 alphanumeric or $, #, @ characters	Names the cataloged or in-stream procedure the system is to call and execute.

Keyword parameters: An EXEC statement can contain the following keyword parameters. You can code any of the keyword parameters in any order in the parameter field after the positional parameter.

Table 20. Keyword parameters

Keyword parameters	Values	Purpose
`ACCT[.procstepname]=(accounting-information)` See section "ACCT parameter" on page 310	accounting-information: up to 142 characters [.procstepname]: name of procedure EXEC containing ACCT to be affected	Specifies accounting information for the step

`ADDRSPC[.procstepname]= {VIRT}`
`                        {REAL}`
 	
See section "ADDRSPC parameter" on page 312
	VIRT: virtual (pageable) storage
	
	REAL: central (nonpageable) storage [.procstepname]: name of procedure EXEC containing ADDRSPC to be affected
		Indicates the type of storage required for the step.

Table 20. Keyword parameters (continued)

Keyword parameters	Values	Purpose
`CCSID=nnnnn`	*nnnnn*: 1 - 65535	Specifies the coded character set identifier indicating the character code conversion performed on reads from and writes to tapes accessed in ISO/ANSI Version 4 format.

`COND[.procstepname]=`
`((code,operator[,stepname][.procstepname])  )`
`([,(code,operator[,stepname][.procstepname])]...)`
`([,EVEN]  )`
`([,ONLY]`
`)`
See section "COND parameter" on page 314		

	code: 0 - 4095
	`operator:  GT  Code from`
	`GE  chart on`
	`EQ  section`
	Table 21 on page 318
	`LT`
	`LE`
	`NE`
	EVEN: execute step even if preceding step ended abnormally
	ONLY: execute step only if preceding step ended abnormally
	stepname: step issuing return code
	procstepname: step is in named procedure
	[.*procstepname*]: name of procedure EXEC containing COND to be affected
		Specifies the return code tests used to determine if this step is to be executed or bypassed.

`DYNAMNBR[.procstepname]=n` See section "DYNAMNBR parameter" on page 321	n: 0 - 3273 minus number of DD statements in step [.procstepname]: name of procedure EXEC containing DYNAMNBR to be affected	Holds a number of data set allocations for reuse and sets the control limit.

`MEMLIMIT={nnnnnM}`
`{nnnnnG}`
`{nnnnnT}`
`{nnnnnP}`
`{NOLIMIT}`
See section "MEMLIMIT parameter" on page 322
	n: 0 - 99999	Specifies the limit on the total number of usable virtual pages above the bar in a single address space.

Table 20. Keyword parameters (continued)

Keyword parameters	Values	Purpose

`PARM[.procstepname]=subparameter`
`PARM[.procstepname]=(subparameter,subparameter)`
`PARM[.procstepname]=('subparameter',subparameter)`
`PARM[.procstepname]='subparameter,subparameter'`
See section "PARM parameter" on page 323		

	subparameter: up to 100 characters [.procstepname]: name of procedure EXEC containing PARM to be affected	Passes variable information to the processing program.
`PERFORM[.procstepname]=n` See section "PERFORM parameter" on page 327	n: 1 - 999 [.procstepname]: name of procedure EXEC containing PERFORM to be affected	In WLM compatibility mode (not available on z/OS V1R3 or later systems), specifies the step's performance group. In WLM goal mode, PERFORM on the EXEC statement is ignored except for the TSO logon procedure, where it can be used to classify the TSO user to a service class or report class.

`RD[.procstepname]= {R  }`
`{RNC}`
`{NR }`
`{NC }`
See section "RD parameter" on page 331
	R: restart, checkpoints allowed
	RNC: restart, no checkpoints
	NR: no restart, checkpoints allowed
	NC: no restart, no checkpoints
	[.procstepname]: name of procedure EXEC containing RD to be affected
		In a non-APPC scheduling environment, indicates whether the operator should perform automatic step restart, if the step fails, and controls whether checkpoints are written for CHKPT macros or DD statement CHKPT parameters.

`REGION[.procstepname]={valueK}`
`{valueM}`
See section "REGION parameter" on page 334
	valueK: 1 - 7 digits from 1 - 2096128
	valueM: 1 - 4 digits from 1 - 2047
	[.procstepname]: name of procedure EXEC containing REGION to be affected
		Specifies the amount of space in kilobytes or megabytes required by the step.

Table 20. Keyword parameters (continued)

Keyword parameters	Values	Purpose
`RLSTMOUT[.procstepname]={nnn}` `{0  }`	*nnn*: a value in seconds ranging from 0 to 9999. 0: this value means that the request has no time out value.	Specifies the maximum time in seconds that a VSAM RLS or DFSMStvs request is to wait for a required lock before the request is assumed to be in deadlock.

`TIME[.procstepname]= {([minutes][,seconds])}`
`{1440  }`
`{NOLIMIT  }`
`{MAXIMUM  }`
See section "TIME parameter" on page 339
	minutes: 1 - 357912
	seconds: 1 - 59
	[.procstepname]: name of procedure EXEC containing TIME to be affected
	NOLIMIT: specifies that the step can use the processor for an unlimited amount of time
	MAXIMUM: specifies that the step can use the processor for the maximum amount of time
		Specifies the maximum time the step is to use the processor and requests messages giving the time used.

Keyword parameters on an EXEC statement that calls a procedure: When an EXEC statement positional parameter calls a cataloged or in-stream procedure, all of the EXEC statement's keyword parameters override matching EXEC keyword parameters in the called procedure. If a keyword parameter is to override a parameter on only one EXEC statement in the procedure, code **.procstepname** immediately following the keyword:

```
keyword.procstepname=value
```

The procstepname is the name field on the procedure EXEC statement containing the keyword parameter to be overridden. For example:

```
//STEP1  EXEC  PROC=WKREPORT,ACCT.PSTEPWED=5670
```

The accounting information **5670** applies only to step PSTEPWED in the procedure WKREPORT.

Symbolic parameters on an EXEC statement that calls procedures: An EXEC statement can assign values to, or nullify, symbolic parameters. See "Using system symbols and JCL symbols" on page 37 for more information about symbolic parameters.

Comments field

The comments field follows the parameter field after at least one intervening blank.

Location in the JCL

An EXEC statement must be the first statement in each job step or cataloged or in-stream procedure step.

Examples of EXEC statements

Example 1

```
//STEP4  EXEC PGM=DREC,PARM='3018,NO'
```

The EXEC statement named STEP4 invokes a program named DREC and passes the value in the PARM parameter to DREC.

Example 2

```
//       EXEC PGM=ENTRY,TIME=(2,30)
```

This EXEC statement, which does not have a stepname, invokes a program named ENTRY and specifies the maximum processor time for execution of the step.

Example 3

```
//FOR    EXEC PROC=PROC489,ACCT=DB1528,RD.PSTEP2=RNC,DEV=3390
```

The EXEC statement named FOR invokes a cataloged or in-stream procedure named PROC489. The ACCT parameter applies to all steps in the procedure. The RD parameter applies to only the step named PSTEP2. The DEV parameter assigns the value 3390 to the symbolic parameter Device Support Bucket in a procedure statement.

ACCT parameter

Parameter type

Keyword, optional

Purpose

Use the ACCT parameter to specify one or more subparameters of accounting information that apply to this step. The system passes the accounting information to the installation's accounting routines.

References

For more information on how to add accounting routines, see *z/OS MVS System Management Facilities (SMF)*.

Syntax

```
ACCT[.procstepname]=(accounting-information)
```

Single subparameter: You can omit the parentheses if the accounting information consists of only one subparameter.

Length: The entire accounting-information must not exceed 142 characters:

- Including any commas, which are considered part of the information.
- Excluding any enclosing parentheses or apostrophes, which are not considered part of the information.

Multiple subparameters: When the accounting-information consists of more than one subparameter, separate the subparameters by commas and enclose the information in parentheses or apostrophes. For example, ACCT=(5438,GROUP6) or ACCT='5438,GROUP6'.

Special characters: When a subparameter contains special characters, other than hyphens or plus zero (+0, an overpunch), enclose it in apostrophes and the information in parentheses or enclose all of the information in apostrophes. For example, ACCT=(387,'72/159') or ACCT='387,72/159'.

Code each apostrophe that is part of the accounting-information as two consecutive apostrophes. For example, code DEPT'D58 as ACCT='DEPT''D58'

If you code a symbolic parameter on the ACCT parameter, you can code the symbolic parameter in apostrophes.

Continuation onto another statement: Enclose the accounting-information in parentheses. End each statement with a comma after a complete subparameter. For example:

```
//STEP1  EXEC  PGM=WRITER,ACCT=(1417,J318,'D58/920','CHG=2',
//    '33.95')
```

Subparameter definition

accounting-information
Specifies one or more subparameters of accounting information, as defined by the installation.

On an EXEC statement that calls a procedure

If an EXEC statement calls a cataloged or in-stream procedure, the ACCT parameter overrides the ACCT parameter on or is added to:

- The EXEC statement named in the procstepname qualifier. The information applies only to the named procedure step. The EXEC statement can have as many ACCT.procstepname parameters as the procedure has steps; each ACCT parameter must specify a unique procstepname.
- All EXEC statements in the procedure if procstepname is not coded. Then the information applies to all steps in the called procedure.

Examples of the ACCT parameter

Example 1

```
//STEP1  EXEC PGM=JP5,ACCT=(LOCATION8,'CHGE+3')
```

This EXEC statement executes program JP5 and specifies accounting information for this job step.

Example 2

```
//STP3  EXEC PROC=LOOKUP,ACCT=('/83468')
```

This EXEC statement calls cataloged or in-stream procedure LOOKUP. The accounting information applies to this job step, STP3, and to all the steps in procedure LOOKUP.

Example 3

```
//STP4  EXEC PROC=BILLING,ACCT.PAID=56370,ACCT.LATE=56470,
//      ACCT.BILL='121+366'
```

This EXEC statement calls cataloged or in-stream procedure BILLING. The statement specifies different accounting information for each of the procedure steps: PAID, LATE, and BILL.

ADDRSPC parameter

Parameter type

Keyword, optional

Purpose

Use the ADDRSPC parameter to indicate to the system that the job step requires virtual storage (which is pageable) or central storage (also called real storage, which is nonpageable).

Syntax

```
ADDRSPC[.procstepname]=   {VIRT}
                          {REAL}
```

Subparameter definition

VIRT
Requests virtual storage. The system **can** page the job step.

REAL
Requests central storage (also called real storage). The system **cannot** page the job step and must place the job step in central storage.

Defaults

If no ADDRSPC parameter is specified, the default is VIRT.

Overrides

The JOB statement ADDRSPC parameter applies to all steps of the job and overrides any EXEC statement ADDRSPC parameters.

Code EXEC statement ADDRSPC parameters when each job step requires different types of storage. The system uses an EXEC statement ADDRSPC parameter only when no ADDRSPC parameter is on the JOB statement and only during the job step.

Relationship to the EXEC REGION parameter

When ADDRSPC=REAL: Code a REGION parameter to specify how much central storage the job needs. If you omit the REGION parameter, the system uses the default.

When ADDRSPC=VIRT or ADDRSPC is omitted: Code a REGION parameter to specify how much virtual storage the job needs. If you omit the REGION parameter, the system uses the default.

On an EXEC statement that calls a procedure

If this EXEC statement calls a cataloged or in-stream procedure, the ADDRSPC parameter overrides the ADDRSPC parameter on or is added to:

- The EXEC statement named in the procstepname qualifier. The parameter applies only to the named procedure step. The EXEC statement can have as many ADDRSPC.procstepname parameters as the procedure has steps; each ADDRSPC parameter must specify a unique procstepname.
- All EXEC statements in the procedure if procstepname is not coded. Then the parameter applies to all steps in the called procedure.

Examples of the ADDRSPC parameter

Example 1

```
//CAC1  EXEC PGM=A,ADDRSPC=VIRT
```

This EXEC statement executes program A and requests virtual (pageable) storage. Because the REGION parameter is not specified, the storage available to this job step is the installation default or the region size specified on the JOB statement.

Example 2

```
//CAC2  EXEC PROC=B,ADDRSPC=REAL,REGION=80K
```

This EXEC statement calls procedure B and requests central (nonpageable) storage. The REGION parameter specifies 80K of storage.

CCSID parameter

Parameter type: Keyword, optional

Purpose: You can request the access method to convert data between the coded character set identifier (CCSID) specified on the JOB or EXEC statement and the CCSID specified on the DD statement. Data conversion is supported on access to ISO/ANSI Version 4 tapes using access methods BSAM or QSAM, but not using EXCP.

ISO/ANSI tapes are identified by the LABEL=(,AL) or LABEL=(,AUL) keyword. The CCSID parameter does not apply to ISO/ANSI Version 1 or ISO/ANSI/FIPS Version 3 tapes or to tapes with labels other than AL or AUL. See *z/OS DFSMS Using Data Sets*for selecting ISO/ANSI Version 4 tapes. It also contains a list of supported CCSIDs.

The CCSID value of 65535 has a special meaning: it suppresses conversion.

When CCSID is not specified at the JOB, EXEC, or DD levels, data passed to BSAM and QSAM is converted to 7-bit ASCII when writing to ISO/ANSI tapes. This might result in data loss on conversion. On READ operations the CCSID (if recorded) on the tape header label is used for conversion.

The CCSID is recorded in the tape header label if conversion is not defaulted.

Syntax

```
CCSID= nnnnn
```

Subparameter definition

nnnnn
The CCSID as a decimal number from 1 through 65535.

Default

If no CCSID parameter is specified on the JOB statement, the default is 500.

Relationship to other parameters

Do not code the following parameters with the CCSID parameter:

*	DDNAME	QNAME
BURST	DYNAM	SYSOUT
CHARS	FCB	TERM
COPIES	FLASH	UCS
DATA	MODIFY	

Examples of the CCSID parameter

For examples of the CCSID parameter see “Examples of the CCSID parameter” on page 111.

COND parameter

Parameter type

Keyword, optional

Purpose

Use the COND parameter to test return codes from previous job steps and determine whether to bypass this job step. You can specify one or more tests on the COND parameter, and you can test return codes from particular job steps or from every job step that has completed processing. If any of the test conditions are satisfied, the system evaluates the COND parameter as true and bypasses the job step. If none of the test conditions specified on the COND parameter are satisfied, the system evaluates the COND parameter as false and executes the job step.

The system performs the COND parameter tests against return codes from the current execution of the job. If a test returns a previously bypassed step, the system evaluates the test as false.

Bypassing a step because of a return code test is not the same as abnormally terminating the step. The system abnormally terminates a step following an error so serious that it prevents successful execution. In contrast, bypassing of a step is merely its omission.

If a step abnormally terminates, the system normally bypasses all following steps in the job unless the step(s) are part of an IF/THEN/ELSE/ENDIF construct that specifies the ABEND, ABENDCC, or ¬ABEND keywords, described in Chapter 18, “IF/THEN/ELSE/ENDIF statement construct,” on page 353. Another way to make the system execute a following step, for instance, to write a dump, is to code EVEN or ONLY on that step’s EXEC statement. The EVEN or ONLY subparameters are interpreted first. If they indicate that the step should be executed, then the return code tests, if specified, are performed. If no return code tests were typed or if none of the coded tests is satisfied, the system executes the step. Finally, steps following a step that terminated abnormally might execute. This occurs if the step that abended contained a recovery routine like ESPIE, ESTAE or FRR that intercepted the abend and requested that normal termination occur.

Instead of coding a JOB statement COND parameter, code an EXEC statement COND parameter when you want to:

- Specify different tests for each job step.
- Name a specific step whose return code the system is to test.
- Specify special conditions for executing a job step.
- Bypass only one step. When a step is bypassed because of a JOB COND parameter, all following steps in the job are bypassed.

Note: Depending on the program invoked, a test showing that a return code from a step is zero is not sufficient to verify that the step did not fail. The system can fail a step (or job) even if the return code is

zero. For example, this could happen as a result of specifying CATLG_ERR FAILJOB(YES) and incurring a "post execution error". To determine if a step failed due to a "post execution error", the SMF type 30, subtype 4 record for the job step can be examined. In this record, bit SMF30SYE in the two-byte SMF30STI field will be on if the job failed due to a "post execution error".

Syntax

```
COND[.procstepname] = (code,operator)
COND[.procstepname] = ((code,operator[,stepname][.procstepname])
                        [,(code,operator[,stepname][.procstepname])]... [,EVEN])
                                                                        [,ONLY]
COND=EVEN
COND=ONLY
```

- One return code test is: (code,operator)
- You can omit the outer parentheses if you code only one return code test or only EVEN or ONLY.
- Specify up to eight return code tests. However, if you code EVEN or ONLY, specify up to seven return code tests.
- You can omit all return code tests and code only EVEN or ONLY.
- Place the EVEN or ONLY subparameters before, between, or after the return code tests.
- Null positional subparameters of the COND parameter are invalid.

Subparameter definition

code
Specifies a number that the system compares to the return codes from all previous steps in the job or from specific steps. ***code*** is a decimal number from 0 through 4095.

Note: Specifying a decimal number greater than 4095 could result in invalid return code testing or invalid return codes in messages.

operator
Specifies the type of comparison to be made to the return code. If the specified test is true, the step is bypassed. Use Table 21 on page 318 to select the correct operator. Operators and their meanings are:

Operator	Meaning
GT	Greater than
GE	Greater than or equal to
EQ	Equal to
LT	Less than
LE	Less than or equal to
NE	Not equal to

stepname
Identifies the EXEC statement of a previous job step that issues the return code to be used in the test. If the specified step is in a procedure, this step must be in the same procedure. Otherwise, the specified step must not be in a procedure; the specified step must contain a PGM keyword, rather than invoke a procedure. Note that if stepnames are not unique within the job, such as when the same procedure is executed multiple times, results might be unpredictable; but in most cases, references to non-unique stepnames will resolve to the first occurrence of that stepname.

If you omit stepname, the code you specify is compared to the return codes from all previous steps. If the return code issued by any of those previous steps causes the test condition to be satisfied, the system evaluates the COND parameter as true and bypasses the job step.

If this step is invoked in JCL that runs as a started task, see "Stepnames for started tasks" on page 305 for information about the stepname the system assigns.

stepname.procstepname
Identifies a step in a cataloged or in-stream procedure called by an earlier job step. Stepname identifies the EXEC statement of the calling job step; procstepname identifies the EXEC statement of the procedure step that issues the return code to be used in the test. The step identified by procstepname must contain the PGM keyword, rather than invoke a procedure. Note that if stepnames are not unique within the job, such as when the same procedure is executed multiple times, results might be unpredictable; but in most cases, references to non-unique stepnames will resolve to the first occurrence of that stepname.

EVEN
Specifies that this job step is to be executed **even if** a preceding job step abnormally terminated. When EVEN is coded, the system:

- Does not test the return code of any steps that terminated abnormally.
- Does test the return code of any steps that terminated normally. If none of the return code tests for these steps is satisfied, this job step is executed.

See "Considerations when using the COND parameter" on page 317 for cautions related to the use of EVEN.

ONLY
Specifies that this job step is to be executed **only if** a preceding step abnormally terminated. When ONLY is coded, the system:

- Does not test the return code of any steps that terminated abnormally.
- Does test the return code of any steps that terminated normally. If none of the return code tests for these steps is satisfied, this job step is executed.

See "Considerations when using the COND parameter" on page 317 for cautions related to the use of ONLY.

Overrides

If you code the COND parameter on the JOB statement and on one or more of the job's EXEC statements, and if a return code test on the JOB statement is satisfied, the job terminates. In this case, the system does not process any subsequent EXEC statement COND parameters.

If the tests on the JOB statement are not satisfied, the system then performs the return code tests on the EXEC statement. If a return code test is satisfied, the step is bypassed.

Location in the JCL

You can specify the COND parameter on any EXEC statement in the job. However, the system evaluates a COND parameter on the first EXEC statement in a job as false.

On an EXEC statement that calls a procedure

The COND parameter on an EXEC statement that calls a cataloged or in-stream procedure (a calling EXEC statement) either overrides or is added to the called EXEC statements.

The COND on the calling EXEC statement overrides the COND on the called EXEC statement. If the called EXEC statement does not have COND coded on it, the COND on the calling EXEC statement will be added to the called EXEC statement.

If an EXEC statement calls a cataloged or in-stream procedure, the COND parameter overrides the COND parameter on or is added to:

- The EXEC statement named in the procstepname qualifier, which is to the left of the equals sign. The parameter applies only to the named procedure step. The EXEC statement can have as many COND.procstepname parameters as the procedure has steps; each COND parameter must specify a unique procstepname.
- All EXEC statements in the procedure if procstepname is not coded. Then the parameter applies to this job step and to all steps in the called procedure.

Considerations when using the COND parameter

Be aware of the following considerations when specifying COND parameters. Some of these considerations relate to errors that prevent step execution, no matter what is specified on the COND parameter, while others are related to the use of the COND parameter.

Errors that prevent step execution, regardless of COND specifications

Certain error conditions prevent the system from executing a step, regardless of any requests specified through the COND parameter. These conditions are as follows:

Abnormal termination by the system: After certain types of abnormal termination by the system, remaining job steps are not executed, regardless of whether EVEN or ONLY were specified. The completion codes associated with these types of abnormal termination are:

122
Operator canceled job

222
Operator or TSO/E user canceled job

You might encounter other system completion codes for which remaining job steps are not executed, regardless of whether EVEN or ONLY was specified. See *z/OS MVS System Codes* for further information about specific system completion codes.

Backward references to data sets: If a step is bypassed because of its COND parameter or if a step abnormally terminates, a data set that was to have been created or cataloged in the step may not exist, may not be cataloged, or may be incomplete. Thus, a job step should not refer to a data set being created or cataloged in a step that could be bypassed or abnormally terminated. If the job step does make such a reference, the system might not be able to execute the step.

When the program does not have control: For the system to act on the COND parameter, the step must abnormally terminate while the program has control. If a step abnormally terminates during scheduling, due to failures such as JCL errors or the inability to allocate space, the system bypasses the remaining steps, no matter what the COND parameter requests.

JES3 considerations

In both JES2 and JES3 systems, an EXEC COND parameter determines if a step is executed or bypassed. However, JES3 processes all jobs as though each step will execute; therefore, JES3 allocates devices for steps that are bypassed. JES3 will fail jobs that delete a data set in one step and attempt to reference the deleted data set in a later step, even if the step that deletes the data set is bypassed during execution. JES3 does not support conditional JCL, although it does permit conditional statements to be specified.

COND parameter on the first statement in a Job

The system evaluates a COND parameter on the first EXEC statement in a job as false.

JOBLIB with COND=ONLY

If the job contains a JOBLIB DD statement and ONLY is specified in a job step, the JOBLIB unit and volume information are not passed to the next step; when the next step is executed, the system searches the catalog for the JOBLIB data set.

When the JOB statement contains a RESTART parameter

When restarting a job, the restart step becomes, in effect, the first step in the job. Therefore, the system evaluates a COND parameter on the restart step as false and executes the step. Subsequent steps might be executed. When a COND parameter on a step following the restarted step refers to a step that precedes the restarted step, the system evaluates the COND parameter as false. If all other COND parameters on that step are also false, the system executes the step. When the JOB statement contains a RESTART parameter with a checkpoint id, the system evaluates the COND parameter on the designated restart step as false and executes the step.

Restarted step	COND parameter processing
The restarted step does not call a procedure and is not a step within a procedure.	The system evaluates any COND parameters on the restarted step as false, and executes the step.
The restarted step calls a procedure and does not contain a COND parameter.	The system evaluates any COND parameters on the first step to be executed within the procedure at restart as false, and executes the step. Subsequent steps containing COND parameters are processed normally.
The restarted step is within a procedure, and the step that called the procedure does not contain a COND parameter.	The system evaluates any COND parameters on the first step to be executed within the procedure at restart as false, and executes the step. Subsequent steps containing COND parameters are processed normally.
The restarted step calls a procedure, and the restarted step contains a COND parameter without a procstepname qualifier.	The system evaluates any COND parameters on the restarted step as false, and executes the step. The system evaluates any COND parameters on steps within the called procedure as false, regardless of whether they were overridden or added from the COND parameter on the step that called the procedure.
The restarted step is within a procedure, and the step that called the procedure contains a COND parameter without a procstepname qualifier.	The system evaluates any COND parameters on the restarted step as false, and executes the step. Any subsequent steps within the procedure that contain COND parameters are processed normally.
The restarted step calls a procedure, and the restarted step contains one or more COND parameters with procstepname qualifiers.	The system evaluates any COND parameters on the first step to be executed within the procedure at restart as false, and executes the step. Subsequent steps that contain COND parameters are processed normally. COND parameters on these subsequent steps are added or overridden as specified in the calling step.
The restarted step is within a procedure, and the step that called the procedure contains one of more COND parameters with procstepname qualifiers.	The system evaluates any COND parameters on the first step to be executed within the procedure at restart as false, and executes the step. Subsequent steps that contain COND parameters are processed normally. COND parameters on these subsequent steps are added or overridden as specified in the calling step.

Summary of COND parameters

Table 21. Bypassing or Execution of Current Step Based on COND Parameter

	Return Code (RC) from a previous step	
Test in COND parameter	**Execute current step**	**Bypass current step**
COND=(code,GT)	code <= RC	code > RC

Table 21. Bypassing or Execution of Current Step Based on COND Parameter (continued)		
	Return Code (RC) from a previous step	
Test in COND parameter	**Execute current step**	**Bypass current step**
COND=(code,GE)	code < RC	code >= RC
COND=(code,EQ)	code ¬= RC	code = RC
COND=(code,LT)	code >= RC	code < RC
COND=(code,LE)	code > RC	code <= RC
COND=(code,NE)	code = RC	code ¬= RC
Note: When the COND parameter does not name a previous step, the system tests all previous steps. If any test is satisfied, the system takes action on the current step, per the char above.		

Table 22. Effect of EVEN and ONLY Subparameters on Step Execution			
EVEN or ONLY Specified?	**Any Preceding Abend?**	**Any Tests Satisfied?**	**Current Step Execute?**
EVEN	No	No	Yes
EVEN	No	Yes	No
EVEN	Yes	No	Yes
EVEN	Yes	Yes	No
ONLY	No	No	No
ONLY	No	Yes	No
ONLY	Yes	No	Yes
ONLY	Yes	Yes	No
Neither	No	No	Yes
Neither	No	Yes	No
Neither	Yes	No	No
Neither	Yes	Yes	No

Examples of the COND parameter

Example 1:

```
//STEP6  EXEC PGM=DISKUTIL,COND=(4,GT,STEP3)
```

In this example, if the return code from STEP3 is 0 through 3, the system bypasses STEP6. If the return code is 4 or greater, the system executes STEP6. Because neither EVEN nor ONLY is specified, the system does not execute this step if a preceding step abnormally terminates.

Example 2:

```
//TEST2  EXEC PGM=DUMPINT,COND=((16,GE),(90,LE,STEP1),ONLY)
```

The system executes this step ONLY if two conditions are met:

1. A preceding job step abnormally terminated.
2. No return code tests are satisfied.

Therefore, the system executes this step only when all three of the following are true:

- A preceding job step abnormally terminated.
- The return codes from all preceding steps are 17 or greater.
- The return code from STEP1 is 89 or less.

The system bypasses this step if any one of the following is true:

- All preceding job steps terminated normally.
- The return code from any preceding step is 0 through 16.
- The return code from STEP1 is 90 or greater.

Example 3:

```
//STEP1  EXEC  PGM=CINDY
          .
          .
//STEP2  EXEC  PGM=NEXT,COND=(4,EQ,STEP1)
          .
          .
//STEP3  EXEC  PGM=LAST,COND=((8,LT,STEP1),(8,GT,STEP2))
          .
```

In this example, if STEP1 returns a code of 4, STEP2 is bypassed. Before STEP3 is executed, the system performs the first return code test. If 8 is less than the return code from STEP1, STEP3 is bypassed; or, restated, if the STEP1 return code is less than or equal to 8, STEP3 is executed. Because 4 is less than 8, STEP3 is executed.

The system does not perform the second return code test because STEP2 was bypassed.

Example 4:

```
//STP4  EXEC PROC=BILLING,COND.PAID=((20,LT),EVEN),
//     COND.LATE=(60,GT,FIND),
//     COND.BILL=((20,GE),(30,LT,CHGE))
```

This statement calls cataloged or in-stream procedure BILLING. The statement specifies different return code tests for each of the procedure steps: PAID, LATE, and BILL. The system executes step PAID even if a preceding step abnormally terminates unless the accompanying return code is satisfied.

Example 5: The procedure TEST exists in SYS1.PROCLIB:

```
//TEST      PROC
//PROCSTP1  EXEC PGM=IEFBR14,COND=(0,NE)
//PROCSTP2  EXEC PGM=IEFBR14,COND=(0,NE)
//PROCSTP3  EXEC PGM=IEFBR14
//PROCSTP4  EXEC PGM=IEFBR14,COND=(4,LT)
//          PEND
```

The job:

```
//JOB1      JOB...RESTART=JOBSTEP
//JOBSTEP   EXEC PROC=TEST
```

JOB1 restarts at JOBSTEP. PROCSTP1 is the first step in the job because of the RESTART specification, and the COND parameter test is not valid because no previous steps have run. Therefore, the system evaluates the COND parameter for PROCSTP1 as false, and PROCSTP1 runs. PROCSTP3 has no COND parameter. The COND parameters for PROCSTP2 and PROCSTP4 are used.

The job:

```
//JOB1      JOB...RESTART=JOBSTEP.PROCSTP2
//JOBSTEP   EXEC PROC=TEST,COND=(8,GT)
```

JOB1 restarts at PROCSTP2 as called by JOBSTEP. The COND parameter on JOBSTEP does not specify a procstepname qualifier and therefore applies to all steps in procedure TEST. The system evaluates the COND parameter for PROCSTP2, the restart step, as false, and the step runs. However, the COND parameter for steps PROCSTP3 and PROCSTP4 evaluates as true (because 8 is greater than the return code of 0 provided by all previous steps in the job), and the steps are bypassed.

The job:

```
//JOB1      JOB...RESTART=JOBSTEP.PROCSTP2
//JOBSTEP   EXEC PROC=TEST,COND.PROCSTP4=(8,GT)
```

JOB1 restarts at PROCSTP2 as called by JOBSTEP. Because of the RESTART specification, PROCSTP2 is the first step in the job. The system evaluates the COND parameter for PROCSTP2 as false, and the step runs. PROCSTP3 has no COND parameter. PROCSTP4 is overridden as specified on JOBSTEP.

DYNAMNBR parameter

Parameter type

Keyword, optional

Purpose

Use the DYNAMNBR parameter to tell the system to hold a number of resources in anticipation of reuse. Code DYNAMNBR instead of several DD statements with DYNAM parameters.

Syntax

```
DYNAMNBR[.procstepname]=n
```

Subparameter definition

n

Specifies a value used to calculate the maximum number of data set allocations that the system can hold in anticipation of reuse. Specify n as a decimal number from 0 through 3273 minus the number of DD statements in the step.

Note that the limit of 3273 is based on the number of single unit DD statements for a 64K TIOT (task input output table). This limit can be different depending on the installation-defined TIOT size. 32K is the default TIOT size. The limit for a 32K TIOT is 1635. (In a JES3 system, the installation might further reduce the limit.)

Note: If you specify DISP=(NEW,PASS) but, at the end of the job, one or more data sets were not received by any job step, then the maximum number of DD statements you can specify decreases by one. For example, if the current limit is 1635 DD statements, you can specify DISP=(NEW,PASS), and up to 1634 DD statements. See the *z/OS MVS Initialization and Tuning Reference*, under ALLOCxx, for more TIOT information.

The number of resources that the system actually holds in anticipation of reuse equals n plus the number of DD statements in the step, including any DD statements in a cataloged or in-stream procedure called by the step. The system uses this sum of n plus the number of DD statements in the step to establish a control limit. See the *z/OS MVS JCL User's Guide* for Dynamic Allocation and Control Limit information. See the *z/OS MVS Programming: Authorized Assembler Services Guide* for additional Control Limit information.

Defaults

If no DYNAMNBR parameter is specified, the default is 0. If you code DYNAMNBR incorrectly, the system uses the default of 0 and issues a JCL warning message.

On an EXEC statement that calls a procedure

If this EXEC statement calls a cataloged or in-stream procedure, the DYNAMNBR parameter overrides the DYNAMNBR parameter on or is added to:

- The EXEC statement named in the procstepname qualifier. The parameter applies only to the named procedure step. The EXEC statement can have as many DYNAMNBR.procstepname parameters as the procedure has steps; each DYNAMNBR parameter must specify a unique procstepname.
- All EXEC statements in the procedure if procstepname is not coded. Then the parameter applies to all steps in the called procedure.

Example of the DYNAMNBR parameter

```
//STEP1  EXEC PROC=ACCT,DYNAMNBR.CALC=12
```

For the procedure step CALC, this statement specifies that the system should hold the following data set allocations for reuse: 12 plus the number of DD statements following this EXEC statement and the number of DD statements in procedure ACCT.

MEMLIMIT parameter

Parameter type

Keyword, optional

Purpose

Use the MEMLIMIT parameter to specify the limit on the total size of usable virtual storage above the bar in a single address space.

Syntax

```
MEMLIMIT={nnnnnM}
         {nnnnnG}
         {nnnnnT}
         {nnnnnP}
         {NOLIMIT}
```

Subparameter definition

nnnnnM
nnnnnG
nnnnnT
nnnnnP
Specifies a value to be used as the limit on the total size of usable virtual storage above the bar in a single address space. The value may be expressed in megabytes (M), gigabytes (G), terabytes (T), or petabytes (P). nnnnn may be a value from 0 to 99999, with a maximum value of 16384P.

NOLIMIT
Specifies that there is no limit on the virtual storage to be used above the bar.

Note: Unlike the REGION parameter, MEMLIMIT=0M (or equivalent in G, T, or P) means that the step can not use virtual storage above the bar.

Defaults

If no MEMLIMIT parameter is specified, the default is the value defined to SMF, except when REGION=0K/0M is specified, in which case the default is NOLIMIT.

Overrides

The JOB statement MEMLIMIT parameter applies to all steps of the job and overrides any EXEC statement MEMLIMIT parameter.

If MEMLIMIT is not specified, SMF provides a default value. The IEFUSI installation exit can override any JCL- or SMF-supplied value.

Relationship to the REGION parameter

A specification of REGION=0K/0M will result in a MEMLIMIT value being set to NOLIMIT, when a MEMLIMIT value has not been specified on either the JOB or EXEC statements, and IEFUSI has not been used to set the MEMLIMIT.

Considerations when using the MEMLIMIT parameter

Specifying a REGION size that gives the job all the available storage, such as 0 K or any value greater than 16,384 K, can cause storage problems if the IBM- or installation-supplied routine IEALIMIT or IEFUSI is not used to establish a limiting value.

Note: Changes to the MEMLIMT parameter might also require changes to associated resource group memory limits. For more information on virtual to real storage relationships, see the *Processor storage management* section in *z/OS MVS Programming: Authorized Assembler Services Guide*.

Example of the MEMLIMIT parameter

```
//STEPA  EXEC PGM=ADDER,MEMLIMIT=10000M
```

This job step specifies a limit of 10000 megabytes of usable virtual storage above the bar, depending on other job and installation factors.

PARM parameter

Parameter type

Keyword, optional

Purpose

Use the PARM parameter to pass variable information to the processing program executed by this job step. To use the information, the processing program must contain instructions to retrieve the information.

References

For details on the format of the passed information and its retrieval, see *z/OS MVS Programming: Assembler Services Guide*.

Syntax

```
PARM[.procstepname]=subparameter
PARM[.procstepname]=(subparameter,subparameter)
PARM[.procstepname]=('subparameter',subparameter)
PARM[.procstepname]='subparameter,subparameter'
```

Length: The length of the subparameters passed must not exceed 100 characters:

- Including any commas, which are passed to the processing program.
- Excluding any enclosing parentheses or apostrophes, which are not passed.

For example, PARM='P1,123,MT5' is received by the program as P1,123,MT5.

Commas: When you code more than one subparameter, separate the subparameters by commas and enclose the subparameters in parentheses or apostrophes. For example, PARM=(P1,123,MT5) or PARM='P1,123,MT5'.

Special characters and blanks: When a subparameter contains special characters or blanks, enclose it in apostrophes and the other subparameters in parentheses, or enclose all the subparameters in apostrophes. For example, PARM=(P50,'12+80') or PARM='P50,12+80'.

Code each apostrophe and ampersand that is part of the subparameter as two consecutive apostrophes or ampersands. For example, code 3462&5 as PARM='3462&&5'.

However, if a subparameter contains a symbolic parameter, code a single ampersand. You can code the symbolic parameter in apostrophes.

Continuation onto another statement: Enclose the subparameters in parentheses. End each statement with a comma after a subparameter. For example:

```
//STEP1  EXEC    PGM=WORK,PARM=(DECK,LIST,'LINECNT=80',
//    '12+80',NOMAP)
```

Do not code an apostrophe in column 71; see “Continuing parameter fields enclosed in apostrophes” on page 17 if you need more information.

Subparameter definition

subparameter
Consists of the information to be passed to the processing program.

On an EXEC statement that calls a procedure

If an EXEC statement calls a cataloged or in-stream procedure, the PARM parameter overrides the PARM parameter on or is added to:

- The EXEC statement named in the procstepname qualifier. The information applies only to the named procedure step. The EXEC statement can have as many PARM.procstepname parameters as the procedure has steps; each PARM parameter must specify a unique procstepname.
- The **first** EXEC statement in the procedure if procstepname is not coded; the system **nullifies any PARM parameters on any following EXEC statements in the procedure**. The information applies to only the first step in the called procedure.

Examples of the PARM parameter

Example 1

```
//RUN3  EXEC PGM=APG22,PARM='P1,123,P2=5'
```

The system passes P1,123,P2=5 to the processing program named APG22.

Example 2

```
//   EXEC PROC=PROC81,PARM=MT5
```

The system passes MT5 to the first step of the procedure named PROC81. If PROC81 contains more steps and their EXEC statements contain PARM parameters, the system nullifies those PARM parameters.

Example 3

```
//STP6  EXEC PROC=ASMFCLG,PARM.LKED=(MAP,LET)
```

The system passes MAP,LET to the procedure step named LKED in procedure ASMFCLG. If any other procedure steps in ASMFCLG contain a PARM parameter, those PARM parameters remain in effect.

Example 4

```
//RUN4  EXEC PGM=IFOX00,PARM=(NOOBJECT,'LINECNT=50', 'TRUNC(BIN)',
//   DECK)
```

The system passes NOOBJECT,LINECNT=50,TRUNC(BIN),DECK to processing program IFOX00. Because the PARM parameter contains a list of more than one subparameter, the information is enclosed in parentheses.

PARMDD parameter

Parameter type

Keyword, optional

Purpose

Use the PARMDD parameter along with a DD statement to pass variable information to the processing program executed by this job step. To use the information, the processing program must contain instructions to retrieve the information. The format of the data that is generated by the PARMDD parameter is compatible with the format of the data that is generated by the PARM parameter, except that PARMDD data is not restricted to a length of 100. For more information about the format of the passed information and its retrieval, see Conventions for passing information through a parameter list.

Specifying the PARMDD keyword causes the job to be scheduled on systems that are at or above the z/OS 2.1 level.

The DD statement can be a SYSIN DD (a DD coded as DD * or DD DATA) or it can reference a data set, UNIX System Service file, or DD statement in a previous step (using DSN=*.stepname.ddname).

The PARMDD keyword is mutually exclusive with the PARM keyword.

References

For details on the format of the passed information and its retrieval, see *z/OS MVS Programming: Assembler Services Guide*.

If the program to be invoked is from an APF-authorized library or is bound with the AC(1) attribute, see *z/OS MVS Program Management: User's Guide and Reference* for a discussion of the Binder LONGPARM option.

Syntax

```
//STEP1   EXEC PGM=pgm,PARMDD=ddname
```

Relationship to other control statements

The DD name specified on the PARMDD statement must exist on a DD statement within the step.

Data set requirements

The data set that is associated with the DDname specified on the PARMDD keyword must be a physical sequential (PS) data set. Other data set organizations are rejected and the job is failed. When using PARMDD, you must comply with the following support statements:

- Partitioned data set (PDS and PDSE) members provide the appearance of a physical sequential (PS) data set and are supported.
- z/OS UNIX System Services files also provide the appearance of a physical sequential (PS) data set and are supported.
- Other data sets that provide the appearance of a physical sequential (PS) data set are not supported.

The data set can have a fixed (F), fixed-block (FB), variable (V) or variable-block (VB) record format (RECFM). Spanned (S) and undefined (U) record formats are rejected and the job is failed.

Record length requirements

The maximum supported record length (LRECL) is 32760 bytes. Fixed record length data sets are examined to determine whether their records contain sequence numbers. If sequence numbers exist, they are assumed to consist of eight contiguous numeric characters occupying the last eight bytes of the input record. If a record is found to contain a sequence number, the length of the input record is adjusted to remove the sequence number.

Note: Ensure that you do not place numeric data that is intended as parameter data in the final eight characters of fixed length records or in-stream data records.

Parameter string requirements

A PARMDD parameter string can be formed by concatenating multiple data sets or files, subject to the constraints of the BSAM like data set concatenation rules.

The parameter string that is passed to the job step program is formed by a simple concatenation of each input record, up to a maximum of 32760 bytes. Blank records and trailing blanks on each record are ignored during the concatenation process. Input in excess of 32760 bytes (after any symbolic substitution, sequence number removal, and trailing blank removal) results in an error message written to the job log and the job being terminated. An input record can contain blank characters that are to become part of the parameter string, but you must end any sequence of blank characters on an input record with a non-blank character, or the blank characters will be ignored.

After concatenation, the parameter string that is passed to the job step program is examined for double ampersand character (&&) sequences. Double ampersands are converted to single ampersands in the same way that double ampersands are converted to single ampersands by PARM= processing.

When the PARMDD= parameter references an in-stream (SYSIN) data set, the DD statement can use the SYMBOLS= parameter, and the data can contain symbols if the symbol name is exported (See “SYMBOLS parameter” on page 259.)

Note: Note that substitution logging can be requested using the SYMBOLS parameter, but will be ignored.

In the following example, the parameter string that is presented to the program MYPGM is SBJ.DASD.LOAD:

```
//       EXPORT SYMLIST=SYMB1
//                SET SYMB1=DASD
//STEP1   EXEC PGM=MYPGM,PARMDD=MYPARMS
//MYPARMS DD   *,SYMBOLS=JCLONLY,DLM=$$
SBJ.&SYMB1..LOAD
```

If the job runs in a job class with the SYSSYM setting enabled, it can also use system symbols within the SYSIN data set.

Parameter strings that contain ampersand (&) characters are examined for symbol names. If there is no valid symbol name after the ampersand (&) character, the string is left unchanged, with the following exception: parameter strings that contain double ampersand characters (&&) within the string are converted to single ampersand characters, as they are done for the PARM= parameter string.

Examples of the PARMDD parameter

Example 1

```
//STEP1   EXEC PGM=IEBCOPY,PARMDD=PARMIN
//PARMIN  DD   *,DLM='/*'
LINECOUNT=75
/*
```

In the example, the PARMDD keyword specifies a DD name of PARMIN, which is then coded on a DD statement that specifies a SYSIN (or in-stream) data set. The DD DATA usage would be similar.

Example 2

```
//STEP1   EXEC PGM=MYPGM,PARMDD=MYPARMS
//MYPARMS DD   DSN=SYS1.PARMLIB(MYPGMPRM)
```

In the example, the PARMDD keyword specifies a DD name of MYPARMS, which is then coded on a DD statement that specifies a data set (in this case, a partitioned data set member) that contains the program's parameter information.

Example 3

```
//STEP1   EXEC PGM=MYPGM,PARMDD=MYPARMS
//MYPARMS DD   PATH=/SYSTEM/tmp/unixparm.txt
```

In the example, the PARMDD keyword specifies a DD name of MYPARMS, which is then coded on a DD statement that specifies a UNIX System Service file that contains the program's parameter information.

PERFORM parameter

Parameter type

Keyword, optional

Purpose

> **Important**
>
> Beginning with z/OS V1R3, WLM compatibility mode is no longer available. Accordingly, the information that pertains specifically to WLM compatibility mode is no longer valid. It has been included for reference purposes, and for use on backlevel systems.

Use the PERFORM parameter in WLM compatibility mode to specify the performance group for the job step. The installation-defined performance groups determine the rate at which associated steps have access to the processor, storage, and channels.

In WLM goal mode, any PERFORM parameter on an EXEC statement for a job or a started procedure is ignored. However, for a TSO session, a PERFORM parameter specified on the EXEC statement of the TSO logon procedure, or entered on the TSO logon panel, can be used for classification of the session to a service class or report class. For details on how to use workload management classification rules to map a PERFORM value to a service class or report class, see *z/OS MVS Planning: Workload Management*.

Syntax

```
PERFORM[.procstepname]=n
```

Subparameter definition

n

The n is a number from 1 through 999.

In WLM compatibility mode, n identifies a performance group that has been defined by your installation. The specified performance group should be appropriate for your step type according to your installation's rules.

Defaults

In WLM compatibility mode, if no PERFORM parameter is specified or if the specified PERFORM number fails validity checks, the system uses an installation default specified at initialization. If the installation did not specify a default, the system uses a built-in default:

Default	Use
1	For non-TSO/E job steps
2	For TSO/E sessions

See *z/OS MVS Initialization and Tuning Guide* for details.

Overrides

A JOB statement PERFORM parameter applies to all steps of the job and overrides any EXEC statement PERFORM parameters.

Code EXEC statement PERFORM parameters when each job step is to execute in a different performance group. The system uses an EXEC PERFORM parameter only when no PERFORM parameter is on the JOB statement and only during the job step.

On an EXEC statement that calls a procedure

If an EXEC statement calls a cataloged or in-stream procedure, the PERFORM parameter overrides the PERFORM parameter on or is added to:

- The EXEC statement named in the procstepname qualifier. The parameter applies only to the named procedure step. The EXEC statement can have as many PERFORM.procstepname parameters as the procedure has steps; each PERFORM parameter must specify a unique procstepname.
- All EXEC statements in the procedure if procstepname is not coded. Then the parameter applies to all steps in the called procedure.

Example of the PERFORM parameter

```
//STEPA  EXEC PGM=ADDER,PERFORM=60
```

This job step will be run in performance group 60 if it passes validity checks. The installation must have defined the significance of this performance group.

PGM parameter

Parameter type

Positional, optional

Purpose

Use the PGM parameter to name the program that the system is to execute. The specified program must be a member of a partitioned data set (PDS) or partitioned data set extended (PDSE) used as a system library, a private library, or a temporary library.

Syntax

```
PGM= {program-name                   }
     {*.stepname.ddname              }
     {*.stepname.procstepname.ddname}
     {JCLTEST                        }
     {JSTTEST                        }
```

The EXEC statement parameter field must begin with a PGM parameter or a PROC parameter. These two parameters must not appear on the same EXEC statement.

Subparameter definition

program-name
Specifies the member name or alias of the program to be executed. The program-name is 1 through 8 alphanumeric or national ($, #, @) characters; the first character must be alphabetic or national.

Use this form of the parameter when the program resides in a system library, such as SYS1.LINKLIB, or in a private library specified in the job by a JOBLIB DD statement or in the step by a STEPLIB DD statement.

***.stepname.ddname**
Refers to a DD statement that defines, as a member of a partitioned data set (PDS) or a partitioned data set extended (PDSE), the program to be executed. Stepname identifies the EXEC statement of the earlier job step that contains the DD statement with ddname in its name field.

Use this form of the parameter when a previous job step creates a temporary library to store a program until it is required.

When referring to a DD statement, the system does not honor requests for special program properties as defined in the program properties table (PPT). (See *z/OS MVS Initialization and Tuning Reference*.)

***.stepname.procstepname.ddname**
Refers to a DD statement that defines, as a member of a partitioned data set (PDS) or a partitioned data set extended (PDSE), the program to be executed. The DD statement is in a cataloged or in-stream procedure that is called by an earlier job step. Stepname identifies the EXEC statement of the calling job step; procstepname identifies the EXEC statement of the procedure step that contains the DD statement with ddname in its name field.

Use this form of the parameter when a previous job step calls a procedure that creates a temporary library to store a program until it is required.

When referring to a DD statement, the system does not honor requests for special program properties as defined in the program properties table (PPT). (See *z/OS MVS Initialization and Tuning Reference*.)

JCLTEST (JES3 only)
JSTTEST (JES3 only)
Requests that the system scan the step's job control statements for syntax errors without executing the job or allocating devices. JCLTEST or JSTTEST provides for a step the same function as provided by the JOB statement TYPRUN=SCAN parameter for a job. See *z/OS JES3 Commands* for details.

Note: JCLTEST and JSTTEST are supported only in JES3 systems.

Examples of the PGM parameter

Example 1

```
//JOB8   JOB  ,BOB,MSGLEVEL=(2,0)
//JOBLIB DD   DSNAME=DEPT12.LIB4,DISP=(OLD,PASS)
//STEP1  EXEC PGM=USCAN
```

These statements indicate that the system is to search the private library DEPT12.LIB4 for the member named USCAN, read the member into storage, and execute the member.

Example 2

```
//PROCESS JOB  ,MARY,MSGCLASS=A
//CREATE  EXEC PGM=IEWL
//SYSLMOD DD   DSNAME=&&PARTDS(PROG),UNIT=3390,DISP=(MOD,PASS),
//             SPACE=(1024,(50,20,1))
//GO      EXEC PGM=*.CREATE.SYSLMOD
```

The EXEC statement named GO contains a backward reference to DD statement SYSLMOD, which defines a library created in the step named CREATE. Program PROG is a member of the partitioned data set &&PARTDS, which is a temporary data set. Step GO executes program PROG. The data set &&PARTDS is deleted at the end of the job.

Example 3

```
//JOBC   JOB  ,JOHN,MSGCLASS=H
//STEP2  EXEC PGM=UPDT
//DDA    DD   DSNAME=SYS1.LINKLIB(P40),DISP=OLD
//STEP3  EXEC PGM=*.STEP2.DDA
```

The EXEC statement named STEP3 contains a backward reference to DD statement DDA, which defines system library SYS1.LINKLIB. Program P40 is a member of SYS1.LINKLIB; STEP3 executes program P40.

PROC and procedure name parameters

Parameter type

Positional, optional

Purpose

Use the PROC parameter to specify that the system is to call and execute a cataloged or in-stream procedure.

Syntax

```
{PROC=procedure-name}
{procedure-name     }
```

- The EXEC statement parameter field must begin with a PGM parameter or a PROC parameter. These two parameters must not appear on the same EXEC statement.
- You can omit PROC= and code only the procedure-name.

Subparameter definition

procedure-name

Identifies the procedure to be called and executed:

- The member name or alias of a cataloged procedure.
- The name on the PROC statement that begins an in-stream procedure. The in-stream procedure must appear earlier in this job.

The procedure-name is 1 through 8 alphanumeric or national ($, #, @) characters; the first character must be alphabetic or national.

Effect of PROC parameter on other parameters and following statements

Because this EXEC statement calls a cataloged or in-stream procedure, the other parameters on the statement are added to or override corresponding parameters on the EXEC statements in the called procedure. See the descriptions of the other parameters for details of their effects.

Any DD statements following this EXEC statement are added to the procedure, or override or nullify corresponding DD statements in the procedure.

Examples of the PROC parameter

Example 1

```
//SP3  EXEC PROC=PAYWKRS
```

This statement calls the cataloged or in-stream procedure named PAYWKRS.

Example 2

```
//BK  EXEC OPERATE
```

This statement calls the cataloged or in-stream procedure named OPERATE.

RD parameter

Parameter Type

Keyword, optional

Purpose

Use the RD (restart definition) parameter to:

- Specify that the system is to allow the operator the option of performing automatic step or checkpoint restart if a job step abends with a restartable abend code. (See the SCHEDxx parmlib member description in *z/OS MVS Initialization and Tuning Guide* for information about restartable abends.)
- Allow JES to perform automatic step restart after a system failure even if the journal option is not specified in the JES initialization parameters or JES control statements.
- Suppress, partially or totally, the action of the assembler language CHKPT macro instruction or the DD statement CHKPT parameter.

The system can perform automatic restart only if all of the following are true:

- The JOB or EXEC statement contains RD=R or RD=RNC.
- The step to be restarted abended with a restartable abend code.
- The operator authorizes a restart.

The system can perform automatic step restart for a job running during a system failure as long as the job has a job journal.

A job journal is a sequential data set that contains job-related control blocks needed for restart. If you use the automatic restart manager (ARM) to restart a job, you do not need to save the journal because ARM does not use the job journal when restarting jobs.

For JES2, specify a job journal by one of the following:

- JOURNAL=YES on the CLASS statement in the JES2 initialization parameters.
- RD=R or RD=RNC on either the JOB statement or any one EXEC statement in the job.

For JES3, specify a job journal in one of the following:

- JOURNAL=YES on the CLASS statement in the JES3 initialization parameters.
- RD=R or RD=RNC on either the JOB statement or any one EXEC statement in the job.
- JOURNAL=YES on a JES3 //*MAIN statement in the job.

References

For detailed information on deferred checkpoint restart, see *z/OS DFSMSdfp Checkpoint/Restart*.

Considerations for an APPC scheduling environment

The RD parameter has no function in an APPC scheduling environment. If you code RD, the system will check it for syntax and ignore it.

Syntax

```
RD[.procstepname]=  {R  }
                    {RNC}
                    {NR }
                    {NC }
```

Subparameter definition

R (Restart, Checkpoints Allowed)
Indicates that the operator can perform automatic step restart if the job step fails.

RD=R does not suppress checkpoint restarts:

- If the processing program executed in a job step does not include a CHKPT macro instruction, RD=R allows the system to restart execution at the beginning of the abnormally terminated step.
- If the program includes a CHKPT macro instruction, RD=R allows the system to restart execution at the beginning of the step, if the step abnormally terminates before the CHKPT macro instruction is executed.
- If the step abnormally terminates after the CHKPT macro instruction is executed, only checkpoint restart can occur. If you cancel the affects of the CHKPT macro instruction before the system performs a checkpoint restart, the request for automatic step restart is again in effect.

RNC (Restart, No Checkpoints)
Indicates that the operator can perform automatic step restart if the job step fails.

RD=RNC suppresses automatic and deferred checkpoint restarts. It suppresses:

- Any CHKPT macro instruction in the processing program: That is, the operator cannot perform an automatic checkpoint restart, and the system is not to perform a deferred checkpoint restart if the job is resubmitted.
- The DD statement CHKPT parameter.
- The checkpoint at end-of-volume (EOV) facility.

NR (No Automatic Restart, Checkpoints Allowed)
Indicates that the operator **cannot** perform automatic step restart if the job fails.

RD=NR suppresses automatic checkpoint restart but permits deferred checkpoint restarts. It permits:

- A CHKPT macro instruction to establish a checkpoint.
- The job to be resubmitted for restart at the checkpoint. On the JOB statement when resubmitting the job, specify the checkpoint in the RESTART parameter.

If you code RD=NR and the system fails, RD=NR does not prevent the job from restarting.

NC (No Automatic Restart, No Checkpoints)
Indicates that the operator **cannot** perform automatic step restart if the job step fails.

RD=NC suppresses automatic and deferred checkpoint restarts. It suppresses:

- Any CHKPT macro instruction in the processing program.
- The DD statement CHKPT parameter.
- The checkpoint at EOV facility.

Defaults

If you do not code the RD parameter, the system uses the installation default from the job's job class specified at initialization.

Overrides

- A JOB statement RD parameter applies to all steps of the job and overrides any EXEC statement RD parameters.

  When no RD parameter is on the JOB statement, the system uses an EXEC statement RD parameter, but only during the job step. Code EXEC statement RD parameters when you want to specify different restart types for each job step.
- A request by a CHKPT macro instruction for an automatic checkpoint restart overrides a request by a JOB or EXEC statement RD=R parameter for automatic step restart.

Relationship to other control statements

Code RD=NC or RD=RNC to suppress the action of the DD statement CHKPT parameter.

On an EXEC statement that calls a procedure

If an EXEC statement calls a cataloged or in-stream procedure, the RD parameter is added to or overrides the RD parameter on:

- The EXEC statement named in the procstepname qualifier. The information applies only to the named procedure step. The EXEC statement can have as many RD.procstepname parameters as the procedure has steps; each RD parameter must specify a unique procstepname.
- All EXEC statements in the procedure if procstepname is not coded. Then the parameter applies to all steps in the called procedure.

Examples of the RD parameter

Example 1

```
//STEP1  EXEC PGM=GIIM,RD=R
```

RD=R specifies that the operator can perform automatic step restart if the job step fails.

Example 2

```
//NEST  EXEC PGM=T18,RD=RNC
```

RD=RNC specifies that, if the step fails, the operator can perform automatic step restart. RD=RNC suppresses automatic and deferred checkpoint restarts.

Example 3

```
//CARD  EXEC PGM=WTE,RD=NR
```

RD=NR specifies that the operator cannot perform automatic step restart or automatic checkpoint restart. However, a CHKPT macro instruction can establish checkpoints to be used later for a deferred restart.

Example 4

```
//STP4  EXEC PROC=BILLING,RD.PAID=NC,RD.BILL=NR
```

This statement calls a cataloged or in-stream procedure BILLING. The statement specifies different restart requests for each of the procedure steps: PAID and BILL.

REGION parameter

Parameter type

Keyword, optional

Purpose

Use the REGION parameter to specify the amount of central or virtual storage that the step requires.

The amount of storage that you request must be enough to accommodate the following:

- Storage for all programs in the step to execute.
- All additional storage that the programs in the step request with GETMAIN, STORAGE, and CPOOL macro instructions.
- Enough unallocated storage for task initialization and termination. Task initialization and termination can issue GETMAIN macro instructions for storage in the user's address space.

Two installation exits, IEFUSI and IEALIMIT, can also affect the size of the user address space assigned to the job step.

References

For more information on address space size, see "Resource Control of Address Space" in *z/OS MVS JCL User's Guide*. For more information on region size with checkpoint/restart jobs, see *z/OS DFSMSdfp Checkpoint/Restart*.

Syntax

```
REGION[.procstepname]=  {valueK}
                        {valueM}
```

Subparameter definition

valueK
Specifies the required storage in kilobytes (1 kilobyte = 1024 bytes). The value is 1 through 7 decimal numbers, from 1 through 2096128. Code a multiple of 4. For example, code REGION=68K. If the value you code is not a multiple of 4, the system rounds it up to the next multiple of 4.

valueM
Specifies the required storage in megabytes (1 megabyte = 1024 kilobytes). The value is 1 through 4 decimal numbers, from 1 through 2047. For example, REGION=3M.

value=0M or 0K
A value equal to 0K or 0M gives the step all the storage available below and above 16 megabytes. The resulting size of the region below and above 16 megabytes depends on system options and what system software is installed. When REGION=0K/0M is specified, the MEMLIMIT is set to NOLIMIT.

Note: This might cause storage problems. See the Considerations When Using the REGION parameter section for more information.

Defaults

If no REGION parameter is specified, the system uses an installation default specified at JES initialization.

If your installation does not change the IBM-supplied default limits in the IEALIMIT or IEFUSI exit routine modules, or by using the SMFLIM rules, then specifying various values for the region size has the following results:

- A value equal to 0K or 0M - gives the job step all the storage available below and above 16 megabytes. The resulting size of the region below and above 16 megabytes depends on system options and what system software is installed. When REGION=0K/0M is specified, the MEMLIMIT is set to NOLIMIT.

  Note: This might cause storage problems. See the Considerations When Using the REGION parameter information for more information.
- A value greater than 0K or 0M and less than or equal to 16,384K or 16M - establishes the size of the private area below 16 megabytes. If the region size specified is not available below 16 megabytes, the job step abnormally ends with an ABEND822. The extended region size is the default value of 32 megabytes.
- A value greater than 16,384K or 16M and less than or equal to 32,768K or 32M - gives the job step all the storage available below 16 megabytes. The resulting size of the region below 16 megabytes depends on system options and what system software is installed. The extended region size is the default value of 32 megabytes.
- A value greater than 32,768K or 32M and less than or equal to 2,096,128K or 2047M - gives the job step all the storage available below 16 megabytes. The resulting size of the region below 16 megabytes depends on system options and what system software is installed. The extended region size is the specified value. If the region size specified is not available above 16 megabytes, the job step receives whatever storage is available above 16 megabytes, up to the requested amount, and the resulting size of the region above 16 megabytes depends on system options and what system software is installed.

Overrides

A JOB statement REGION parameter applies to all steps of the job and overrides any EXEC statement REGION parameters.

When no REGION parameter is on the JOB statement, the system uses an EXEC statement REGION parameter, but only during the job step. Code EXEC statement REGION parameters when you want to specify a different region size for each job step.

Relationship to the EXEC ADDRSPC parameter

When ADDRSPC=REAL: Code a REGION parameter to specify how much central storage (also called real storage) the step needs.

When ADDRSPC=VIRT or ADDRSPC is Omitted: Code a REGION parameter to specify how much virtual storage the step needs.

On an EXEC statement that calls a procedure

If an EXEC statement calls a cataloged or in-stream procedure, the REGION parameter is added to or overrides the REGION parameter on:

- The EXEC statement named in the procstepname qualifier. The information applies only to the named procedure step. The EXEC statement can have as many REGION.procstepname parameters as the procedure has steps; each REGION parameter must specify a unique procstepname.
- All EXEC statements in the procedure if procstepname is not coded. Then the parameter applies to all steps in the called procedure.

Relationship to the MEMLIMIT parameter

A specification of REGION=0K/0M will result in a MEMLIMIT value being set to NOLIMIT, when a MEMLIMIT value has not been specified on either the JOB or EXEC statements, and IEFUSI has not been used to set the MEMLIMIT.

Relationship to the REGIONX parameter

REGION and REGIONX are mutually exclusive keywords within the same job. If REGION is specified on the JOB or any EXEC statement in a job, REGIONX must not also be specified on the JOB or any EXEC statement within the same job.

Considerations when using the REGION parameter

Specifying a REGION size that gives the job all of the available storage, such as 0 K or any value greater than 16,384 K, can cause storage problems if the IBM- or installation-supplied routine IEALIMIT, or IEFUSI, or SMFLIMrules is not used to establish a limiting value.

Changes to the REGION parameter might also require changes to associated resource group memory limits. For more information on virtual to real storage relationships, see the *Processor storage management* section in *z/OS MVS Programming: Authorized Assembler Services Guide*.

Examples of the REGION parameter

Example 1:

```
//MKBOYLE EXEC PROC=A,ADDRSPC=REAL,REGION=40K
```

The system assigns 40K bytes of central (real) storage to this job step.

Example 2:

```
//STP6  EXEC PGM=CONT,REGION=120K
```

The system assigns a region of 120K bytes. When the ADDRSPC parameter is not specified, the system defaults to ADDRSPC=VIRT.

REGIONX parameter

Parameter type

Keyword, optional

Purpose

Use the REGIONX parameter to specify the amount of central or virtual storage that the step requires below and above the 16 MB line.

The amount of storage that you request must be enough to accommodate the following:

- Storage for all programs in the step to run.
- All additional storage that the programs in the step request with GETMAIN, STORAGE, and CPOOL macro instructions.
- Enough unallocated storage for task initialization and termination. Task initialization and termination can issue GETMAIN macro instructions for storage in the user's address space.

Syntax

```
REGIONX[.procstepname]=  {value1}
                         {([value1][,value2])}
```

Subparameter definition

value1
Specifies the amount of memory to be assigned below the 16 MB line.

value2
Specifies the amount of memory to be assigned above the 16 MB line, but below 2 GB (that is, “below the bar”.)

Values for REGIONX are defined with *nnnnnM* for megabytes or *nnnnnG* for gigabytes.

The installation might reduce the amount of memory to be assigned through the MEMLIMIT keywords in the SMFPRM*xx* parmlib member, and/or by way of the IEFUSI installation exit or keywords in the SMFLIM*xx* parmlib member.

Defaults

If no REGIONX value is specified on the EXEC statement, but REGIONX is specified on the JOB statement, the JOB REGIONX specification is used for each step in the job. Otherwise, system defined storage settings are used.

If, however, a REGIONX keyword is specified but with null values, the system uses the following rules for each value:

- First value: The system uses the JOB REGIONX first value, the REGION value from the JOBCLASS (if less than 16M) or 0M, in that order. When 0M is used, this implies that all below the line private storage is available to the program.
- Second value: The system uses the JOB REGIONX second value, the REGION value from the JOBCLASS (if greater than 16M) or 128M, in that order. If the program needs access to all available above the line private storage, a value of 0M is to be explicitly coded for the second value.

The installation might reduce these numbers through the MEMLIMIT keyword in the SMFPRM*xx* parmlib member, by way of the IEFUSI installation exit or by keywords in the SMFLIM*xx* parmlib member.

Overrides

Unlike the REGION keyword, a JOB statement REGIONX parameter is used as a default for any step of the job that does NOT have a REGIONX keyword on the EXEC statement. You can set the region default and only use REGIONX as an override on the EXEC statements that require different region values.

JCL procedure overrides:

Because REGIONX is mutually exclusive with REGION,

REGIONX can replace REGION, and
REGION can replace REGIONX

during EXEC statement procedure override processing.

This might result in mutually exclusive conflicts with a REGION= or REGIONX= specification on the JOB statement.

Examine the JCL output listing for uses of REGION and REGIONX.

START command invocation

For a started job, a REGION= or REGIONX= specification on the START command can replace a REGION= or REGIONX= specification on the JOB statement.

For a started procedure, a REGION= or REGIONX= specification on the START command can replace a REGION= specification on the JOB statement that is generated internally by START command processing.

This might result in mutually exclusive conflicts with a REGION= or REGIONX= specification on the EXEC statement of the procedure.

Examine the JCL output listing for uses of REGION and REGIONX with the started job or started procedure.

Relationship to the EXEC ADDRSPC parameter

When ADDRSPC=REAL: Code a REGIONX parameter to specify how much central storage (also called real storage) the step needs.

When ADDRSPC=VIRT or ADDRSPC is Omitted: Code a REGIONX parameter to specify how much virtual storage the step needs.

On an EXEC statement that calls a procedure

The REGIONX keyword is mutually exclusive with the ADDRSPC keyword. If you need to use the ADDRSPC keyword, use REGION instead.

Relationship to the MEMLIMIT parameter

A specification of REGIONX=(0M,0M) results in a MEMLIMIT value being set to NOLIMIT, when a MEMLIMIT value has not been specified on either the JOB or EXEC statements, and IEFUSI and SMFLIM have not been used to set the MEMLIMIT.

Considerations when using the REGIONX parameter

Changes to the REGIONX parameter might also require changes to associated resource group memory limits. For more information on virtual to real storage relationships, see the *Processor storage management* section in *z/OS MVS Programming: Authorized Assembler Services Guide*.

Examples of the REGIONX parameter

```
//REGNX001  JOB MSGLEVEL=1
//STEP001  EXEC PGM=ZTT,REGIONX=(512K,1G)
//ZTTOUT   DD SYSOUT=*,DCB=(LRECL=133,BLKSIZE=133,RECFM=FBA)
//ZTTIN    DD *
/*
```

To request 1 gigabyte above the line and 500 KB below the line for REGIONX values, specify:

```
REGIONX=(500K,1G)
```

RLSTMOUT parameter

Parameter type

Keyword, optional

Purpose

Use the RLSTMOUT parameter to specify the maximum time in seconds that a VSAM RLS or DFSMStvs request is to wait for a required lock before the request is assumed to be in deadlock and ended with VSAM return code 8 and reason code 22(X'16'). Specify RLSTMOUT as a value in seconds in the range of 0 to 9999. A value of 0 means that the VSAM RLS or DFSMStvs request has no timeout value; the request waits as long as necessary to obtain the required lock.

The value does not apply to promote requests. A promote request is a request to change shared access to exclusive access without releasing the lock

VSAM RLS detects deadlocks within VSAM and DFSMStvs. It cannot detect deadlocks across other resource managers, and uses the timeout value to determine when such deadlocks might have occurred.

In addition to specifying RLSTMOUT on the JCL step level, you can also specify a global timeout value in the IGDSMSxx member of SYS1.PARMLIB, or on the RPL passed for each VSAM request. For a particular VSAM RLS or DFSMStvs request, the value used for time out is in the following order:

1. The value specified in the RPL, if any
2. The value specified in JCL at the step level, if any
3. The value specified in the IGDSMS*xx* member of SYS1.PARMLIB, if any

CICS specifies the timeout value in the VSAM RPL. The value used is specified in the CICS System Initialization Table or the transaction definition.

Syntax

```
//[stepname]  EXEC
positional-parm[,RLSTMOUT={nnn|0}]
```

Defaults

If you do not code the RLSTMOUT parameter, the value defaults to the RPL value or to the value specified in PARMLIB.

Examples of the RLSTMOUT parameter

Example 1

```
//STEP04  EXEC PGM=VALKYRIE,RLSTMOUT=0
```

RLSTMOUT specifies that the VSAM RLS or DFSMStvs request has no timeout value. The request waits as long as necessary to obtain the required lock.

TIME parameter

Parameter type

Keyword, optional

Purpose

Use the TIME parameter to specify the maximum amount of time that a job step may use the processor or to find out through messages how much processor time a step used.

You can use the TIME parameter on an EXEC statement to increase or decrease the amount of processor time available to a job step over the default value.

A step that exceeds its allotted time abnormally terminates and causes the job to terminate, unless an installation exit routine extends the time for the job. The exit routine IEFUTL is established through System Management Facilities (SMF).

References

See “TIME parameter” on page 428 (the TIME parameter on the JOB statement) or *z/OS MVS Installation Exits*.

Syntax

```
TIME[.procstepname]=   {([minutes][,seconds])}
                       {1440                 }
                       {NOLIMIT              }
                       {MAXIMUM              }
                       {0                    }
```

You can omit the parentheses if you code only 1440, 0, or the processor time in minutes.

Subparameter definition

minutes
Specifies the maximum number of minutes the step can use the processor. Minutes must be a number from 0 through 357912 (248.55 days).

seconds
Specifies the maximum number of seconds that the step can use the processor, in addition to any minutes that are specified. Seconds must be a number from 0 through 59.

1440 *or* NOLIMIT
Indicates that the step can use the processor for an unlimited amount of time. ("1440" literally means "24 hours.")

Also code TIME=1440 or TIME=NOLIMIT to specify that the system is to allow this step to remain in a continuous wait state for more than the installation time limit, which is established through SMF. "Continuous wait time" is defined as time spent waiting while the application program is in control. For example, the time required to recall a data set from HSM Migration Levels 1 or 2 and/or the time required to mount a tape is counted towards the job's continuous wait time if the allocation of the data set was dynamic (that is, issued while the program was running) while the time required for those activities will not be counted towards the job's continuous wait time if the allocation was static (that is, for a DD statement).

MAXIMUM
Indicates that the step can use the processor for the maximum amount of time. Coding TIME=MAXIMUM allows the step to run for 357912 minutes.

0
Indicates that the step is to use the time remaining from the previous step. If the step exceeds the remaining time available, the step abnormally terminates.

Defaults

Each job step has a time limit. If you do not specify a TIME parameter on the JOB statement, the time limit for any job step is:

- The value you specify for the TIME parameter on its EXEC statement, or
- The default time limit (that is, the JES default job step time limit), if you do not specify a TIME parameter on its EXEC statement.

Overrides

If you specify either MAXIMUM or a value in minutes or seconds other than 1440 for the JOB statement TIME parameter, the system can reduce the processor time available to a job step. In those two cases, the system sets the time limit for the step to the smaller of the two following values:

- The job time remaining after all previous job steps have completed.
- The time limit that was specified or the default time limit.

See “Defaults” on page 340 for an explanation of default time limits.

On an EXEC statement that calls a procedure

If an EXEC statement calls a cataloged or in-stream procedure, the TIME parameter is added to or overrides the TIME parameter on:

- The EXEC statement named in the procstepname qualifier. The information applies only to the named procedure step. The EXEC statement can have as many TIME.procstepname parameters as the procedure has steps; each TIME parameter must specify a unique procstepname.

If procstepname is not coded, the TIME parameter applies to the entire procedure and nullifies any TIME parameters on EXEC statements in the procedure. For example, suppose you specify TIME=5 on an EXEC statement that calls a procedure. The first step in the procedure is allowed 5 minutes, the second step is allowed 5 minutes minus the time used by the first step, the third step is allowed 5 minutes minus the time used by the first and second steps, and so forth, regardless of any TIME parameter values on EXEC statements in the procedure.

TIME=1440 and TIME=NOLIMIT also nullify any TIME parameters on EXEC statements in the procedure. Specifying TIME=1440 or TIME=NOLIMIT on the calling EXEC statement allows the procedure to have unlimited processor time.

Examples of the TIME parameter

For examples of TIME coded on both the JOB and EXEC statements, see "Examples of the TIME parameter on JOB and EXEC statements" on page 430.

Example 1

```
//STEP1  EXEC PGM=GRYS,TIME=(12,10)
```

This statement specifies that the maximum amount of time the step can use the processor is 12 minutes, 10 seconds.

Example 2

```
//FOUR  EXEC PGM=JPLUS,TIME=(,30)
```

This statement specifies that the maximum amount of time the step can use the processor is 30 seconds.

Example 3

```
//INT  EXEC PGM=CALC,TIME=5
```

This statement specifies that the maximum amount of time the step can use the processor is 5 minutes.

Example 4

```
//LONG  EXEC PGM=INVANL,TIME=NOLIMIT
```

This statement specifies that the step can have unlimited use of the processor. Therefore, the step can use the processor and can remain in a wait state for an unspecified period of time, if not restricted by the JOB statement TIME parameter.

Example 5

```
//STP4  EXEC PROC=BILLING,TIME.PAID=(45,30),TIME.BILL=(112,59)
```

This statement calls cataloged or in-stream procedure BILLING. The statement specifies different time limits for each of the procedure steps: PAID and BILL.

Example 6

```
//STP6  EXEC  PGM=TIMECARD,TIME=MAXIMUM
```

This statement specifies that the step can use the processor for 357912 minutes, if not restricted by the JOB statement TIME parameter.

Example 7

```
//TEST1 JOB MSGLEVEL=(1,1)
//STEP1 EXEC PGM=USES40,TIME=(,50)
//STEP2 EXEC PGM=USESREST,TIME=0
```

STEP1 can use the processor for 50 seconds. If STEP1 actually uses the processor for only 40 seconds, STEP2 can use the processor for 10 seconds, because that is the time remaining from the previous step.

Example 8

```
//TEST1 JOB MSGLEVEL=(1,1),TIME=(,50)
//STEP1 EXEC PGM=USES15,TIME=(,25)
//STEP2 EXEC PGM=USES30,TIME=(,40)
//STEP3 EXEC PGM=USESREST,TIME=0
```

STEP1 can use the processor for 25 seconds. If STEP1 actually uses the processor for only 15 seconds, the time limit for STEP2 is the smaller of the following values:

- The job time remaining (35 seconds)
- The time limit specified on the EXEC statement for STEP2 (40 seconds).

In this case, the job time remaining is the smaller value, so STEP2 can use the processor for 35 seconds. If STEP2, then, actually uses the processor for only 30 seconds, STEP3 can use the processor for 5 seconds, because that is the time remaining from the previous step.

Example 9

```
//TEST2 JOB MSGLEVEL=(1,1),TIME=8,CLASS=5
//STEP1 EXEC PGM=USES4
//STEP2 EXEC PGM=USESREST
```

Assume that the default time limit for class 5 is 5 minutes. The time limit for STEP1 is 5 minutes (the default). If STEP1 actually uses the processor for 4 minutes, the time limit for STEP2 is the smaller of the following values:

- The job time remaining (4 minutes)
- The default time limit (5 minutes).

In this case, the job time remaining is the smaller value, so STEP2 can use the processor for 4 minutes.

TVSMSG parameter

Parameter type

Keyword, optional

Purpose

Use the TVSMSG parameter to specify whether Transactional VSAM should issue a message every time a COMMIT or BACKOUT, or both, is performed for a unit of recovery (UR) in the job step where TVSMSG was specified. Usage is most appropriate when testing new applications or when application problems might cause unexpected BACKOUTs.

Syntax

```
TVSMSG= COMMIT|BACKOUT|ALL
```

Subparameter definition

COMMIT
Specifies that Transactional VSAM issues message IGW10121I every time the application in the job step implicitly or explicitly invokes COMMIT.

BACKOUT
Specifies that Transactional VSAM issues message IGW10103I every time the application in the job step implicitly or explicitly invokes BACKOUT.

ALL
Specifies that Transactional VSAM issues message IGW10121I every time the application implicitly or explicitly invokes COMMIT and also issues IGW10103I every time the application implicitly or explicitly invokes BACKOUT.

Defaults

None.

Overrides

None.

Examples of the TVSMSG parameter

Example 1

```
//TVSSTP01 EXEC PGM=VALKYRIE,TVSMSG=BACKOUT
```

Transactional VSAM issues message IGW10103I every time the application implicitly or explicitly invokes BACKOUT.

Example 2

```
//TVSSTP01 EXEC PGM=VALKYRIE,TVSMSG=COMMIT
```

Transactional VSAM issues message IGW10121I every time the application implicitly or explicitly invokes COMMIT.

TVSAMCOM parameter

Parameter type

Keyword, optional

Purpose

Use the TVSAMCOM parameter to specify the number of update requests that must occur before Transactional VSAM issues an automatic commit on behalf of the batch application.

Syntax

```
TVSAMCOM=({minval},{maxval})
```

Subparameter definition

minval
Specifies the minimum number of update requests to complete before Transactional VSAM issues an automatic commit on behalf of the batch application. Acceptable values are numerals between 0 and 99999.

Important: When maxval is not 0, minval must not be 0 and must be less than or equal to the maxval value.

maxval
Specifies the maximum number of update requests to complete before Transactional VSAM issues an automatic commit on behalf of the batch application. Acceptable values are numerals between 0 and 99999.

Note: This value takes effect only if Transactional VSAM does not dynamically adjust the commit frequency to a number lower than the maximum value.

The following table describes the Transactional VSAM behavior for the various values of minval and maxval.

Table 23. Transactional VSAM behavior for the various values of minval and maxval

Minval	Maxval	Transactional VSAM behavior
0	0	The automatic commit feature is disabled. Transactional VSAM does not issue commits on behalf of the application, even if the parameter is specified in the IGDSMSxx member of parmlib.

Table 23. Transactional VSAM behavior for the various values of minval and maxval (continued)

Minval	Maxval	Transactional VSAM behavior
1	0	Transactional VSAM issues a commit point as soon as the Transactional VSAM threshold criteria that is based on lock contention is met. If no critical record lock contention is found, Transactional VSAM does not issue automatic commits.
1	> 1	Transactional VSAM issues a commit point as soon as the Transactional VSAM threshold criteria that is based on lock contention is met. If no critical record lock contention is found and a commit point is not issued before the number of updates equal to the maxval value occur, an automatic commit is performed as soon as the number of updates equals the maxval.
> 1	0	Transactional VSAM issues a commit point when the Transactional VSAM threshold criteria that is based on lock contention is met and the number of processed updates is greater than or equal to the minval value. If critical lock contention is found and minval is not reached, the automatic commit is performed as soon as the number of requests equals the minval value. Since maxval is 0, if no critical record lock contention is found for the unit of recovery, Transactional VSAM does not issue automatic commits.
> 1	> 1 (not = minval)	Transactional VSAM issues a commit point when the Transactional VSAM threshold criteria that is based on lock contention is met and the number of processed updates is greater than or equal to the minval value. If critical lock contention is found and minval is not reached, the automatic commit is performed as soon as the number of requests equals the minval. If no critical record lock contention is found and a commit point is not issued before the number of updates equal to the maxval value occur, an automatic commit is performed as soon as the number of updates equals the maxval.
= maxval	= minval	Transactional VSAM performs automatic commits for the unit of recovery after the number of processed updates is equal to the minval value. Transactional VSAM does not analyze record lock contention for the unit of recovery.

Defaults

If TVSAMCOM is not specified, the default is the value in the IGDSMS*xx* member of SYS1.PARMLIB. If TVSAMCOM is specified without subparameters, the subparameters default to the following values:

- When not specified, *minval* defaults to 1.
- When not specified, *maxval* defaults to 0.
- When *minval* and *maxval* are not specified, the default is the value in IGDSMS*xx*.

Overrides

The value specified in the JCL overrides the value specified in member IGDSMSXX of SYS1.PARMLIB.

See “Defaults” on page 344 for an explanation of default values.

Examples of the TVSAMCOM parameter

Example 1

```
//TVSSTEP01 EXEC PGM=VALKYRIE,TVSAMCOM=(10,100)
```

Transactional VSAM is allowed to dynamically adjust the commit frequency to a number between 10 and 100. If Transactional VSAM does not find a critical record lock contention for the unit of recovery, an automatic commit is issued after 100 record updates are processed.

Chapter 18. IF/THEN/ELSE/ENDIF statement construct

This topic describes the IF/THEN, ELSE, and ENDIF statements, collectively called the IF/THEN/ELSE/ENDIF statement construct.

Purpose: Use the IF/THEN/ELSE/ENDIF statement construct to conditionally execute job steps within a job.

The IF statement is always followed by a relational-expression and a THEN clause. Optionally, an ELSE clause can follow the THEN clause. An ENDIF statement always follows the ELSE clause, if present, or the THEN clause.

- The THEN clause specifies the job steps that the system processes when the evaluation of the relational-expression for the IF statement is a true condition. The system evaluates the relational-expression at execution time.
- The ELSE clause specifies the job steps that the system processes when the evaluation of the relational-expression for the IF statement is a false condition.
- The ENDIF statement indicates the end of the IF/THEN/ELSE/ENDIF statement construct, and must be coded for each construct.

You can nest IF/THEN/ELSE/ENDIF statement constructs up to a maximum of 15 levels. The steps that execute in a THEN clause and an ELSE clause can be another IF/THEN/ELSE/ENDIF statement construct.

Description

Syntax

```
//[name] IF  [(]relational-expression[)] THEN   [comments]
    .
    .    action when relational-expression is true
    .
//[name] ELSE   [comments]
    .
    .    action when relational-expression is false
    .
//[name] ENDIF   [comments]
```

The IF statement consists of the characters // in columns 1 and 2 and the five fields: name, operation (IF), the relational-expression, the characters THEN, and comments. The relational-expression can be enclosed in parentheses.

The ELSE statement consists of the characters // in columns 1 and 2 and the three fields: name, operation (ELSE), and comments.

The ENDIF statement consists of the characters // in columns 1 and 2 and the three fields: name, operation (ENDIF), and comments.

Name field

A name is optional on IF/THEN, ELSE, and ENDIF statements. If used, code it as follows:

- The name should be unique within the job.
- The name must begin in column 3.
- The name is 1 through 8 alphanumeric or national ($, #, @) characters.

© Copyright IBM Corp. 1988, 2017

- The first character must be alphabetic or national ($, #, @).
- The name must be followed by at least one blank.
- The name may be preceded by up to 8 alphanumeric or national characters, and then separated by a period. Coding the name in this way should not be confused with specifying an override, as can be done when coding DD statements.

If a name is not coded, column 3 must be blank.

Operation field

The operation field consists of the characters IF, ELSE, or ENDIF and must be preceded and followed by at least one blank. It can begin in any column.

Relational-expression field

The relational-expression field follows the IF operation field after at least one intervening blank and is followed by at least one blank before the characters THEN. For example, to test that a return code is greater than 4, code:

```
//   IF  RC > 4  THEN
```

You can enclose the relational-expression in parentheses. For example:

```
//   IF  (RC > 4)  THEN
```

A relational-expression indicates the condition that the system evaluates. The result of the evaluation of the relational-expression always depends on two factors: the operation specified, and the values of the operands or expressions that are compared at execution time. The result of evaluating a relational-expression is either true or false.

If you specify a stepname as part of a relational-expression, the system first determines whether the step executed. If the step did not execute, the evaluation of the relational-expression is false.

Continuing a relational expression

You can continue relational-expressions on the next JCL statement. Break the relational-expression where a blank is valid on the current statement, and continue the expression beginning in column 4 through 16 of the next statement. Do not put comments on the statement that you are continuing. You can code comments after you have completed the statement. For example:

```
//TESTCON  IF (RC = 8 | RC = 10 | RC = 12 |
//            RC = 14)  THEN         COMMENTS OK HERE
            .
            .
```

A relational-expression consists of:

- Comparison operators
- Logical operators
- NOT (¬) operators
- Relational-expression keywords
- Numeric values

Priorities of operators

The operators that you can use in a relational-expression and their processing priority are shown in Figure 1 on page 355.

The system evaluates operators in the order indicated. Code operators with the same priority in the order in which you want the system to evaluate them.

You can specify either the alphabetic characters or the special characters for an operator. For example, GT and > have the same meaning. (RC GT 10) and (RC > 10) are the same.

```
                                                Order of
Operator        Operation                       Evaluation
--------        ---------                       ----------

NOT operator:

NOT or  ¬       NOT                             first

Comparison operators:

GT  or  >       Greater than                    second
LT  or  <       Less than                       second
NG  or  ¬>      Not greater than                second
NL  or  ¬<      Not less than                   second
EQ  or  =       Equal to                        second
NE  or  ¬=      Not equal to                    second
GE  or  >=      Greater than or equal to        second
LE  or  <=      Less than or equal to           second

Logical operators:

AND or  &       AND                             third
OR  or  |       OR                              third
```

Figure 1. Operators on IF/THEN/ELSE/ENDIF Statement Construct

Comparison operators

Use comparison operators in a relational-expression to compare a keyword with a numeric value. The comparison results in a true or false condition.

For example, to test for a return code of 8, code:

```
   //TESTA  IF  (RC = 8)  THEN
```

In the example, if a return code is 8, the expression is true; otherwise, the expression is false.

Blanks are not required to precede and follow special character comparison operators (such as > or ¬=). However, it is good practice to use blanks so your code is easier to read. Blanks are required to precede and follow alphabetic comparison operators (such as GT or EQ). Precede and follow the special character & with at least one blank so that it is not confused with symbolic parameters.

Logical operators

Use the & (AND) and | (OR) logical operators in a complex relational-expression to indicate that the Boolean result of two or more relational-expressions is to be evaluated.

You must precede and follow the & (AND) and | (OR) operators with at least one blank.

The & (AND) operator indicates that all of the specified expressions must be true. For example, to test that a return code is both greater than 8 and less than 24 (in the range 9 through 23), code:

```
   //TESTAND  IF  (RC > 8 & RC < 24)  THEN
```

The | (OR) operator specifies that only one of the expressions need be true. For example, to test that a return code is either equal to 8, equal to 10, or greater than 24, code:

```
   //TESTOR  IF  (RC = 8 | RC = 10 | RC > 24)  THEN
```

NOT operator

Use the ¬ (NOT) operator to reverse the testing of relational-expressions.

For example, the statements TESTNOTA and TESTNOTB make the same test. The relational expression is true when the return code is between 0 and 8:

```
//TESTNOTA  IF  ¬(RC > 8)  THEN
//TESTNOTB  IF  (RC <= 8)  THEN
```

The statements TESTNOTC and TESTNOTD make the same test; the relational expression is true when the return code is 0, 1, 2, 3, 4, 5, 6, 7, 9, or 10.

```
//TESTNOTC  IF  ¬(RC = 8 | RC > 10)  THEN
//TESTNOTD  IF  (RC ¬= 8 & RC <= 10)  THEN
```

Note that the use of the ¬ operator reverses both the logical and comparison operators.

You do not need to code a blank between the ¬ operator and the expression it is reversing.

Relational-expression keywords

The following keywords are the only keywords supported by IBM and recommended for use in relational-expressions. Any other keywords, even if accepted by the system, are not intended or supported keywords.

Keyword
Use

RC
indicates a return code

ABEND
indicates an abend condition occurred

¬ABEND
indicates no abend condition occurred

ABENDCC
indicates a system or user completion code

RUN
indicates that the specified step started execution

¬RUN
indicates that the specified step did not start execution

Descriptions of the keywords follow:

RC
Indicates that the relational-expression tests a return code. Evaluate a return code by coding RC, a comparison operator, and a numeric value. For example, the expression (RC = 8) tests for a return code equal to 8, and (RC >= 10) tests for a return code greater than or equal to 10.

The return code must be within the range of 0 - 4095.

If you omit stepname, RC refers to the highest job step return code that occurred during job processing prior to the time of evaluation. This applies only to steps that execute. Any step that did not start execution, is cancelled, or abnormally ends is not evaluated.

Note: At the start of execution, RC is initially set to zero.

stepname.RC
Indicates that the relational-expression tests a return code for a specific step (stepname) of the job.

stepname.procstepname.RC
Indicates that the relational-expression tests a return code for a specific step (stepname) and procedure step (procstepname) of the job.

ABEND
ABEND=TRUE
Indicates that the relational-expression tests for an abend condition that occurred during processing of the job prior to the time of evaluation. The statement IF ABEND THEN tests true when an abend occurred on any previous job step.

If stepname is omitted, ABEND and ABEND=TRUE refer to all previous steps.

Certain types of abnormal termination by the system prevent the execution of the THEN or ELSE clauses of an IF/THEN/ELSE/ENDIF statement construct, regardless of any tests for abnormal termination conditions. See "Errors that prevent execution, regardless of if statement tests" on page 360 for further information.

stepname.ABEND
stepname.ABEND=TRUE
Indicates that the relational-expression tests for an abend that occurred on a specific step (stepname) of the job.

stepname.procstepname.ABEND
stepname.procstepname.ABEND=TRUE
Indicates that the relational-expression tests for an abend that occurred on a specific step (stepname) and procedure step (procstepname) of the job.

¬ABEND
ABEND=FALSE
Indicates that the relational-expression tests that an abend condition did not occur during the processing of the job prior to the time of evaluation. The statement IF ¬ABEND THEN tests true when no abend occurred on any previous job step.

If stepname is omitted, ¬ABEND and ABEND=FALSE refer to all previous steps.

Certain types of abnormal termination by the system prevent the execution of the THEN or ELSE clauses of an IF/THEN/ELSE/ENDIF statement construct, regardless of any tests for abnormal termination conditions. See "Errors that prevent execution, regardless of if statement tests" on page 360 for further information.

¬stepname.ABEND
stepname.ABEND=FALSE
Indicates that the relational-expression tests that no abend occurred on a specific step (stepname) of the job.

¬stepname.procstepname.ABEND
stepname.procstepname.ABEND=FALSE
Indicates that the relational-expression tests that no abend occurred on a specific step (stepname) and procedure step (procstepname) of the job.

ABENDCC=Sxxx
ABENDCC=Uxxxx
Indicates that the relational-expression tests for a system abend completion code (Sxxx) or user-defined abend completion code (Uxxxx). Specify S with a hexadecimal value (3 characters) for system abend codes, and U with a decimal value (4 digits) for user abend codes. For example, ABENDCC=S0C4 tests for system abend code 0C4, and ABENDCC=U0100 tests for user abend code 0100.

If stepname is omitted, ABEND=Sxxx and ABENDCC=Uxxxx refer to the most recent abend code that occurred during the execution of the job prior to the time of evaluation.

Certain types of abnormal termination by the system prevent the execution of the THEN or ELSE clauses of an IF/THEN/ELSE/ENDIF statement construct, regardless of any tests for abnormal termination completion codes. See "Errors that prevent execution, regardless of if statement tests" on page 360 for further information.

stepname.ABENDCC=Sxxx
stepname.ABENDCC=Uxxxx
Indicates that the relational-expression tests the abend code for a specific step (stepname) of the job.

stepname.procstepname.ABENDCC=Sxxx
stepname.procstepname.ABENDCC=Uxxxx
Indicates that the relational-expression tests the abend code for a specific step (stepname) and procedure step (procstepname) of the job.

stepname.RUN
stepname.RUN=TRUE
Indicates that the relational expression tests that a specific job step (stepname) started execution.

stepname.procstepname.RUN
stepname.procstepname.RUN=TRUE
Indicates that the relational expression tests that a specific job step (stepname) and procedure step (procstepname) started execution.

¬stepname.RUN
stepname.RUN=FALSE
Indicates that the relational expression tests that a specific job step (stepname) did not start execution.

¬stepname.procstepname.RUN
stepname.procstepname.RUN=FALSE
Indicates that the relational expression tests that a specific job step (stepname) and procedure step (procstepname) did not start execution.

Specification of step names in relational expression keywords

If you specify stepname.keyword, where keyword is any of the relational expression keywords, stepname must identify a step containing the EXEC PGM keyword rather than one that invokes a procedure. If you specify stepname.procstepname.keyword, procstepname must identify a step containing the PGM keyword. In this case, stepname identifies the EXEC statement that invokes the procedure in which procstepname appears. Note that if stepnames are not unique within the job, such as when the same procedure is executed multiple times, results might be unpredictable; but in most cases, references to non-unique stepnames will resolve to the first occurrence of that stepname.

When you specify a step name as part of a relational expression keyword, the system tests whether the specified step started executing. If the step started executing, the system performs the test indicated by the relational expression. If the step did not start executing, the system evaluates that part of the expression as false.

You must always specify a step name when using the RUN relational-expression keywords to determine if a step or procedure step executed. For more information about step names in relational expression keywords, see *z/OS MVS JCL User's Guide*.

Use of parentheses with relational expressions

The system evaluates relational-expressions that are enclosed within parentheses prior to expressions found outside of parentheses. Therefore, you can control the way in which complex relational-expressions are evaluated.

For example, code the following to test that a return code is 0, 1, 2, or 3:

```
//TESTPAR  IF  (RC LT 4 & (RC LT 12 | RC = 16))  THEN
```

By keeping the same expressions but changing the position of the parentheses, you can test that a return code is 0, 1, 2, 3 or 16:

```
//TESTPAR1  IF  ((RC LT 4 & RC LT 12) | RC = 16)  THEN
```

Comments field

The comments field follows THEN, ELSE, and ENDIF after at least one intervening blank.

Location in the JCL

An IF/THEN/ELSE/ENDIF statement construct can appear anywhere in the job. However, an IF statement specified before the first EXEC statement in a job is not evaluated before the first step executes. If the IF statement applies to later steps in the job, the statement will be evaluated when the system decides whether to process the later steps.

Relationship to other parameters

When you specify both an IF/THEN/ELSE/ENDIF statement construct and a COND parameter for an EXEC statement, the system executes the job step represented by the EXEC statement only when both the IF/THEN/ELSE/ENDIF statement construct and the COND parameter evaluate to execute.

Defaults

By default, job steps within the IF/THEN/ELSE/ENDIF statement construct do not execute when

- An abend occurred, and
- the IF/THEN/ELSE/ENDIF structure containing the job steps does not specify the ABEND, ABENDCC, or ¬ABEND keyword. If any of these keywords is specified (with or without stepname or procstepname), the job steps do execute despite the abend.
- The step's COND parameter, if any, does not specify an abend condition (COND=EVEN or COND=ONLY).

THEN and ELSE clauses

A THEN clause consists of the JCL statements between the IF/THEN statement and, if specified, its matching ELSE statement; otherwise, its matching ENDIF statement. If you do not specify any statements, it is a null THEN clause.

An ELSE clause consists of the JCL statements between the ELSE statement and its matching ENDIF statement. If you do not specify any statements, it is a null ELSE clause.

In an IF/THEN/ELSE/ENDIF statement construct, the THEN clause or the ELSE clause must contain at least one EXEC statement to identify a job step.

The system executes the following statements conditionally, in either the THEN clause or the ELSE clause of an IF/THEN/ELSE/ENDIF statement construct. Execution of the statement depends on the evaluation of the relational-expression at execution time:

- Nested IF/THEN/ELSE/ENDIF statement constructs
- EXEC statements
- DD (including DD * and DD DATA) statements
- STEPLIB DD statements
- SYSABEND, SYSMDUMP, and SYSUDUMP DD statements
- SYSCHK (step level) and SYSCKEOV DD statements
- SYSIN DD statements
- OUTPUT JCL statements
- CNTL and ENDCNTL statements

Do not place the following statements in a THEN or ELSE clause:

- JOB statement
- JCLLIB statement
- JOBLIB statement
- SYSCHK (job level) statement

- XMIT JCL statement

The system processes the following statements regardless of the logic of the IF/THEN/ELSE/ENDIF statement construct. They can be placed in a THEN or ELSE clause, but they are not executed conditionally.

- PROC and PEND statements
- JES2 and JES3 statements and commands
- JCL command statements
- Comment (//*) statements
- INCLUDE statements
- Delimiter (/*) statements
- Null statements
- SET statements

Considerations when using the IF/THEN/ELSE/ENDIF construct

Be aware of the following considerations when using the IF/THEN/ELSE/ENDIF statement construct:

- The IF/THEN/ELSE/ENDIF statement construct does not conditionally control the processing of JCL; rather, it conditionally controls the execution of job steps.
- The result of processing an IF/THEN/ELSE/ENDIF statement construct, once determined, remains unchanged regardless of the outcome from running any remaining steps in a job. The system does not reexamine the original condition at any later job step termination, either normal or abnormal. See Example 9.
- The system allocates all DD statements defined to a step if the execution time evaluation of the relational-expression determines that a step is to be executed.
- All data sets defined on DD statements in the job must be available at the time the job is selected for execution.
- You can nest IF/THEN/ELSE/ENDIF statement constructs up to a maximum of 15 levels.
- You can specify symbolic parameters on IF/THEN/ELSE/ENDIF statements provided that they resolve to one of the supported relational-expression keywords. Any other symbolic parameters, even if accepted by the system, are not intended or supported. Refer to “Relational-expression keywords” on page 356.
- An IF statement specified before the first EXEC statement in a job is not evaluated before the first step executes. If the IF statement applies to later steps in the job, the statement will be evaluated when the system decides whether to process the later steps.
- When you specify an IF statement before the first EXEC statement in a job and the job contains a JOBLIB DD statement, the maximum limit for the number of steps in the job is 254 steps.

There are additional considerations related to errors that prevent execution of the THEN or ELSE clause, no matter what is specified on the IF statement, and there are special considerations related to restarted jobs.

Errors that prevent execution, regardless of if statement tests

Certain error conditions prevent the system from executing the THEN or ELSE clauses of an IF/THEN/ELSE/ENDIF statement construct. When such an error condition occurs, the system does not execute the THEN or ELSE clause, regardless of any tests on the IF statement. These conditions are as follows:

Abnormal termination by the system: After certain types of abnormal termination by the system, remaining job steps are not executed, regardless of any tests for abnormal termination conditions. The completion codes associated with these types of abnormal termination are:

122
Operator canceled job

222
Operator or TSO/E user canceled job

You might encounter other system completion codes for which the THEN or ELSE clause is not executed, regardless of any tests for abnormal termination conditions. See *z/OS MVS System Codes* for further information about specific system completion codes.

When job time expires: The system abnormally terminates processing if a step has exceeded the time limit for the job. The specification of the IF/THEN/ELSE/ENDIF construct has no effect on this type of abnormal termination.

When a referenced data set is not complete: When a job step that contains the IF/THEN/ELSE/ENDIF statement construct references a data set that was to be created or cataloged in a preceding step, the data set

- Will not exist if the step creating it was bypassed, or
- May be incomplete if the step creating it abnormally terminated.

As a result, the system may be unable to execute the step.

When the program does not have control: For the system to act on the IF/THEN/ELSE/ENDIF statement construct, the step must abnormally terminate while the program has control. If a step abnormally terminates during scheduling, (due to failures such as JCL errors or the inability to allocate space), the system bypasses the remaining steps. The steps specified by the IF/THEN/ELSE/ENDIF statement construct do not execute.

Considerations for restarted jobs

There are four types of restarts:

- Automatic step restart
- Automatic checkpoint restart
- Deferred step restart
- Deferred checkpoint restart

Only the automatic restarts retain the information (step completion codes) necessary to perform valid evaluations of any relational expressions based on prior steps.

If you plan to use either type of deferred restart, you should keep certain points in mind when coding the JCL for the job. Planning ahead in this manner can help prevent the need to update the JCL when the job is submitted for restart. The points to consider are the following:

- Relational expressions on IF/THEN statements that refer to a step preceding the restarted step are evaluated as false.
- Relational expressions on IF/THEN statements on steps following the restarted step can still refer to these following steps, but should also check to see whether the referenced steps actually ran during this invocation. The default value for relational expressions on IF/THEN statements is false, which, unlike COND, will cause the system to skip steps. Adding a ¬STEP.RUN condition is recommended. See “Example 7” on page 364 for an example of a statement construct with a deferred checkpoint restart.

Examples of IF/THEN/ELSE/ENDIF statement constructs

Example 1

The following example shows the use of the alphabetic characters rather than special characters for comparison operators.

```
//IFBAD     IF  (ABEND | STEP1.RC > 8) THEN
    or
//IFBAD     IF  (ABEND OR STEP1.RC GT 8) THEN
    .
    .
//IFTEST2   IF  (RC > 4 & RC < 8) THEN
    or
//IFTEST2   IF  (RC GT 4 AND RC LT 8) THEN
```

Example 2

The following example shows a simple IF/THEN/ELSE/ENDIF statement construct without an ELSE statement.

```
//JOBA      JOB   ...
//STEP1     EXEC  PGM=RTN
              .
              .
//IFBAD     IF  (ABEND | STEP1.RC > 8) THEN
//TRUE      EXEC  PROC=ERROR
//IFBADEND  ENDIF
//NEXTSTEP  EXEC  PROC=CONTINUE
```

The IF statement named IFBAD invokes procedure ERROR if either an abend has occurred on a previous step of the job, or STEP1 has returned a return code that is greater than 8. Otherwise, step TRUE is bypassed and the system processes step NEXTSTEP.

Example 3

The following example shows a simple IF/THEN/ELSE/ENDIF statement construct with a null ELSE clause.

```
//JOBB      JOB   ...
//STEP1     EXEC  PGM=RTN
              .
              .
//IFBAD     IF  (ABEND | STEP1.RC > 8) THEN
//TRUE      EXEC  PROC=ERROR
//          ELSE
//IFBADEND  ENDIF
//NEXTSTEP  EXEC  PROC=CONTINUE
```

The IF statement named IFBAD invokes procedure ERROR if either an abend has occurred on a previous step of the job, or STEP1 has returned a return code that is greater than 8. Otherwise, the system bypasses step TRUE, and the null ELSE clause passes to NEXTSTEP.

Example 4

The following example shows a simple IF/THEN/ELSE/ENDIF statement construct with an ELSE clause.

```
//JOBC      JOB    ...
//STEP0     EXEC   PGM=RTN1
              .
              .
//IFTEST2   IF  (RC > 4 & RC < 8) THEN
//*                *** WARNING CONDITION REPORTING GROUP ***
//STEP1     EXEC   PGM=IEFBR14
//REPORT    EXEC   PROC=REPTRTN
//*                *** WARNING CONDITION REPORTING GROUP END ***
//          ELSE
//ERRORSTP  EXEC   PROC=ERRORTN
//ENDTEST2  ENDIF
//NEXTSTEP  EXEC   PROC=CONTINUE
```

Processing for this IF/THEN/ELSE/ENDIF statement construct is:

1. If the relational-expression for the IF/THEN statement construct named IFTEST2 is true (the highest step return code for the job is greater than 4 and less than 8 at the point when this statement is being processed), the system processes the THEN clause. The system executes program IEFBR14 and procedure REPTRTN on EXEC statements STEP1 and REPORT.
2. Otherwise, the relational-expression for IFTEST2 is false and the system processes the ELSE clause (procedure ERRORTN on EXEC statement ERRORSTP).
3. Processing then continues with procedure CONTINUE on step NEXTSTEP.

Example 5

The following example shows nested IF/THEN/ELSE/ENDIF statement constructs with ELSE clauses. The nested statements are indented so that they are easier to read.

```
//JOBD      JOB   ...
//PROC1     PROC
//PSTEPONE  EXEC  PGM=...
//PSTEP11   EXEC  PGM=...
//PSTEP12   EXEC  PGM=...
//          PEND
//PROC2     PROC
//PSTEPTWO  EXEC  PGM=...
//          PEND
//EXP1      EXEC  PROC=PROC1
//EXP2      EXEC  PROC=PROC2
//IFTEST3   IF  (RC > 12) THEN
//STEP1BAD    IF  (EXP1.PSTEP11.RC > 12 OR EXP1.PSTEP12.RC > 12) THEN
//STEP1ERR    EXEC  PGM=ERRTN,PARM=(EXP1)
//            ELSE
//STEP2ERR    EXEC  PGM=ERRTN,PARM=(EXP2)
//END1BAD     ENDIF
//          ELSE
//NOPROB    EXEC  PROC=RUNOK
//ENDTEST3  ENDIF
//NEXTSTEP  EXEC  ...
```

Processing for the IF/THEN/ELSE/ENDIF construct named IFTEST3 is:

1. If the relational-expression for IFTEST3 is true (the highest step return code for the job is greater than 12 at the point where this statement is being processed), the system processes the THEN clause of IFTEST3. It evaluates the relational-expression of the IF/THEN/ELSE/ENDIF construct named STEP1BAD.
2. If the STEP1BAD relational-expression is true (the return code is greater than 12 for either of the two steps in procedure PROC1, which is invoked by step EXP1), the system processes the THEN clause of STEP1BAD. Step STEP1ERR invokes program ERRTN, passing EXP1 as a parameter.
3. If the STEP1BAD relational-expression is not true, the system processes the ELSE clause for STEP1BAD. Step STEP2ERR invokes program ERRTN, passing EXP2 as a parameter.
4. However, if the relational-expression for IFTEST3 is false, the system processes the ELSE clause. Step NOPROB invokes procedure RUNOK.
5. Processing then continues with step NEXTSTEP.

Example 6

The following example shows two IF/THEN/ELSE/ENDIF statement constructs, one of which is nested in the ELSE clause of the other. The nested statements are indented so that they are easier to read.

```
//JOBE      JOB   ...
//PROC1     PROC
//PSTEPONE  EXEC  PGM=...
//          PEND
//PROC2     PROC
//PSTEPTWO  EXEC  PGM=...
//          PEND
//EXP1      EXEC  PROC=PROC1
//EXP2      EXEC  PROC=PROC2
//IFTEST4   IF  (EXP1.PSTEPONE.RC > 4) THEN
//STEP1ERR  EXEC  PGM=PROG1
//          ELSE
//IFTEST5     IF  (EXP2.PSTEPTWO.ABENDCC=U0012) THEN
//STEP2ERR    EXEC  PGM=PROG2
//            ELSE
//NOERR       EXEC  PGM=PROG3
//ENDTEST5    ENDIF
//ENDTEST4  ENDIF
//NEXTSTEP  EXEC  ...
```

Processing for the IF/THEN/ELSE/ENDIF construct named IFTEST4 is:

1. If the relational-expression for IFTEST4 is true (the return code is greater than 4 for PSTEPONE in procedure PROC1, which is invoked by step EXP1), the system processes the THEN clause of IFTEST4. EXEC statement STEP1ERR invokes program PROG1. The system then passes control to ENDIF statement ENDTEST4, and processing continues with step NEXTSTEP.
2. However, if the relational-expression for IFTEST4 is false (the return code is 4 or less for PSTEPONE in procedure PROC1, which is invoked by step EXP1), the system processes the ELSE clause of IFTEST4. It evaluates the IF/THEN/ELSE/ENDIF statement construct IFTEST5.

   Processing for the IF/THEN/ELSE/ENDIF construct named IFTEST5 is:

   a. If the relational-expression for IFTEST5 is true (the user-defined abend completion code is 0012 from PSTEPTWO in procedure PROC2, which is invoked by step EXP2), the system processes the THEN clause of IFTEST5. EXEC statement STEP2ERR invokes program PROG2. The system then passes control to ENDIF statement ENDTEST5, and then ENDTEST4. Processing continues with EXEC statement NEXTSTEP.
   b. However, if the relational-expression for IFTEST5 is false (that is, the user-defined abend completion code is not 0012 from PSTEPTWO in procedure PROC2, which is invoked by step EXP2), the system processes the ELSE clause of IFTEST5. EXEC statement NOERR invokes program PROG3. Processing then continues with step NEXTSTEP.

Example 7

The following example shows an IF/THEN/ELSE/ENDIF statement construct with a deferred checkpoint restart.

```
//DEFER1 JOB RESTART=(STEP2,CHECK004)
//STEP1  EXEC PGM=IEFBR14
//IF1    IF STEP1.RC=0 | ¬STEP1.RUN THEN
//STEP2  EXEC PGM=DEBIT1
//STEP3  EXEC PGM=CREDIT1
//STEP4  EXEC PGM=SUMMARY1
//       ELSE
//STEP5  EXEC PGM=DEBIT2
//STEP6  EXEC PGM=CREDIT2
//STEP7  EXEC PGM=SUMMARY2
//       ENDIF
```

Processing for the IF/THEN/ELSE/ENDIF construct named IF1 is as follows:

1. The conditions on statement IF1 will be checked before executing STEP2.
2. STEP1.RC=0 tests false because STEP1 did not execute and cannot be correctly evaluated.
3. ¬STEP1.RUN tests true; therefore, STEP2, STEP3, and STEP4 will execute and STEP5, STEP6, and STEP7 will not execute.

Note: Without the ¬STEP.RUN condition, STEP2, STEP3, and STEP4 would not execute and STEP5, STEP6, and STEP7 would execute.

Example 8

The following example shows an IF/THEN/ELSE/ENDIF statement construct with a deferred step restart.

```
//DEFER2 JOB RESTART=(STEP3)
//STEP1  EXEC PGM=IEFBR14
//IF1    IF STEP1.RC=0 | ¬STEP1.RUN THEN
//STEP2  EXEC PGM=DEBIT1
//STEP3  EXEC PGM=CREDIT1
//STEP4  EXEC PGM=SUMMARY1
//       ELSE
//STEP5  EXEC PGM=DEBIT2
//STEP6  EXEC PGM=CREDIT2
//STEP7  EXEC PGM=SUMMARY2
//       ENDIF
```

Processing for the IF/THEN/ELSE/ENDIF construct named IF1 is:

1. The conditions on statement IF1 will be checked before executing STEP3.

2. STEP1.RC=0 tests false because STEP1 did not execute and cannot be correctly evaluated.
3. ¬STEP1.RUN tests true; therefore, STEP3 and STEP4 will execute and STEP5, STEP6, and STEP7 will not execute.

Note: Without the ¬STEP1.RUN condition, STEP3 and STEP4 would not run, and STEP5, STEP6, and STEP7 would run.

Example 9

The following example specifies that if STEP1 does not abend, the system is to run STEP2 and STEP3. Otherwise it is to run STEP4.

```
//JOBF     JOB ...
//STEP1    EXEC  PGM=...
//IFTEST6  IF    ¬ABEND THEN
//STEP2    EXEC  PGM=...
//STEP3    EXEC  PGM=...
//         ELSE
//STEP4    EXEC  PGM=...
//         ENDIF
```

The determination of which steps to run is made when the IF/THEN/ELSE/ENDIF statement construct is processed immediately after STEP1 executes. This determination is not subject to change based on the results of running steps after STEP1.

Thus, if STEP1 does not abend, even if STEP2 does, STEP3 (and not STEP4) still runs. If, however, STEP1 does abend, STEP4 is the next step to run, as prescribed by the ELSE clause.